



NOMUTY

(Hand Sign Recognition Application for Deaf-Mute People)

Final Year Project

Submitted by:

Hafiz Muhammad Abdullah Qureshi 70078201

Hafiz Ahmed Usman 70077880

Muhammad Umar Rasheed 70078516

Project Supervisor:

Ma'am Hafsa Naveed

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

**DEPARTMENT OF SOFTWARE ENGINEERING,
UNIVERSITY OF LAHORE**

Date: 01/26/2023

FINAL YEAR PROJECT DOCUMENTATION

STATEMENT OF SUBMISSION

This is to certify that **Hafiz Muhammad Abdullah Qureshi** Roll No. **70078201**, **Hafiz Ahmed Usman** Roll No. **70077880** and **Muhammad Umar Rasheed** Roll No. **70078516** have successfully submitted the Final Year Project Phase 1 documentation named as: “**NOMUTY (Hand Sign Recognition Application for Deaf-Mute People)**”, to the University of Lahore for the award of degree of Bachelors of Science in Software Engineering (BSSE).

By:

- Hafiz Muhammad Abdullah Qureshi 70078201

- Hafiz Ahmed Usman 70077880

- Muhammad Umar Rasheed 70078516

PROJECT SUPERVISOR

Ma'am Hafsa Naveed

(Lecturer, UOL)

Department of SE

Abstract

Around 5% ([researchgate.net](https://www.researchgate.net)) of people across the globe have difficulty in speaking or are unable to speak. So, to overcome this difficulty, sign language came into the picture. It is a method of non-verbal communication which is usually used by mute people. Another problem that arises with sign language is that people without speaking problems do not learn this language. This problem is severe as it creates a barrier between them. To resolve this issue, this project makes use of computer vision and machine learning along with Convolutional Neural Network. The objective of this project is to facilitate communication among mute and other people. For achieving this objective, a system is built to convert hand signs into text. This system will be helpful for mute people as it will increase their communication with other people.

Dedication

We dedicate this project to our parents, our friends, our FYP supervisor and all the generous teachers who taught us so many valuable skills and shared their knowledge with us throughout our degree and all those humble beings who aided us to be able to become what we are today. So many people made a lot of sacrifices which became the seed for our success; especially our parents and teachers who have been there to help us in the weakest and most vulnerable moments of our life. We dedicate this project to all those people who supported us, prayed for us and believed in our abilities.

To our Parents, Teachers and all our humble supporters

Table of Contents

Abstract	iii
Dedication	iv
List of Figures	xiii
List of Tables	xv
Chapter 1: Introduction to the Problem	1
1.1 Introduction:	2
1.2 Purpose:	3
1.3 Objective:	3
1.4 Existing Solution:	3
1.5 Proposed Solution:	4
1.6 Gap Analysis:	4
1.7 Comprehensive Competitive Analysis:	5
1.8 Executive Summary:	5
1.9 References	6
Chapter 2: Software Requirement Specification (SRS)	7
2.1 General Description	8
2.1.1 Product Perspective	8
2.1.2 Product Functions	8
2.1.3 User Characteristics	8
2.1.4 General Requirements	9
2.1.4.1 Software requirements	9
2.1.4.2 Hardware requirements	9
2.1.5 Assumptions and Dependencies	9
2.2 Specific Requirements	9

2.2.1	External Interface Requirements -----	10
2.2.1.1	User Interfaces -----	10
2.2.1.2	Hardware Interfaces -----	11
2.2.1.3	Software Interfaces -----	11
2.2.1.4	Communication Interfaces -----	11
2.2.2	Functional Requirements -----	11
2.2.2.1	Image Capturing -----	11
2.2.2.1.1	ID -----	11
2.2.2.1.2	Introduction -----	12
2.2.2.1.3	Inputs -----	12
2.2.2.1.4	Processing -----	12
2.2.2.1.5	Output -----	12
2.2.2.2	Image pre-processing -----	12
2.2.2.2.1	ID -----	12
2.2.2.2.2	Introduction -----	12
2.2.2.2.3	Inputs -----	13
2.2.2.2.4	Processing -----	13
2.2.2.2.5	Output -----	13
2.2.2.3	Region extraction -----	13
2.2.2.3.1	ID -----	13
2.2.2.3.2	Introduction -----	13
2.2.2.3.3	Inputs -----	13
2.2.2.3.4	Processing -----	14
2.2.2.3.5	Output -----	14
2.2.2.4	Feature extraction -----	14
2.2.2.4.1	ID -----	14
2.2.2.4.2	Introduction -----	14
2.2.2.4.3	Inputs -----	14
2.2.2.4.4	Processing -----	14

2.2.2.4.5	Output-----	15
2.2.2.5	Feature matching -----	15
2.2.2.5.1	ID-----	15
2.2.2.5.2	Introduction -----	15
2.2.2.5.3	Input-----	15
2.2.2.5.4	Processing-----	16
2.2.2.5.5	Output-----	16
2.2.2.6	Pattern recognition -----	16
2.2.2.6.1	ID-----	16
2.2.2.6.2	Introduction -----	16
2.2.2.6.3	Input-----	16
2.2.2.6.4	Processing-----	16
2.2.2.6.5	Output-----	17
2.2.3	Non-Functional Requirements-----	17
2.2.3.1	Performance-----	17
2.2.3.2	Reliability -----	17
2.2.3.3	Availability-----	18
2.2.3.4	Security -----	18
2.2.3.5	Maintainability-----	18
2.2.3.6	Portability -----	18
2.2.4	Inverse Requirements-----	18
2.2.5	Design Constraints -----	18
2.2.5.1	Software Constraints -----	19
2.2.5.2	Hardware Constraints -----	19
2.2.6	Logical Database Requirements-----	19
Chapter 3:	Analysis Use Case Description and Use Case Model -----	20
3.1	Aggregated Use Case Diagram -----	21
3.2	Use Case # 1 -----	22

3.3	Use Case # 2 -----	23
3.4	Use Case # 3 -----	24
3.5	Use Case # 4 -----	25
3.6	Use Case # 5 -----	26
3.7	Use Case # 6 -----	27
Chapter 4: Design -----		29
4.1	Architectural Diagram -----	30
4.2	Entity Relationship Diagram -----	31
4.3	Data Flow Diagram -----	32
4.3.1	DFD Level 0 -----	32
4.3.2	DFD Level 1 -----	33
4.4	Class Diagram -----	34
4.5	Activity Diagram -----	35
4.5.1	Image Capturing -----	35
4.5.2	Image Pre-processing -----	36
4.5.3	Region Extraction -----	37
4.5.4	Feature Extraction -----	38
4.5.5	Feature Matching -----	39
4.5.6	Pattern Recognition -----	40
4.6	Sequence Diagram -----	41
4.6.1	Image Capturing -----	41
4.6.2	Image Pre-processing -----	42
4.6.3	Region Extraction -----	43
4.6.4	Feature Extraction -----	44
4.6.5	Feature Matching -----	45

4.6.6	Pattern Recognition -----	46
4.7	Collaboration Diagram -----	47
4.7.1	Image Capturing-----	47
4.7.2	Image Pre-processing -----	47
4.7.3	Region Extraction -----	48
4.7.4	Feature Extraction-----	48
4.7.5	Feature Matching-----	49
4.7.6	Pattern Recognition -----	49
4.8	State Transition Diagram-----	50
4.8.1	Image Capturing-----	50
4.8.2	Image Pre-processing -----	51
4.8.3	Region Extraction -----	52
4.8.4	Feature Extraction-----	53
4.8.5	Feature Matching-----	54
4.8.6	Pattern Recognition -----	55
4.9	Component Diagram -----	56
4.10	Deployment Diagram -----	57
4.11	Data Dictionary -----	58
4.12	Logical Schema -----	58
Chapter 5:	Testing -----	59
5.1	Test Case Specifications. -----	60
5.1.1	Test Case for Camera Activation-----	60
5.1.2	Test Case for Image Capturing -----	61
5.1.3	Test Case for Image Pre-Processing -----	63
5.1.4	Test Case for Region Extraction -----	66

5.1.5	Test Case for Feature Extraction -----	67
5.1.6	Test Case for Feature Matching -----	70
5.1.7	Test Case for Pattern Recognition-----	72
5.2	Black Box Test Cases-----	75
5.2.1	Equivalence partitions (EP)-----	75
5.2.1.1	Image Capturing -----	75
5.2.1.2	Image Pre-processing -----	75
5.2.1.3	Region Extraction -----	76
5.2.1.4	Feature Extraction -----	76
5.2.1.5	Feature Matching -----	77
5.2.1.6	Pattern Recognition -----	78
5.2.2	Boundary Value Analysis -----	78
5.2.2.1	Image Capturing -----	78
5.2.2.2	Image Pre-processing -----	79
5.2.2.3	Region Extraction -----	80
5.2.2.4	Feature Extraction -----	80
5.2.2.5	Feature Matching -----	81
5.2.2.6	Pattern Matching -----	81
5.2.3	Decision Table Testing -----	82
5.2.3.1	Image Capturing -----	82
5.2.3.2	Image Pre-processing -----	83
5.2.3.3	Region Extraction -----	85
5.2.3.4	Feature Extraction -----	86
5.2.3.5	Feature Matching -----	87
5.2.3.6	Pattern Recognition -----	88
5.2.4	State transition Testing -----	89
5.2.4.1	Image Capturing -----	89
5.2.4.2	Image Preprocessing -----	89

5.2.4.3	Region Extraction	90
5.2.4.4	Feature Extraction	90
5.2.4.5	Feature Matching	91
5.2.4.6	Pattern Recognition	91
5.2.5	Use Case Testing	92
5.2.5.1	Image Capturing	92
5.2.5.2	Image Pre-processing	93
5.2.5.3	Region Extraction	95
5.2.5.4	Feature Extraction	96
5.2.5.5	Feature Matching	97
5.2.5.6	Pattern Recognition	98
5.3	White Box Testing	99
5.3.1	Cyclometric complexity	99
5.3.1.1	Cyclometric Complexity for Image capturing	100
5.3.1.2	Cyclometric Complexity for Image Pre-Processing	101
5.3.1.3	Cyclometric Complexity for Region Extraction	102
5.3.1.4	Cyclometric Complexity for Feature Extraction	103
5.3.1.5	Cyclometric Complexity for Feature Matching	104
5.3.1.6	Cyclometric Complexity for Pattern Recognition	105
5.4	Performance Testing	105
5.5	Stress Testing	106
5.6	System Testing	106
5.7	Regression Testing	106
5.7.1	Selecting Regression Test	109
5.7.2	Regression Testing Steps	109
Chapter 6: Tools and Techniques		111
6.1	Tools Used	112

6.1.1	Python -----	112
6.1.2	Jupyter Notebook -----	113
6.1.3	Android studio-----	114
6.1.4	Open CV -----	115
6.1.5	MediaPipe-----	116
6.1.6	PyCharm -----	117
6.1.7	Tensorflow -----	118
6.1.8	Matplotlib -----	119
6.2	Results -----	121
6.2.1	Classification Report -----	121
6.2.2	Confusion Matrix -----	121
6.2.3	Graph -----	122
Chapter 7: Summary and Conclusion -----		123
7.1	Summary and Conclusion -----	124
7.1.1	Summary -----	124
7.1.2	Conclusion -----	125
Chapter 8: User Manual-----		126
8.1	User Manual -----	127
8.1.1	Title Page -----	127
8.1.2	Introduction -----	128
8.1.3	Main Page: -----	129
Chapter 9: Lessons Learnt and Future Enhancements-----		130
9.1	Lessons Learnt-----	131
9.2	Future Enhancements -----	131
9.3	References -----	132

List of Figures

Figure 1 : Image Capturing.....	22
Figure 2 : Image Preprocessing	23
Figure 3 : Region Extraction	24
Figure 4 : Feature Extraction	25
Figure 5 : Feature Matching	26
Figure 6 : Pattern Recognition.....	27
Figure 7 : Architectural Diagram.....	30
Figure 8 : Entity Relation Diagram.....	31
Figure 9 : DFD Level 0	32
Figure 10 : DFD Level 1	33
Figure 11 : Class Diagram.....	34
Figure 12 : Image Capturing Activity Diagram.....	35
Figure 13 : Image Pre processing Activity Diagram.....	36
Figure 14 : Region Extraction Activity Diagram.....	37
Figure 15 : Feature Extraction Activity Diagram	38
Figure 16 : Feature Matching Activity Diagram	39
Figure 17 : Pattern Recognition Activity Diagram	40
Figure 18 : Image Capturing Sequence Diagram.....	41
Figure 19 : Image Preprocessing Sequence Diagram	42
Figure 20 : Region Extraction Sequence Diagram	43
Figure 21 : Feature Extraction Sequence Diagram	44
Figure 22 : Feature Matching Sequence Diagram	45
Figure 23 : Pattern Recognition Sequence Diagram.....	46
Figure 24 : Image Capturing Collaboration Diagram	47

Figure 25 : Image Preprocessing Collaboration Diagram.....	47
Figure 26 : Region Extraction Collaboration Diagram	48
Figure 27 : Feature Extraction Collaboration Diagram.....	48
Figure 28 : Feature Matching Collaboration Diagram	49
Figure 29 : Pattern Recognition Collaboration Diagram	49
Figure 30 : Image Capturing State Transition Diagram.....	50
Figure 31 : Image Preprocessing State Transition Diagram.....	51
Figure 32 : Region Extraction State Transition Diagram.....	52
Figure 33 : Feature Extraction State Transition Diagram	53
Figure 34 : Feature Matching State Transition Diagram.....	54
Figure 35 : Pattern Recognition State Transition Diagram	55
Figure 36 : Component Diagram	56
Figure 37 : Deployment Diagram	57
Figure 38 : Cyclometric Complexity for Image capturing	100
Figure 39 : Cyclometric Complexity for Image Pre-Processing	101
Figure 40 : Cyclometric Complexity for Region Extraction	102
Figure 41 : Cyclometric Complexity Feature Extraction	103
Figure 42 : Cyclometric Complexity Feature Matching	104
Figure 43 : Cyclometric Complexity Pattern Recognition	105

List of Tables

Table 1 : Image Capturing Use-case Description	22
Table 2 : Image Preprocessing Use-case Description	23
Table 3 : Region Extraction Use-case Description	24
Table 4 : Feature Extraction Use-case Description.....	25
Table 5 : Feature Matching Use-case Description	26
Table 6 : Pattern Recognition Use-case Description	27
Table 7 : Data Dictionary	58
Table 8 : Logical Schema	58
Table 9 : Camera Activation Test Case-Positive	60
Table 10 : Camera Activation Test Case – Negative	60
Table 11 : Image Capturing Test Case – Positive.....	61
Table 12 : Image Capturing Test Case – Negative	62
Table 13 : Image Pre-Processing Test Case – Positive	63
Table 14 : Image Pre-Processing Test Case – Negative.....	64
Table 15 : Region Extraction Test Case – Positive.....	66
Table 16 : Region Extraction Test Case – Negative	67
Table 17 : Test Case for Feature Extraction	67
Table 18 : Feature Extraction Test Case – Negative.....	68
Table 19 : Feature Matching Test Case – Positive	70
Table 20 : Feature Matching Test Case – Negative	71
Table 21 : Pattern Recognition Test Case – Positive	72
Table 22 : Pattern Recognition Test Case – Negative.....	73
Table 23 : Equivalence partition (EP) for Image Capturing.....	75
Table 24 : Equivalence partition (EP) for Image Preprocessing	75

Table 25 : Equivalence partition (EP) for Region Extraction	76
Table 26 : Equivalence partition (EP) for Feature Extraction	76
Table 27 : Equivalence partition (EP) for Feature Matching	77
Table 28 : Equivalence partition (EP) for Pattern Recognition	78
Table 29 : Boundary Value Analysis for Image Capturing	78
Table 30 : Boundary Value Analysis for Image Preprocessing	79
Table 31 : Boundary Value Analysis for Region Extraction	80
Table 32 : Boundary Value Analysis for Feature Extraction	80
Table 33 : Boundary Value Analysis for Feature Matching	81
Table 34 : Boundary Value Analysis for Pattern Matching	81
Table 35 : Decision Table for Image Capturing	83
Table 36 : Decision Table for Image Preprocessing	85
Table 37 : Decision Table for Region Extraction	85
Table 38 : Decision Table for Feature Extraction	86
Table 39 : Decision Table for Feature Matching	87
Table 40 : Decision Table for Pattern Recognition	88
Table 41 : State Transition Testing for Image Capturing	89
Table 42 : State Transition Testing for Image Preprocessing	89
Table 43 : State Transition Testing for Region Extraction	90
Table 44 : State Transition Testing for Feature Extraction	90
Table 45 : State Transition Testing for Feature Matching	91
Table 46 : State Transition Testing for Pattern Matching	91
Table 47 : Testing Use Case for Image Capturing	92
Table 48 : Testing Use Case for Image Preprocessing	93
Table 49 : Testing Use Case for Region Extraction	95

Table 50 : Testing Use Case for Feature Extraction	96
Table 51 : Testing Use Case for Feature Matching	97
Table 52 : Testing Use Case for Pattern Recognition.....	98
Table 53 : Regression Testing Table.....	107

Chapter 1: Introduction to the Problem

Title	Outline
Introduction to the Problem	Introduction
	Purpose
	Objective
	Existing Solution
	Proposed Solution
	Gap Analysis
	Competitive Analysis
	Summary

1.1 Introduction:

The language for deaf and mute people is sign language. This language is composed of conventional gestures, hand signs, finger spellings, and hand gestures that represent the letters of the alphabet. Sign language dates back to the 17th century ([Wikipedia](#)) when it was known as a visual language. Sign languages are different in different countries, like America, American Sign Language, India its Indian Sign Language, and likewise in other countries. Sign language as defined earlier is primarily composed of gestures. The gesture is a form of nonverbal communication in which visible actions of the hand, face, and other body parts convey a particular message. Manual gestures led to the evolution of language and this theory is known as Gestural Theory. This theory came in the 18th century and was the work of Abbé de Condillac. Broca's and Wernicke's areas of the brain are used for gesture processing. With technology, gestures can be recognized by the computer and are termed gesture recognition. It is a well-known topic in computer science as well as language technology. Interpretation of human gestures by the use of mathematical algorithms is the goal of gesture recognition. The use of camera and computer vision algorithms for the interpretation of sign language is one of the many advances made in this field. Techniques from computer vision and image processing can also be used for gesture recognition. The utilization of computer algorithms for managing and extracting useful information from digital images is called Image Processing. Image processing is modeled in the form of multi-dimensional systems. The use of computer algorithms gives image processing an advantage over analog means as it avails the implementation of those methods that provide sophisticated performance at various tasks. Image processing is used for projection, multiscale signal analysis, feature extraction, and classification. Various techniques are used by image processing, a few of which are Principal Component Analysis (PCA), Pixelation, Independent Component Analysis (ICA), Linear Filtering, Partial Differential Equations (PDE), Neural Networks, Image editing, Image restoration, Hidden Markov Models. To make computers gain understanding from videos and digital images is altogether a separate field, and the ways to do this job are dealt with by Computer Vision (CV). Computer vision is an interdisciplinary field that automates the tasks that humans do visually. Computer vision provides decisions by producing symbolic and numeric information from the real world. This generation of information is made possible by extraction of high dimensional features which is further made possible by acquiring the digital images, then processing those images that further led to an understanding of those images.

Using models constructed by aids of physics, geometry, statistics, and learning theory, the understanding of images is seen as the disentangled form of symbolic information that can be extracted from digital images. This paper is divided into 6 sections. The second section is all about the related works in this field. The third section is about other hand gesture recognition techniques that can be used. The fourth section is some basic concepts that are to be known before moving to the proposed system architecture. The fifth section is about how the system proposed by this paper works along with the results of the recognition of various words. The last section is about the conclusion and future work that would be accomplished with help of the proposed paper.

1.2 Purpose:

Deaf is a disability that impairs their hearing and makes them unable to hear, while mute is a disability that impairs their speaking and makes them unable to speak. Both are only disabled in their hearing and/or speaking, therefore can still do many other things. The only thing that separates them and the normal people is communication. If there is a way for normal people and deaf-mute people to communicate, deaf-mute people can easily live like normal person. And the only way for them to communicate is through sign language. This system is built for the betterment of deaf and mute people.

1.3 Objective:

The following objectives are supposed to be achieved by the end of this project:

- Image Acquire
- Image processing
- Feature extraction
- Image classifier
- Image recognition
- Conversion to text

1.4 Existing Solution:

For the detection of hand gestures, there are several types of hardware and sensors available. These approaches are neither accurate nor appropriate for the task. The system is difficult to

operate due to the utilization of several sensors and hardware. In addition, systems have relied on stereo cameras, which are more expensive and consume more resources. The current software for hand recognition is extremely sluggish and does not generate accurate results. This is a significant flaw in the existing system. Many consumers are having issues with the software they are utilizing.

1.5 Proposed Solution:

Many different competitors are working on smart technology but almost all of them are working simply on motion detection and gesture detection. This project is more about the combined approach to gesture and sign recognition.

1.6 Gap Analysis:

For deaf and dumb persons, we suggest converting hand gestures into text in this project. The main goal of our project is to recognize hand gestures, detect gestures, and display the results as text. In front of the camera, the end user must make hand motions. Our application will identify the motions as they are made by the user and will convert them into text in real-time. The video obtained from the camera unit will be presented on the screen, and inside that video, alongside the hand, the required output will be displayed. Our project serves as a deaf and dumb translator. It solves several issues, including the necessity for human translation. Our program will allow deaf and dumb individuals to express themselves. We'll use the camera to identify hand motions. To use a camera to detect these motions, we must first isolate the hand region, deleting any undesired sections from the video sequence collected by the camera. We count the fingers visible to the camera after segmenting the hand region to direct a program based on the finger count. As a result, the entire problem can be handled in five easy steps: We must first locate and segment the hand region in the video stream. Then, from the segmented hand region in the video sequence, count the number of fingers and the size of the palm. The segmented section will then be matched to the available dataset. Then, for quicker data selection, choose the most accurate data from the dataset and apply weight for the next comparison. Finally, we'll translate the data we've gathered into text and display it. In front of the camera, the end user must make hand motions. Our application will identify the motions as they are made by the user and will convert them into text in real-time.

1.7 Comprehensive Competitive Analysis:

The competitors we have:

- **Microsoft Kinect:** Kinect is a line of motion-sensing input devices produced by Microsoft and first released in 2010. The devices generally contain RGB cameras, infrared projectors, and detectors that map depth through either structured light or time of flight calculations, which can in turn be used to perform real-time gesture recognition and body skeletal detection, among other capabilities. They also contain microphones that can be used for speech recognition and voice control ([Wikipedia](#)).
- **Glove Based Sign Language Translator:** The system includes a pair of gloves with thin, stretchable sensors that run the length of each of the five fingers. These sensors, made from electrically conducting yarns, pick up hand motions and finger placements that stand for individual letters, numbers, words, and phrases ([newsroom](#)).
- **Decision Tree Classifier:** A decision tree classifier was designed for recognizing hand gestures. We had two groups of gesture sets: ASL Gestures Set and General Gestures Set. According to the experiment, the average accuracy of the ASL number gesture set was 94.33%, and that of the general gestures set was 95.01% ([Researchgate](#)).

1.8 Executive Summary:

The application can be used to recognize sign language gestures made for deaf-mute people. The systems described here use the back camera of our phone and processes like region extraction and pattern recognition which are built using different APIs to track the movement of the hands and fingers. These systems then translate the gestures into text or audio output, which can be used to communicate with hearing people. Some of the benefits of using these systems include the ability to communicate with a wider range of people, increased independence, and improved access to information. Additionally, these systems can be used in a variety of settings, including educational, workplace, and social settings.

1.9 References

[1] Shubham Kr. Mishra, Sheona Sinha, Sourabh Sinha, and Saurabh Bilgaiyan. Recognition of Hand Gestures and Conversion of Voice for Betterment of Deaf and Mute People.

[2] Chen, F., Fu, C., Huang, C.: Hand gesture recognition using a real-time tracking method and hidden Markov models. *Image Vis. Comput.* 21, 745–758 (2003)

[3] Coa, Z., Wei, Z., Zhang, G.: A no-reference sharpness metric based on the notion of relative blur for Gaussian blurred images. *J. Vis. Commun. Image Represent.* 25(7), 1763–1773 (2014)

[4] Shinde, S.S., Autee, R.M., Bhosale, V.K.: Real-time two-way communication approach for hearing impaired and dumb person based on image processing. In: *IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, pp. 1–5. IEEE (2016)

[5] Shi, D., Zhu, L., Cheng, Z., Li, Z., Zhang, H.: Unsupervised multi-view feature extraction with dynamic graph learning. *J. Vis. Commun. Image Represent.* 56, 256–264 (2018)

Chapter 2: Software Requirement Specification (SRS)

Title	Outline
Software Requirement Specification (SRS)	General Description
	Specific Requirements

2.1 General Description

This “Sign Language Recognition Application for deaf-mute people” is intended to provide AI-based services to the client which are but not limited to Sign recognition (including detection as well as conversion to text or voice). The smart camera will be working on AI-based models to be able to recognize hand signs happening in the sight of a camera. The perceiving site depends upon the camera type being used.

2.1.1 Product Perspective

The “Sign Language Recognition Application for deaf-mute people” is an AI-based system. The system will be developed using different AI-based models and python programming language. Different hardware equipment will also be required but it will be explained in its section further in this document. This system is a revolutionary system that can be upgraded as time goes on because of its immense potential. The system potential has already been discussed in Section 1.6 “Gap Analysis”. This system is being developed to assist deaf-mute people so that communication with them would be easier.

2.1.2 Product Functions

This AI-based system is supposed to perform the following functions

1. Image capturing
2. Image pre-processing
3. Region extraction
4. Feature extraction
5. Feature matching
6. Pattern recognition
7. Conversion of pattern to the text

2.1.3 User Characteristics

A deaf-mute person would be placed in front of the camera and would perform hand signs. The user would use the application that would capture the hand signs being performed. The system would perform its functions independently and flawlessly and the signs would be

converted to text. The user would read the text and then could know what the deaf-mute person is trying to communicate.

2.1.4 General Requirements

This system involves two types of general requirements that are mentioned as follows:

2.1.4.1 Software requirements

- Python
- Sublime text
- Jupyter Notebook
- TensorFlow
- OpenCV
- R-CNN
- Android Studio

2.1.4.2 Hardware requirements

- Smartphone
- Smartphone camera

2.1.5 Assumptions and Dependencies

It is assumed that the system will be lightweight so that it could be deployed on as many smartphones as possible. The camera of the smartphone should be of high quality and high MP and is supposed to work on AI models so that the capturing and scans are as precise as possible. The system isn't supposed to be able to work independently and human assistance is necessary. Different state-of-the-art AI-based models will be used to be able to achieve the desired results. So, the system will be dependent on Artificial intelligence technology with human interference.

2.2 Specific Requirements

- External Interface Requirements
- Functional Requirements

2.2.1 External Interface Requirements

- User Interfaces
- Hardware Interfaces
- Software Interfaces
- Communication Interfaces

2.2.1.1 User Interfaces

There is user interference in the system to run the application, perform signs, and read the result. The logical characteristics and GUI standards of the user interface that the system will follow are briefed as follows:

1. **Consistency of the Design:** This means that different parts of the application should use the same elements so that when a user is familiar with one part of the user interface, they can easily interact with other parts using the initial understanding.
2. **Clarity of Layout and Design:** Images should be of good quality and properly spaced. This can easily be achieved by understanding the use of white space. The size of different elements should also enable easy viewership. Size can additionally be used to emphasize the importance of various aspects.
3. **Familiarity:** This means that even if a user is new to the application, they can easily recall familiar elements and be able to navigate with ease.
4. **Aesthetics:** This means the proper selection of colors, layout, and graphic designs that appeal to the user. Creativity in this area separates the best from the average.
5. **Error Recovery and Prevention:** The best way to deal with errors is to avoid them from the beginning. It is essential to analyze the application to understand activities that can introduce potential errors and eliminate them if possible. If you can't remove them, find a potential flagging mechanism that will allow the designer to detect these errors when they appear.
6. **Ease of Access:** It is never a good thing to have an application that continually requires the user to request help because this means that the user interface design has failed.

2.2.1.2 Hardware Interfaces

The application would ask for camera access and once the access has been provided, the camera would capture the images of hand signs and the application would start its functionality. Image pre-processing, region extraction, feature extraction, feature matching, and pattern recognition algorithms would be performed and the signs would be converted to text. The user would be able to understand what the deaf-mute person is trying to say.

2.2.1.3 Software Interfaces

This is an AI-based system that will be working on many different models for hand sign recognition. Only state-of-the-art AI models have been chosen for this system so that the results are as precise as possible. The system will not be interacting with other or one may say “outside” components. All the functionalities and components are embedded within the application. There is no other collaboration in this system.

2.2.1.4 Communication Interfaces

The AI-based system will be functioning on its own since it is an AI-based system and will not be connected to any network. The users will use an application developed specifically for displaying the result.

2.2.2 Functional Requirements

- Image capturing
- Image pre-processing
- Region extraction
- Feature extraction
- Feature matching
- Pattern recognition

2.2.2.1 Image Capturing

Obtaining a digital image from a vision sensor, such as a camera.

2.2.2.1.1 ID

This functional requirement's ID is FR-1.

2.2.2.1.2 Introduction

The process of obtaining a digital image from a vision sensor, such as a camera. Usually, this entails a hardware interface known as a frame grabber, which captures single frames of video, converts the analog values to digital, and feeds the result into the computer memory.

2.2.2.1.3 Inputs

Since this whole system is based on an optical instrument being used to capture the frames, the only possible input can be the frames perceived by the camera. In simpler words, the “Incoming Footage” will be the input for this functionality to be performed.

2.2.2.1.4 Processing

The optical device (camera) captures the frames from the perceiving site and stores them in the computer memory for further processing.

2.2.2.1.5 Output

The output of this function is to store the captured image in the memory for further processing.

2.2.2.2 Image pre-processing

The term for operations on images at the lowest level of abstraction.

2.2.2.2.1 ID

This functional requirement’s ID is FR-2.

2.2.2.2.2 Introduction

Image pre-processing is the term for operations on images at the lowest level of abstraction. These operations do not increase image information content but they decrease it if entropy is an information measure. The aim of pre-processing is an improvement of the image data that suppresses undesired distortions or enhances some image features relevant for further processing and analysis tasks.

2.2.2.2.3 Inputs

Since this whole system is based on an optical instrument being used to capture the frames, the only possible input can be the frames perceived by the camera and stored in the memory. In simpler words, the “stored image in memory” will be the input for this functionality to be performed.

2.2.2.2.4 Processing

In image pre-processing, different algorithms like smoothing, and Gaussian Blur would be applied to the captured image. By use of OpenCV, the input sequence of RGB images gets converted to gray images.

2.2.2.2.5 Output

The image is enhanced and good results would be obtained.

2.2.2.3 Region extraction

Region extraction is the process of separating the object (foreground) from the background considering certain features of the image such as color, intensity, texture, etc.

2.2.2.3.1 ID

This functional requirement's ID is FR-3

2.2.2.3.2 Introduction

Region extraction is an important pre-processing step for all computer vision and image understanding tasks. Region extraction is the process of separating the object (foreground) from the background considering certain features of the image such as color, intensity, texture, etc.

2.2.2.3.3 Inputs

Since this whole system is based on an optical instrument being used to capture the frames, the only possible input can be the frames perceived by the camera and stored in the memory. In simpler words, the “stored image in memory” will be the input for this functionality to be performed.

2.2.2.3.4 Processing

In this functionality, the hand feature or hand objects in an image would be separated from its background by applying Background Segmentation.

2.2.2.3.5 Output

The image is enhanced and good results would be obtained.

2.2.2.4 Feature extraction

Feature extraction is a part of the dimensional reduction process, in which, an initial set of raw data is divided and reduced into more manageable groups.

2.2.2.4.1 ID

This functional requirement's ID is FR-4

2.2.2.4.2 Introduction

Feature extraction is a part of the dimensional reduction process, in which, an initial set of raw data is divided and reduced into more manageable groups. So, when you want to process it will be easier. The most important characteristic of these large data sets is that they have a large number of variables. These variables require a lot of computing resources to process. So, Feature extraction helps to get the best feature from those big data sets by selecting and combining variables into features, thus, effectively reducing the amount of data. These features are easy to process, but still able to describe the actual data set with accuracy and originality.

2.2.2.4.3 Inputs

Since this whole system is based on an optical instrument being used to capture the frames, the only possible input can be the frames perceived by the camera and stored in the memory. In simpler words, the “stored image in memory” will be the input for this functionality to be performed.

2.2.2.4.4 Processing

For feature extraction different features like binary area, centroid, peak calculation, angle calculation, thumb detection, finger region, or edge detection of the hand region are calculated.

From the initial set, data is processed and hence builds derived values for the work of Feature extraction. Feature extraction is intended to be non-redundant and informative. It facilitates sequential learning with generalization and in some cases leads to better human interpretations. The raw and initial set of variables is reduced to groups (features) for processing that precisely and describe the original set.

2.2.2.4.5 Output

The image is enhanced and good results would be obtained.

2.2.2.5 Feature matching

Feature matching refers to finding corresponding features from two similar images based on a search distance algorithm.

2.2.2.5.1 ID

This functional requirement's ID is FR-5

2.2.2.5.2 Introduction

Feature matching refers to finding corresponding features from two similar images based on a search distance algorithm. One of the images is considered the source and the other is a target, and the feature matching technique is used to either find or derive and transfer attributes from the source to the target image. The feature-matching process generally analyses the source and target's image topology, detects the feature patterns, matches the patterns, and matches the features within the discovered patterns. The accuracy of feature matching depends on image similarity, complexity, and quality. Normally, a high percentage of successful matching can be achieved using the correct method, while uncertainty and errors may occur and would require post-inspection and corrections.

2.2.2.5.3 Input

Since this whole system is based on an optical instrument being used to capture the frames, the only possible input can be the frames perceived by the camera and stored in the memory. In simpler words, the "stored image in memory" will be the input for this functionality to be performed.

2.2.2.5.4 Processing

A classification system database is very important because that contains predefined sample patterns of the object under consideration that compare with the test object to classify its appropriate class. In a typical classification, the system image is captured by a camera and consequently processed. The neural network is one of the classification algorithms.

2.2.2.5.5 Output

The image will be matched with the images stored in our Database and results would be obtained.

2.2.2.6 Pattern recognition

Pattern recognition is a data analysis method that uses machine learning algorithms to automatically recognize patterns and regularities in data.

2.2.2.6.1 ID

This functional requirement's ID is FR-5

2.2.2.6.2 Introduction

Pattern recognition is a data analysis method that uses machine learning algorithms to automatically recognize patterns and regularities in data. This data can be anything from text and images to sounds or other definable qualities. Pattern recognition systems can recognize familiar patterns quickly and accurately. They can also recognize and classify unfamiliar objects, recognize shapes and objects from different angles, and identify patterns and objects even if they're partially obscured.

2.2.2.6.3 Input

The processed image on which all the algorithms are applied is the input of this section.

2.2.2.6.4 Processing

Making use of mathematical algorithms, intending to interpret human gestures is a topic of Gesture Recognition in the field of language technology and computer science. Mobile and computers are input devices that read input of gestures of the hand. Gestures are taken as input

when the computer reads the movements of the body and sends them to the computer. The input is then interpreted by making use of artificial intelligence. The recognized word is converted to voice by making use of the Google audio built-in function.

2.2.2.6.5 Output

Hand signs would be converted to text/audio.

2.2.3 Non-Functional Requirements

- Performance
- Reliability
- Availability
- Security
- Maintainability
- Portability

2.2.3.1 Performance

Performance requirements are listed below:

- The system should be based on a lightweight hand sign recognition model.
- The system should be able to recognize multiple hand signs at a time.
- The system should be able to return the confidence value of detected signs.
- The system should be able to recognize hand signs in the minimum possible time.
- The system should be latency-free.

2.2.3.2 Reliability

Reliability requirements are listed below:

- The system should not generate false alerts.
- The system should be able to restart itself in case of any crash.
- The application should be error-free and bug-free as well.
- The application camera should be latency-free.
- The application should have a report section for users to report any issue in the application.

- The application should be in an international language i.e., English.

2.2.3.3 Availability

Availability requirements are listed below:

- The availability of the system should be 24/7.
- Users should be able to use the application at any time.

2.2.3.4 Security

Security requirements are listed below:

- The application should not have any public access points.
- The application should be secure from unauthorized access.
- The application should be secure from malware attacks.

2.2.3.5 Maintainability

Maintainability requirements are listed below:

- The application should be highly cohesive and low-coupled for ease of maintenance.
- Errors and bugs in the application should be removed in upcoming updates.

2.2.3.6 Portability

Portability requirements are listed below:

- The application should be developed using cross-platform app development.
- The system should be developed in Python language.

2.2.4 Inverse Requirements

- The application should not disclose users' credentials.
- System working should not take more than 20 seconds.

2.2.5 Design Constraints

- Software constraints

- Hardware constraints

2.2.5.1 Software Constraints

- The model should be developed in PyCharm IDE.
- TensorFlow python library should be used for fast numerical computing.
- Keras python library should be used for developing and evaluating deep learning models.
- The used model should be lightweight.
- The model should be efficient enough.

2.2.5.2 Hardware Constraints

- A High-resolution Smartphone camera should be used.
- The megapixels of Smart phone cameras should be high for better results.

2.2.6 Logical Database Requirements

MySQL would be used as a Database repository for this system. MySQL has different storage engines, but the one used in this system is InnoDB. InnoDB is a general-purpose storage engine with high performance and reliability with features that will enhance the integrity of the database. It would also use the tool phpMyAdmin which is a software written in PHP that intends to handle MySQL database administration via the Web. It supports most MySQL features to work with databases, allows to export and import of data in many formats, and creates graphics of the database.

Chapter 3: Analysis | Use Case Description and Use Case Model |

Title	Outline
Analysis	Aggregated Use Case Diagram
	Requirements' Use Cases

3.1 Aggregated Use Case Diagram

The system-based use-case diagram is drawn as follows:

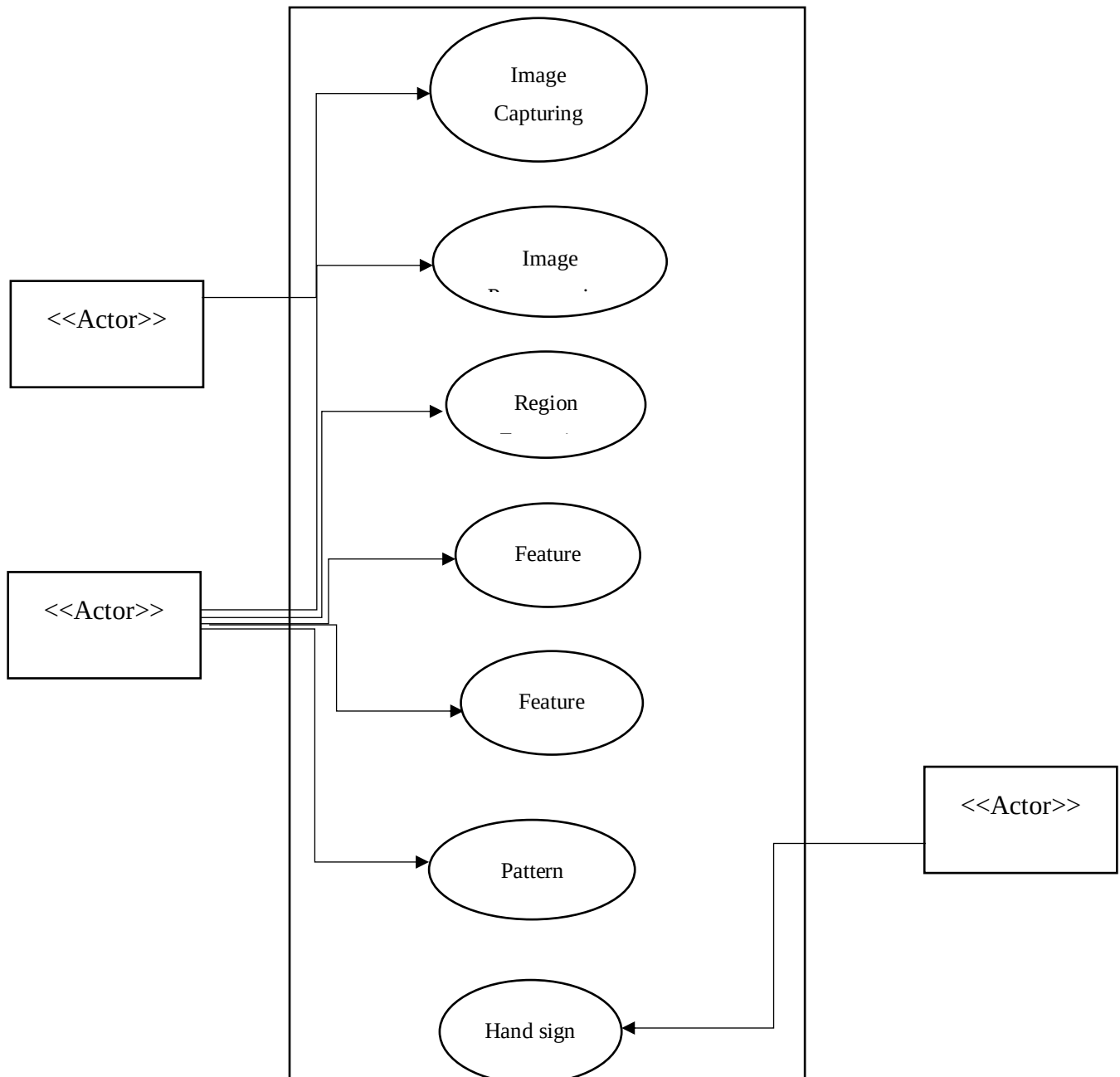


Figure 1: Aggregated Use Case Diagram

3.2 Use Case # 1

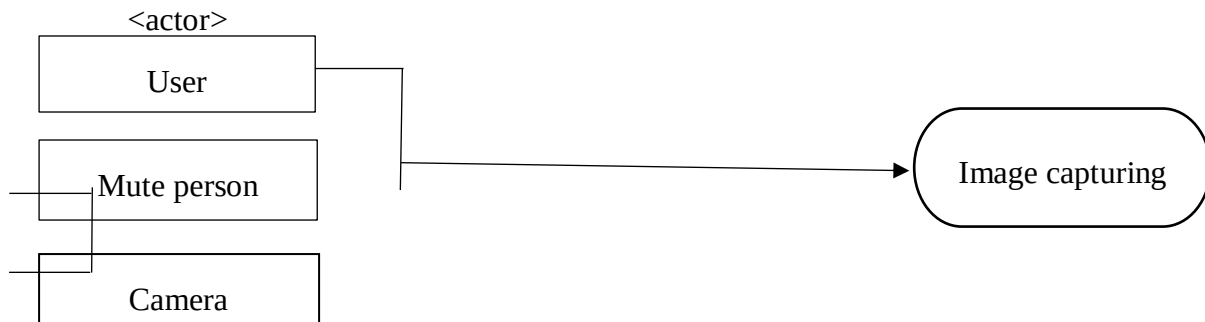


Figure 2: Image Capturing

Table 1: Image Capturing Use-case Description

Use Case ID	FR-1	
Use Case Name	Image Capturing	
Date Created:	December 11,2022	
Actor	User, Deaf -Mute person & Camera	
Description	The Optical device (camera) captures the frames from the perceiving site of the deaf-mute person and save them into memory for processing.	
Preconditions	1. The camera should be active 2. The frame should be clear	
Post conditions	Your model should be trained with enough data.	
Normal flow:		
	User	Deaf-Mute person
	1. Capture the image of the deaf-mute person	
		2. Show the gesture
		3. Show his/her hand sign

Alternative flows	1. If the image is not captured properly. 2. Then the further process is difficult to proceed with.	

3.3 Use Case # 2

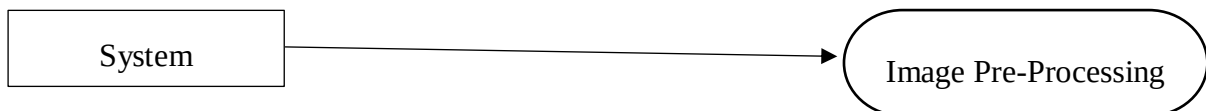


Figure 3: Image Pre-processing

Table 2: Image Pre-processing Use-case Description

Use Case ID	FR-2		
Use Case Name:	Image Pre-processing		
Date Created:	December 11,2022		
Actors	System		
Description	In image pre-processing, different algorithms like smoothing, and Gaussian Blur would be applied to the captured image. By use of OpenCV, the input sequence of RGB images gets converted to gray images.		
Preconditions	All the Algorithms work properly		
Post Condition	The image is enhanced and good results would be obtained.		
Normal Flow	<table><tr><td>System</td></tr><tr><td>1. Convert the image into greyscale using gaussian blur.</td></tr></table>	System	1. Convert the image into greyscale using gaussian blur.
System			
1. Convert the image into greyscale using gaussian blur.			
Alternative flaws	1. If the algorithm is not perfect to work properly. 2. Then the enhanced and good results would not be obtained.		

3.4 Use Case # 3



Figure 4: Region Extraction

Table 3: Region Extraction Use-case Description

Use Case ID	FR-3		
Use Case Name	Region Extraction		
Date Created	December 11,2022		
Actor	System		
Description	In Region Extraction In this functionality, the hand feature or hand objects in an image would be separated from its background by applying Background Segmentation.		
Preconditions	All the algorithms work properly.		
Post Conditions	The image is enhanced and good results would be obtained.		
Normal Flow	<table><tr><td>System</td></tr><tr><td> 1. The system separated the hand image. 2. The system removes background by applying background segmentation. </td></tr></table>	System	1. The system separated the hand image. 2. The system removes background by applying background segmentation.
System			
1. The system separated the hand image. 2. The system removes background by applying background segmentation.			

3.5 Use Case # 4



Figure 5: Feature Extraction

Table 4: Feature Extraction Use-case Description

Use Case ID	FR-4		
Use Case Name	Feature Extraction		
Date Created	December 11,2022		
Actor	System		
Description	For feature extraction different features like binary area, centroid, peak calculation, angle calculation, thumb detection, finger region, or edge detection of the hand region are calculated. From the initial set, data is processed and hence builds derived values for the work of Feature extraction. Feature extraction is intended to be non-redundant and informative. It facilitates sequential learning with generalization and in some cases leads to better human interpretations. The raw and initial set of variables is reduced to groups (features) for processing that precisely describe the original set.		
Precondition	All the algorithms work properly.		
Postcondition	R-CNN algorithm use to extract feature		
Normal Flow	<table><tr><td>System</td></tr><tr><td>1. The system extract feature using R-CNN</td></tr></table>	System	1. The system extract feature using R-CNN
System			
1. The system extract feature using R-CNN			

3.6 Use Case # 5



Figure 6: Feature Matching

Table 5: Feature Matching Use-case Description

Use Case ID	FR-5		
Use Case Name	Feature Matching		
Date Created	December 11,2022		
Actor	System		
Description	A classification system database is very important because that contains predefined sample patterns of the object under consideration that compare with the test object to classify its appropriate class. In a typical classification, the system image is captured by a camera and consequently processed. The neural network is one of the classification algorithms.		
Precondition	1. If the feature is extracted perfectly. 2. All the previous algorithms work perfectly. 3. Then the feature is matched.		
Postcondition	1. All models should be able to identify features perfectly. 2. The dataset should be trained with appropriate data pre-processing.		
Normal Flow	<table><tr><th>System</th></tr><tr><td> 1. The system compares the captured hand signs with the existing dataset. 2. The label hand sign is compared one by one to look for a match. </td></tr></table>	System	1. The system compares the captured hand signs with the existing dataset. 2. The label hand sign is compared one by one to look for a match.
System			
1. The system compares the captured hand signs with the existing dataset. 2. The label hand sign is compared one by one to look for a match.			

Alternative Flaws	1. If the sign is not recognized with the trained data set 2. Then the pattern didn't match.

3.7 Use Case # 6



Figure 7: Pattern Recognition

Table 6: Pattern Recognition Use-case Description

Use Case ID	FR-6
Use Case Name	Pattern Recognition
Date Created	December 11,2022
Description	Making use of mathematical algorithms, intending to interpret human gestures is a topic of Gesture Recognition in the field of language technology and computer science. Mobile and computers are input devices that read input of gestures of the hand. Gestures are taken as input when the computer reads the movements of the body and sends them to the computer. The input is then interpreted by making use of artificial intelligence. The recognized word is converted to voice by making use of the Google audio built-in function.
Precondition	1. Model should be able to detect object. 2. AI model should be trained properly on multiple objects.
Postcondition	1. After detecting the object, the model should label objects in the frame.
Normal Flow	

	System	
	1. The system will run the captured frames from series of block and layer to extract features. 2. The system will predict the bounding boxes for the objects. 3. The system will label the identified objects.	

Chapter 4: Design

Title	Outline
Design	Architecture Diagram
	Entity Relationship Diagram
	Data Flow Diagram
	Class Diagram
	Activity Diagram
	Sequence Diagram
	Collaboration Diagram
	State Transition Diagram
	Component Diagram
	Deployment Diagram
	Data Dictionary
	Logical Schema

4.1 Architectural Diagram

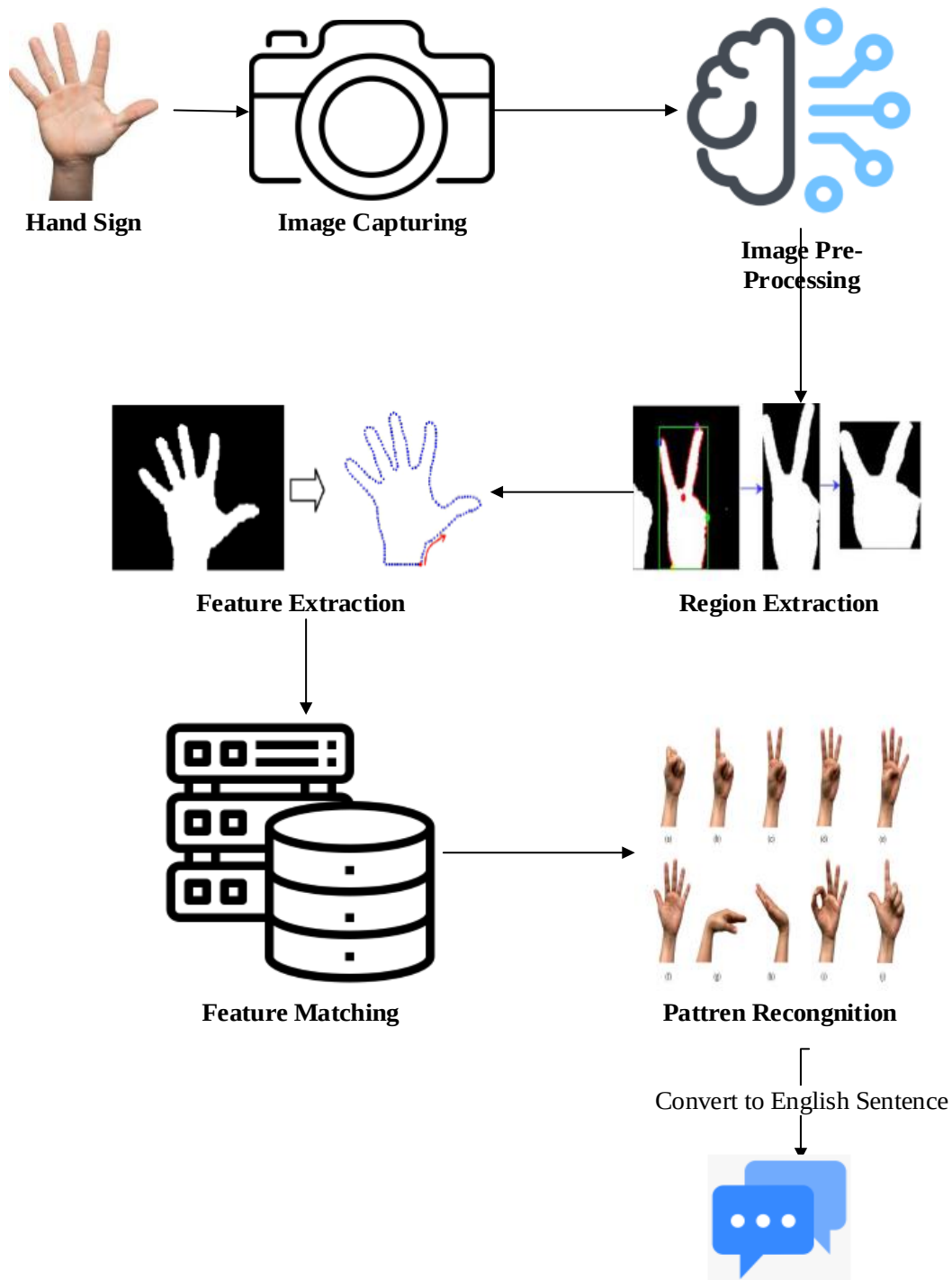


Figure 8: Architectural Diagram

4.2 Entity Relationship Diagram

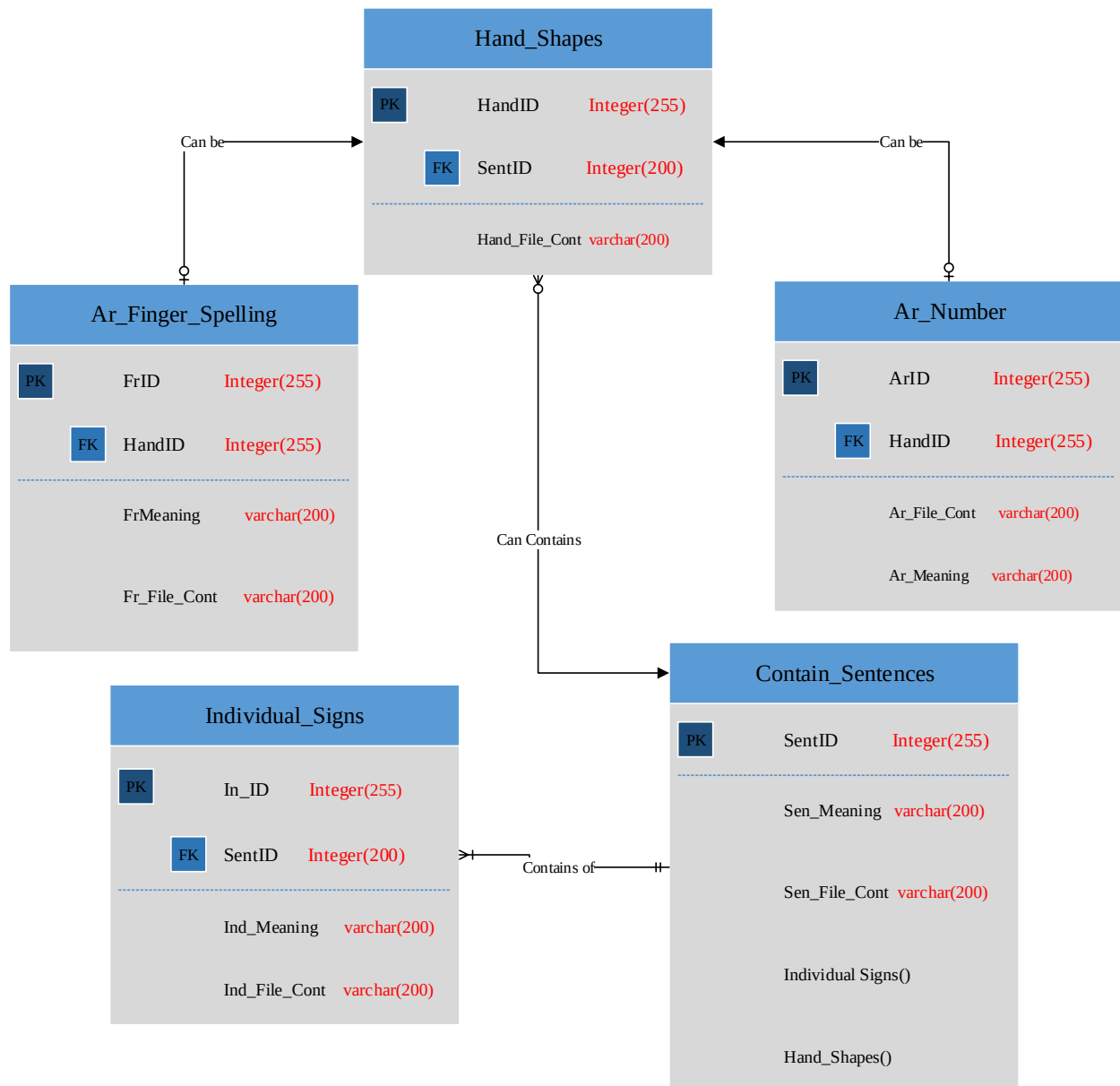


Figure 9: Entity Relation Diagram

4.3 Data Flow Diagram

4.3.1 DFD Level 0

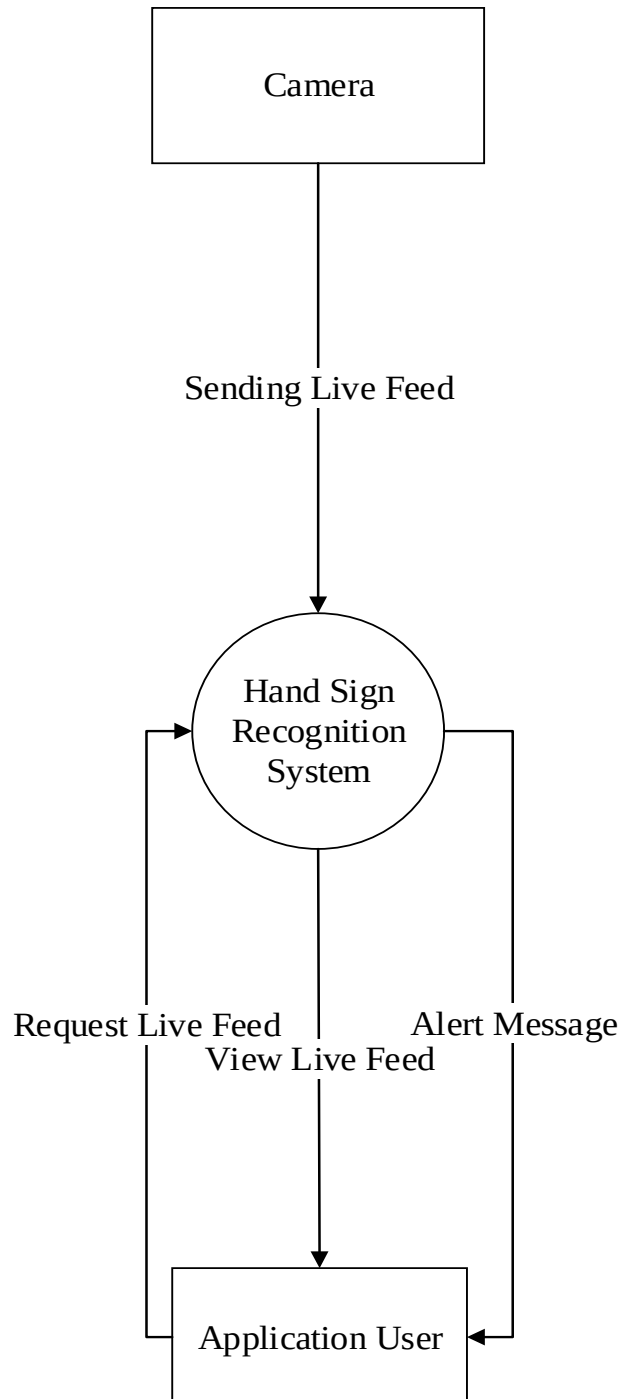


Figure 10: DFD Level 0

4.3.2 DFD Level 1

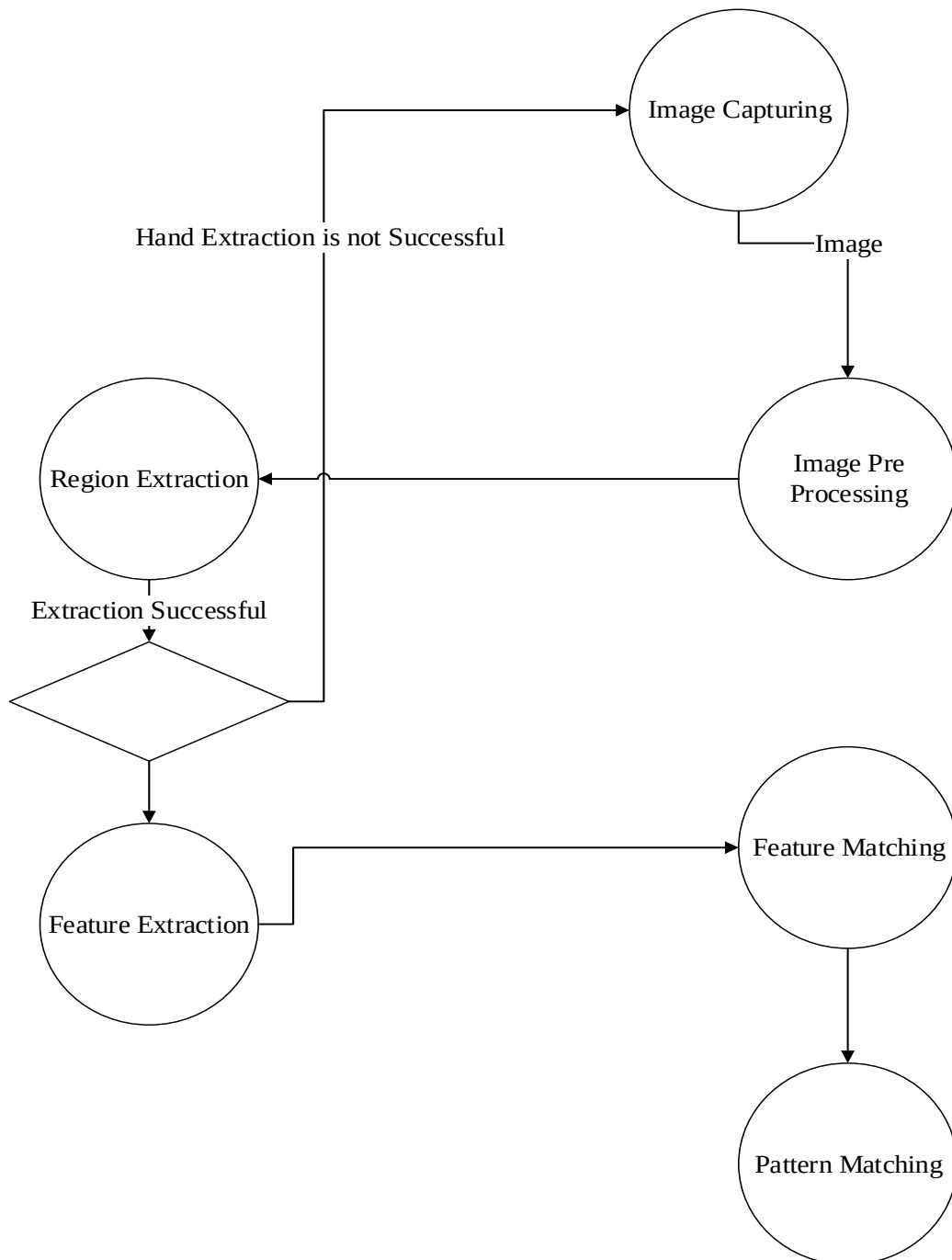


Figure 11: DFD Level 1

4.4 Class Diagram

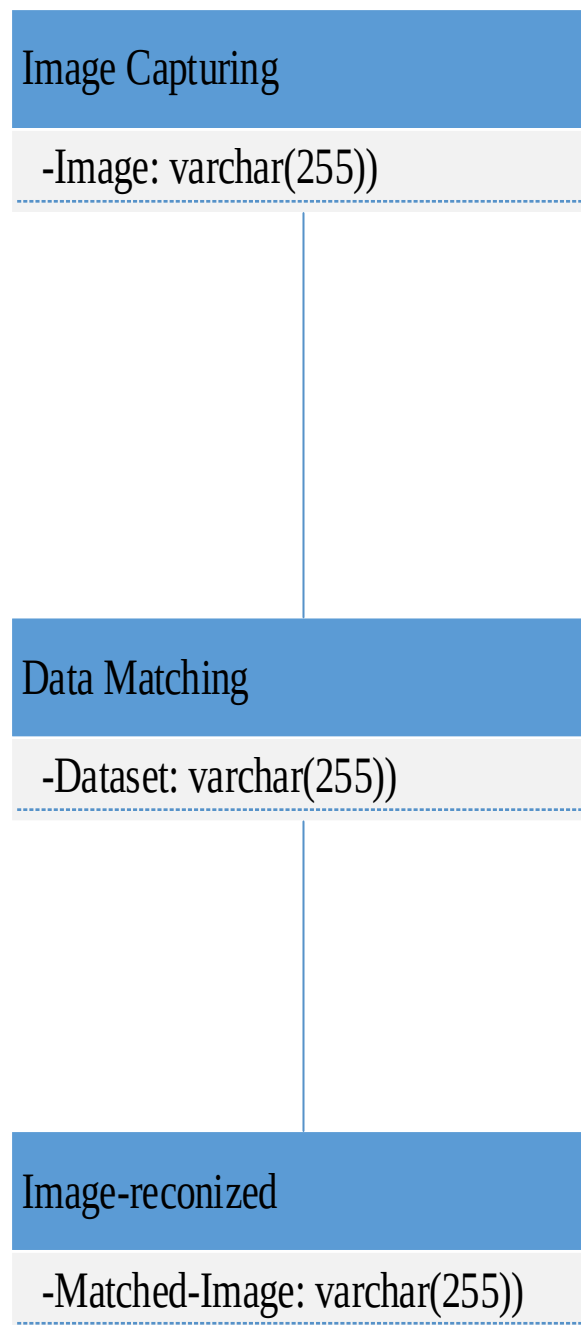


Figure 12: Class Diagram

4.5 Activity Diagram

4.5.1 Image Capturing

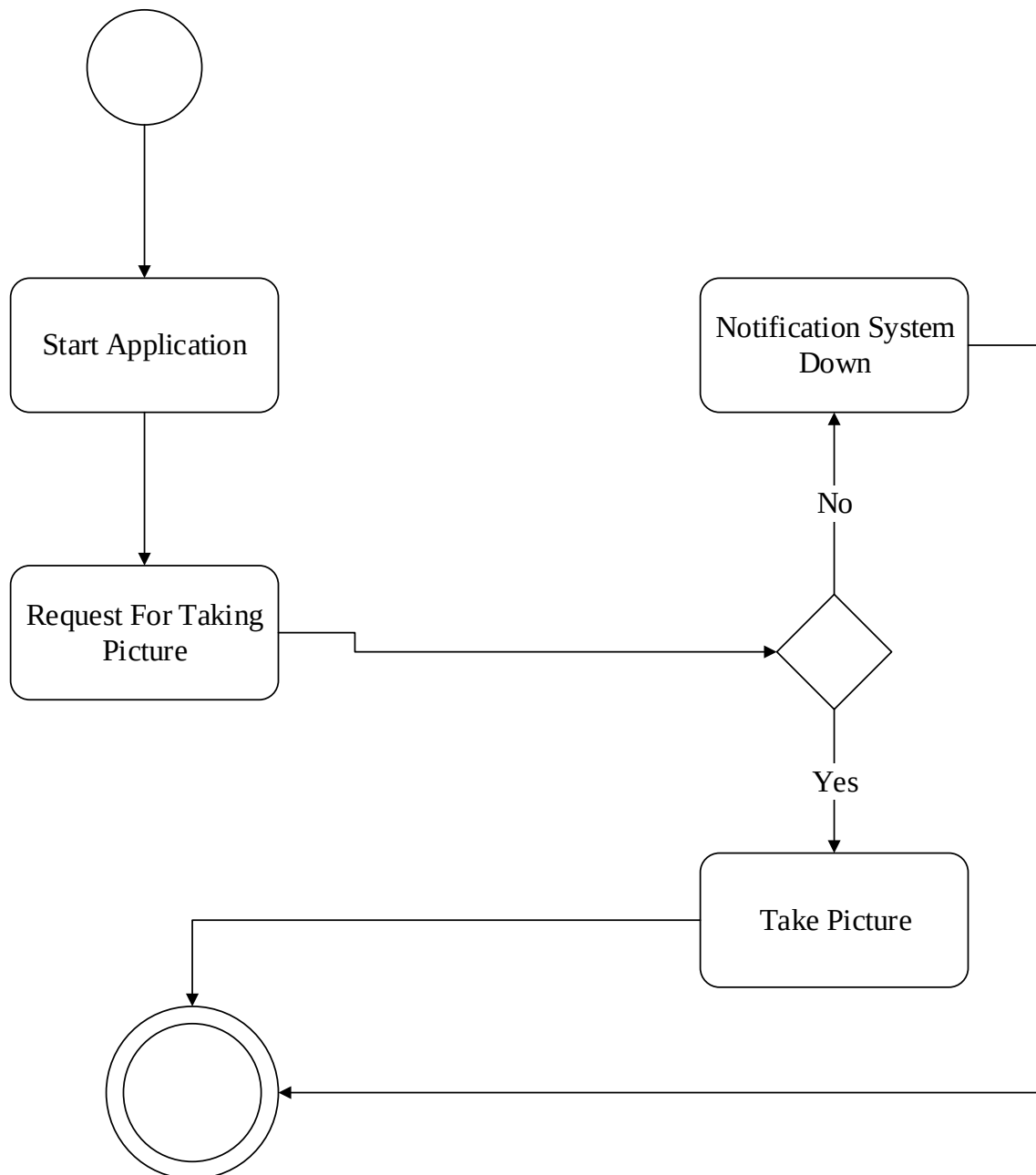


Figure 13: Image Capturing Activity Diagram

4.5.2 Image Pre-processing

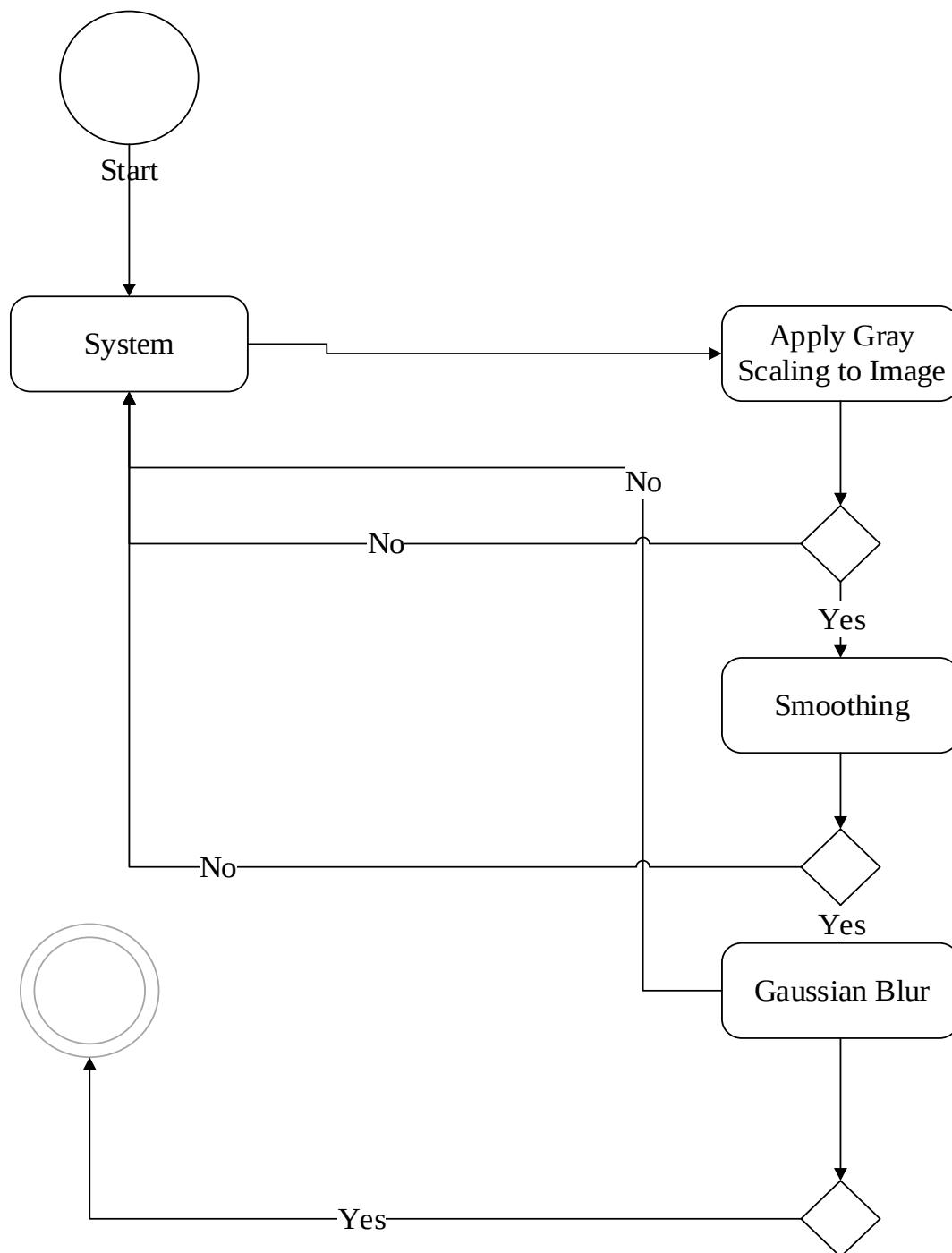


Figure 14: Image Pre-processing Activity Diagram

4.5.3 Region Extraction

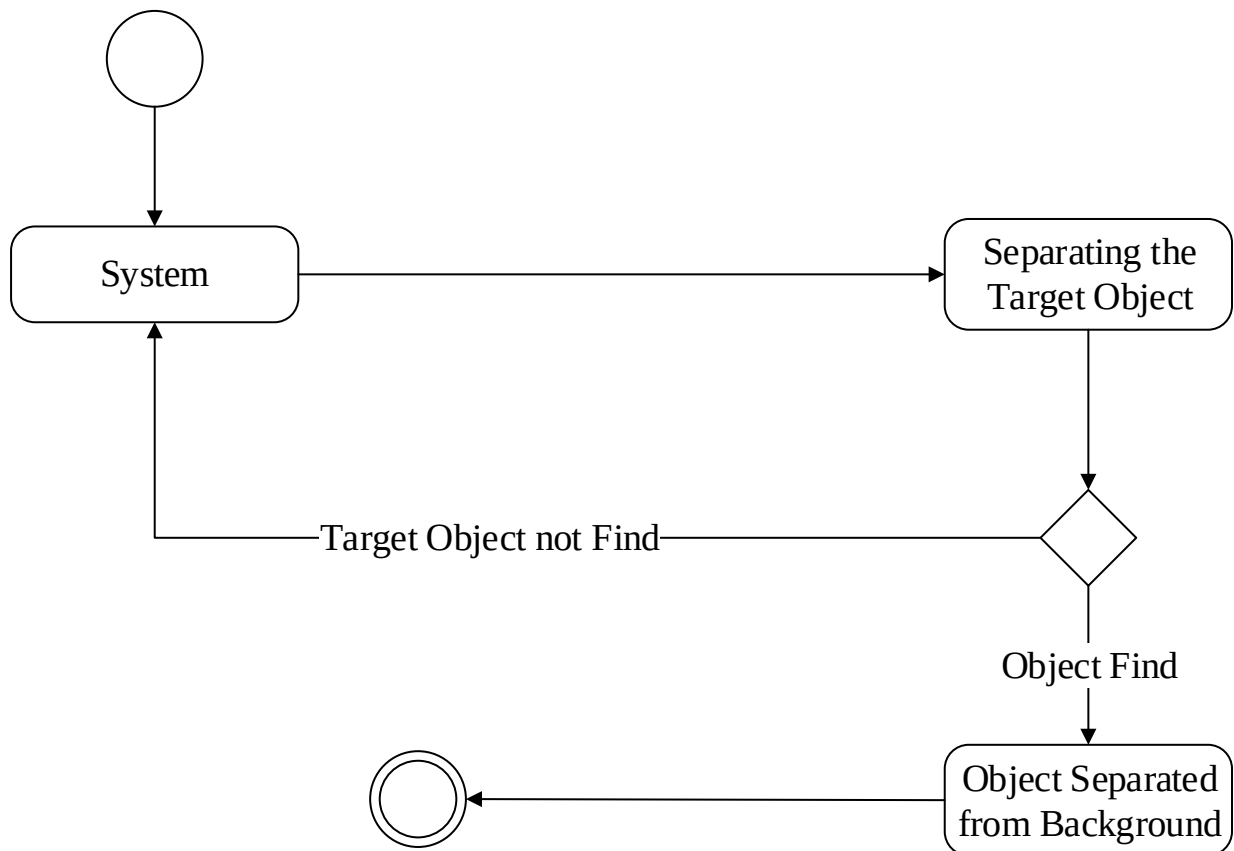


Figure 15: Region Extraction Activity Diagram

4.5.4 Feature Extraction

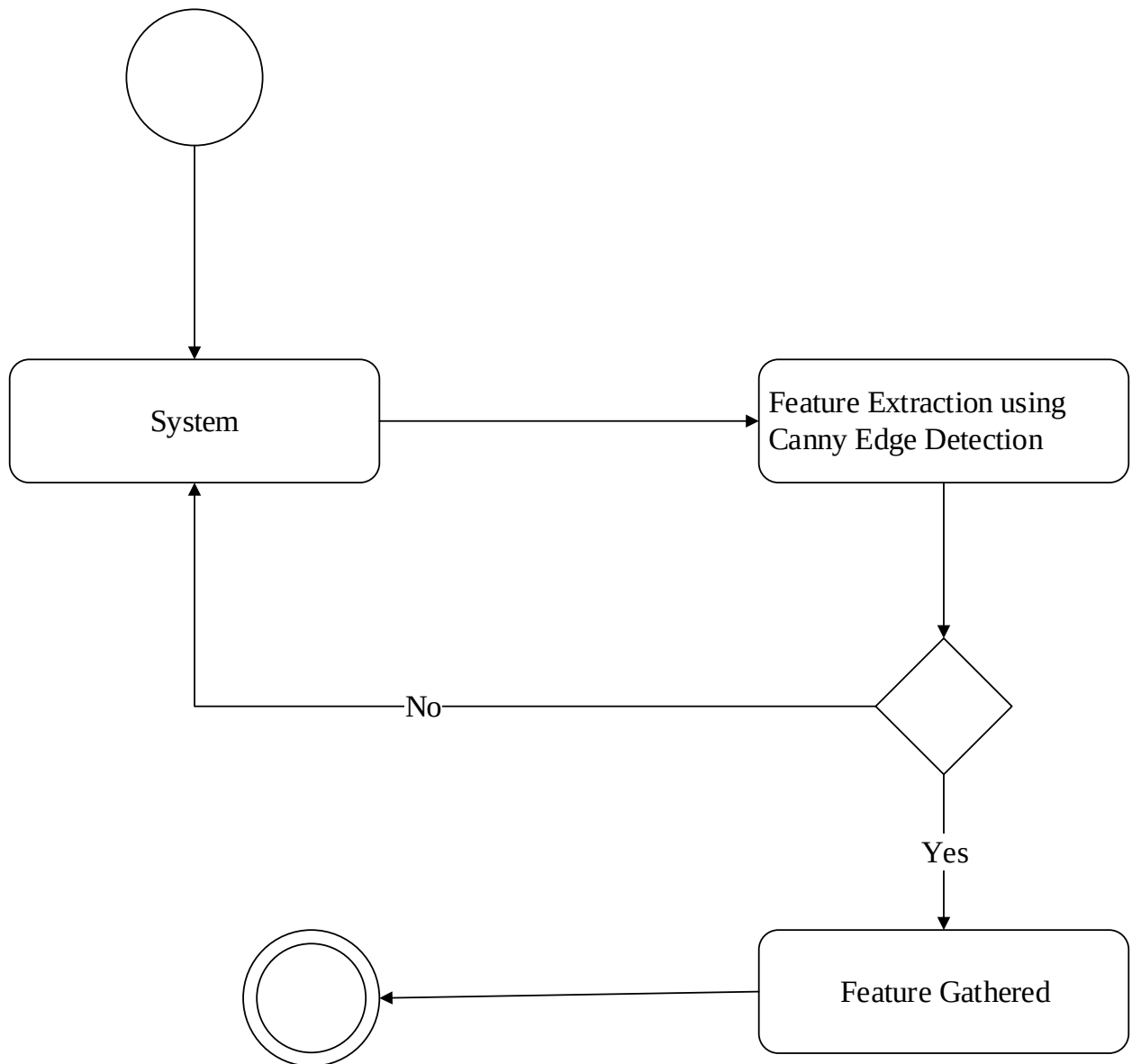


Figure 16: Feature Extraction Activity Diagram

4.5.5 Feature Matching

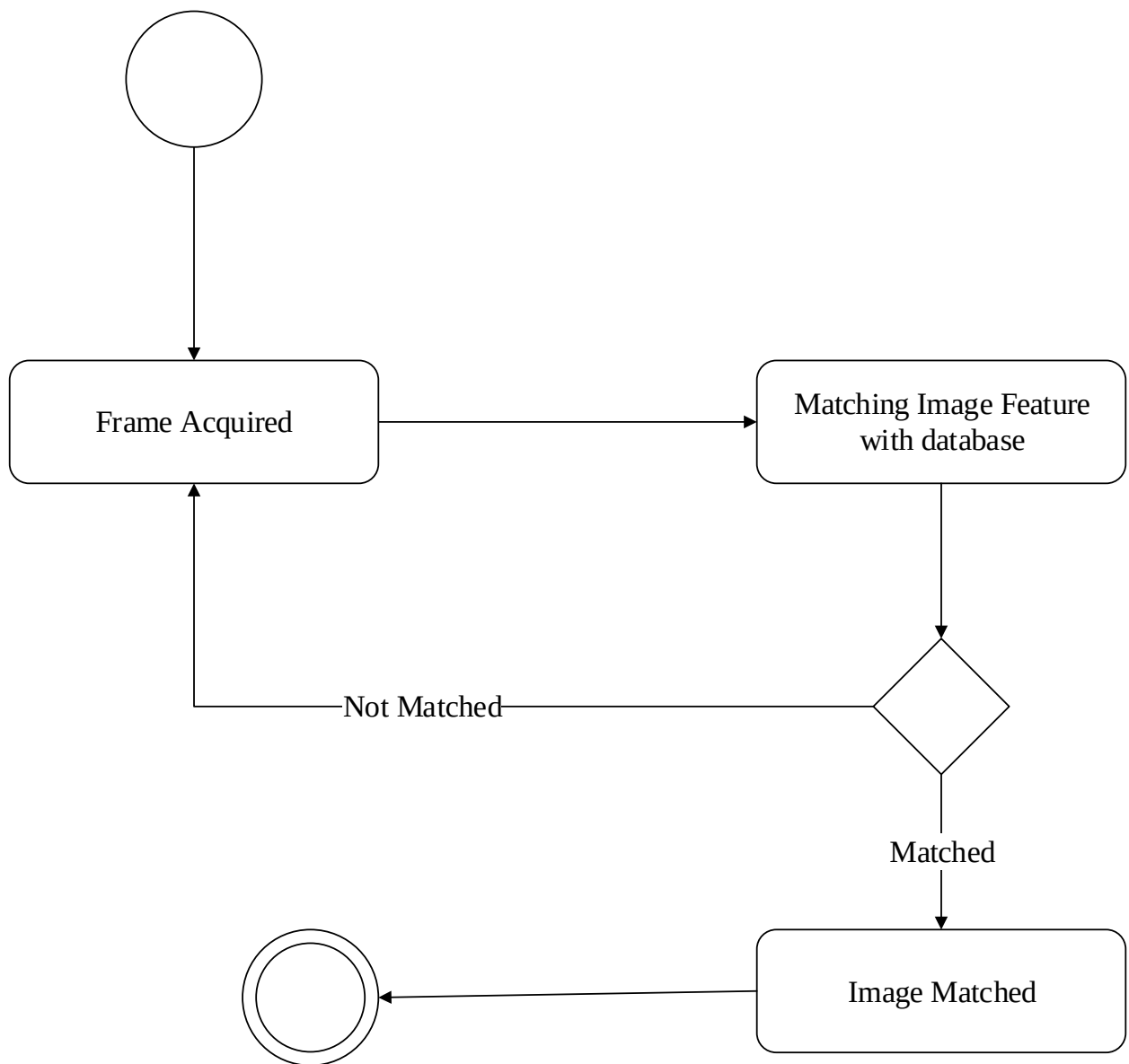


Figure 17: Feature Matching Activity Diagram

4.5.6 Pattern Recognition

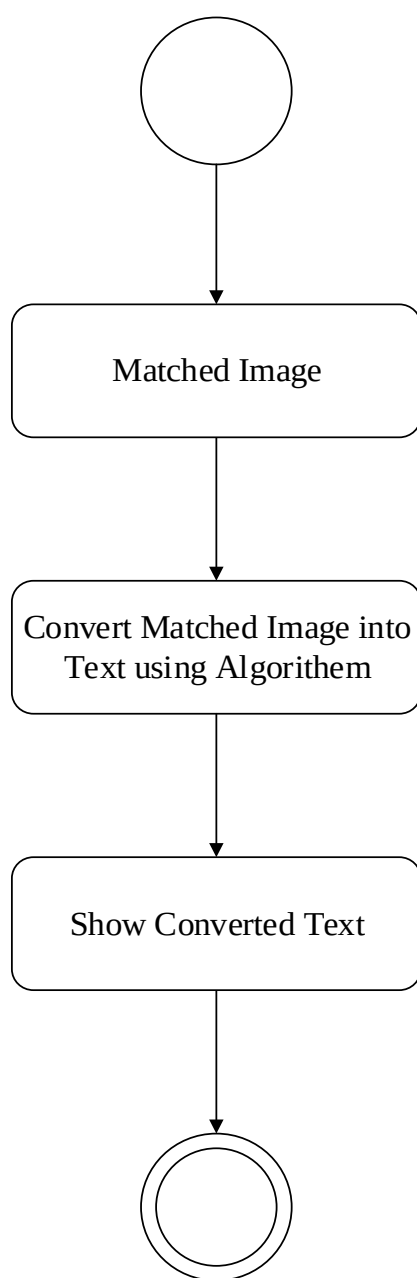


Figure 18: Pattern Recognition Activity Diagram

4.6 Sequence Diagram

4.6.1 Image Capturing

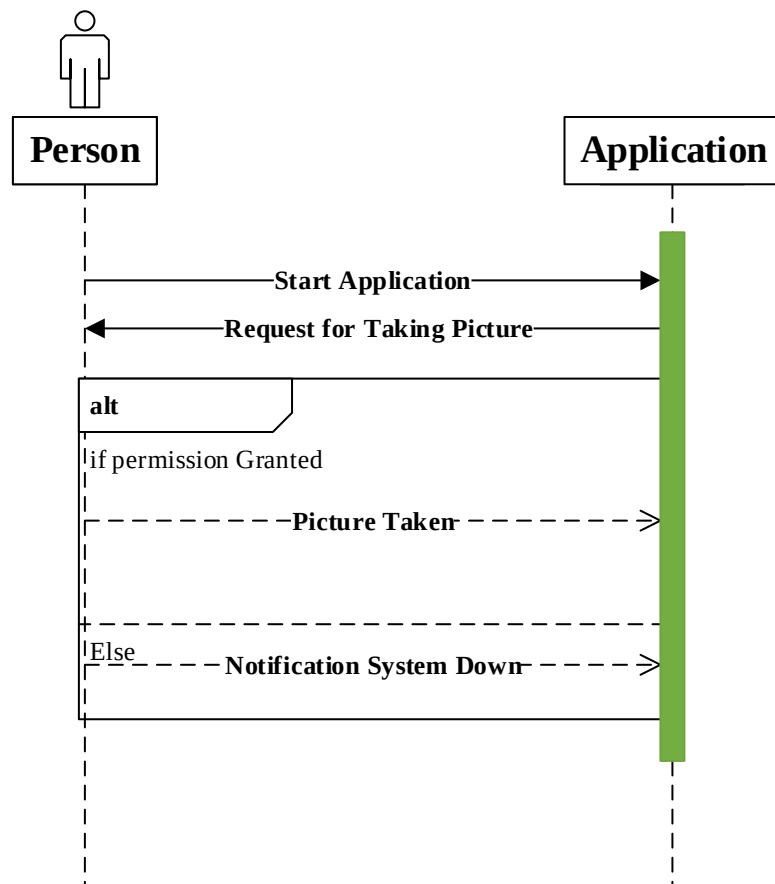


Figure 19: Image Capturing Sequence Diagram

4.6.2 Image Pre-processing

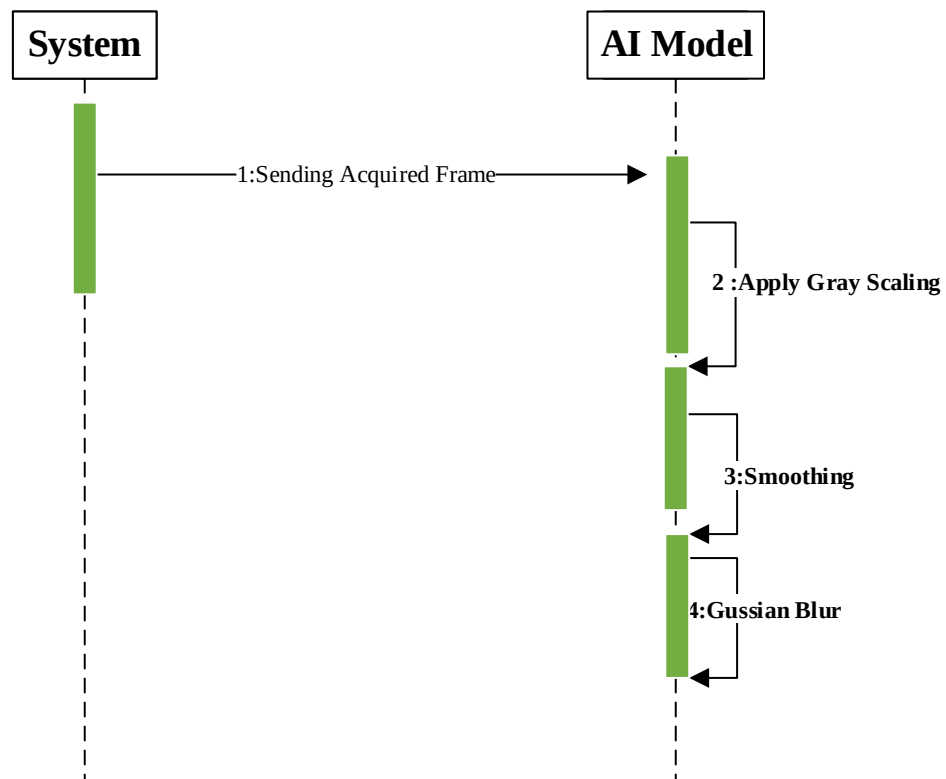


Figure 20: Image Pre-processing Sequence Diagram

4.6.3 Region Extraction

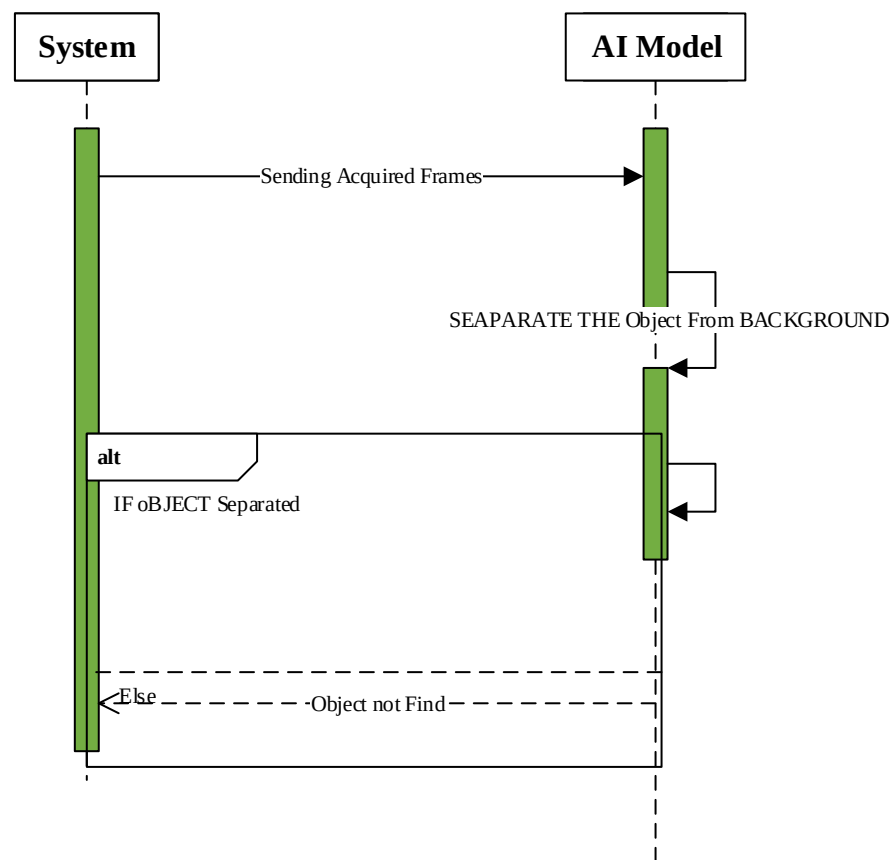


Figure 21: Region Extraction Sequence Diagram

4.6.4 Feature Extraction

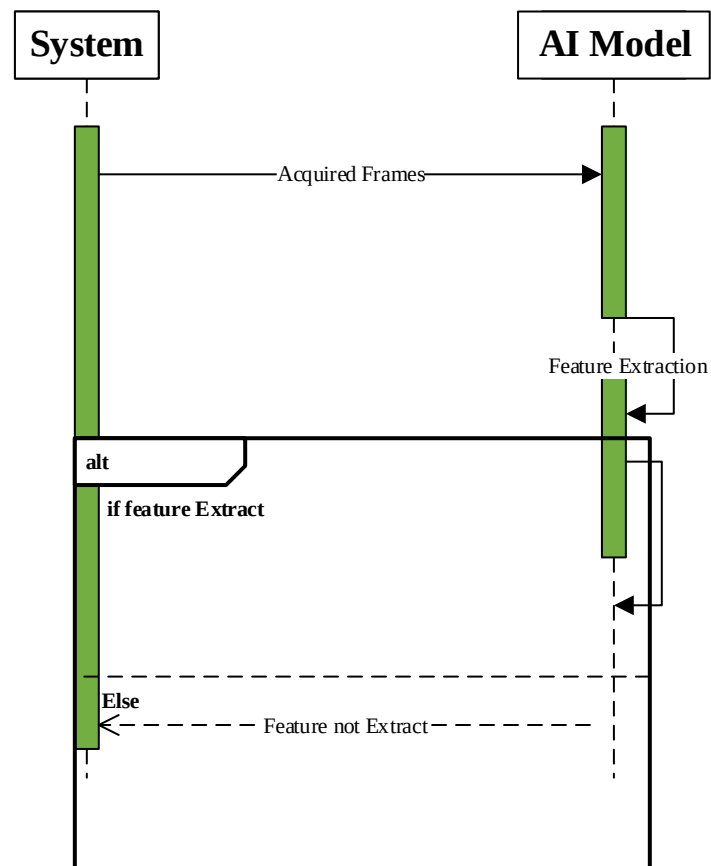


Figure 22: Feature Extraction Sequence Diagram

4.6.5 Feature Matching

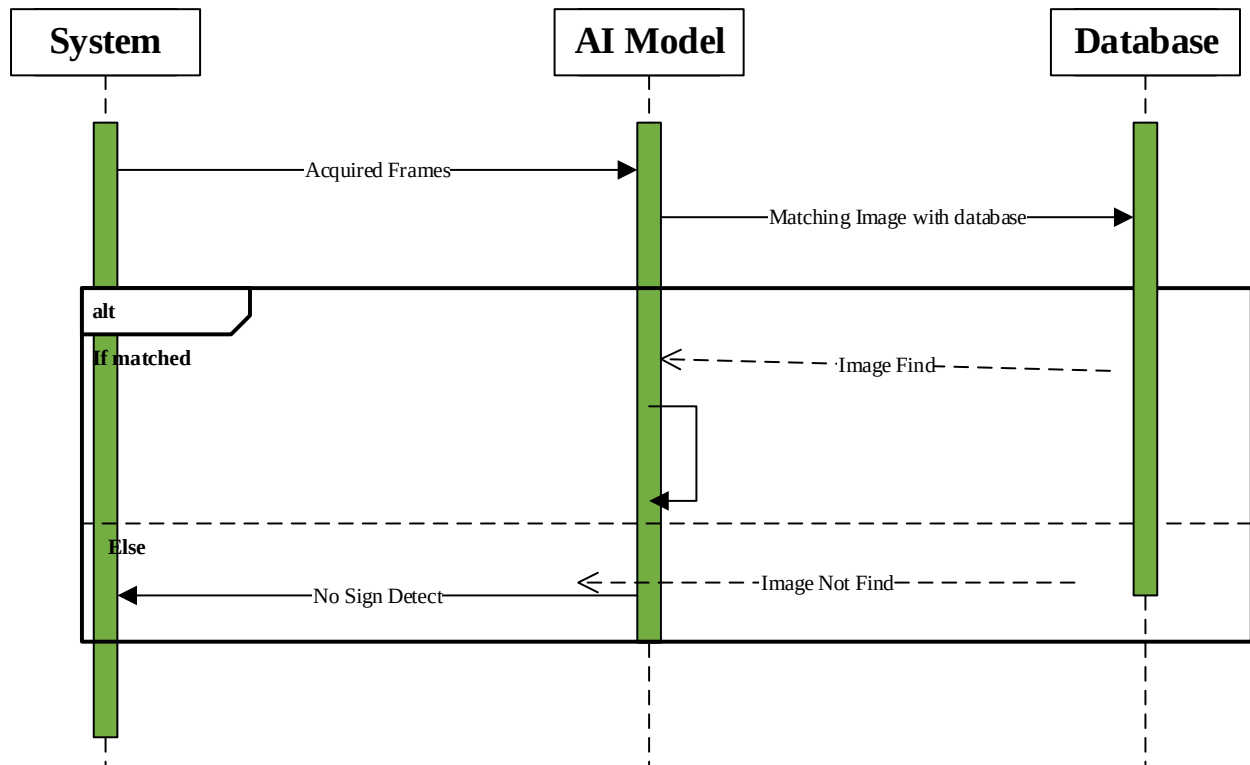


Figure 23: Feature Matching Sequence Diagram

4.6.6 Pattern Recognition

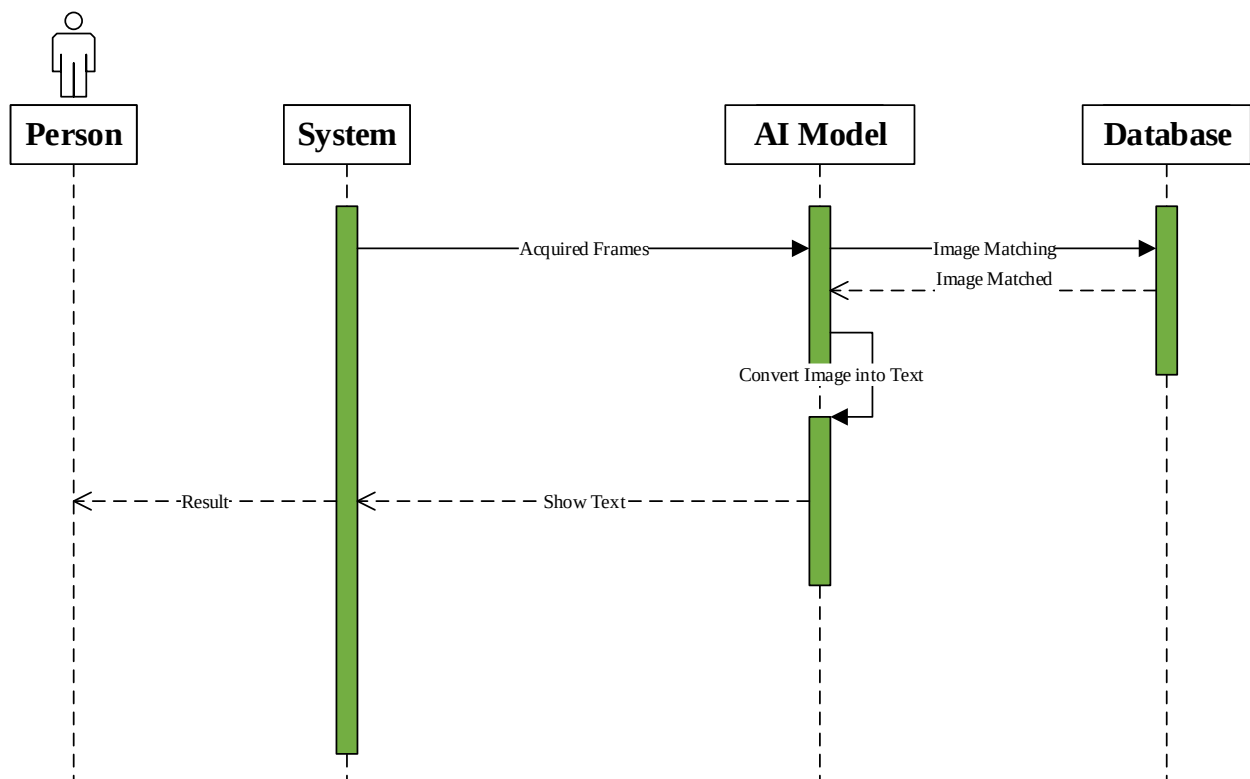


Figure 24: Pattern Recognition Sequence Diagram

4.7 Collaboration Diagram

4.7.1 Image Capturing

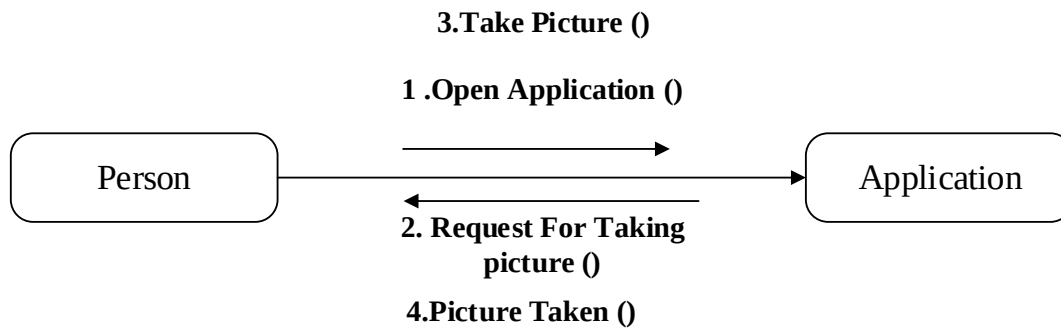


Figure 25: Image Capturing Collaboration Diagram

4.7.2 Image Pre-processing

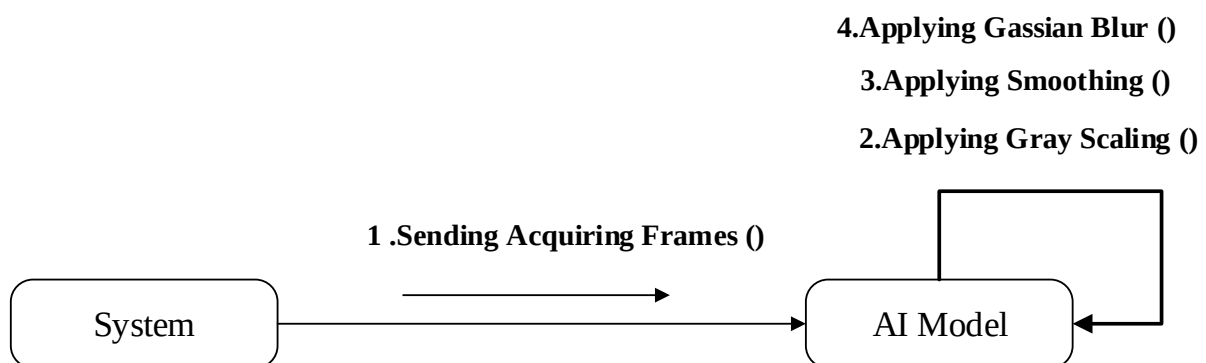


Figure 26: Image Pre-processing Collaboration Diagram

4.7.3 Region Extraction

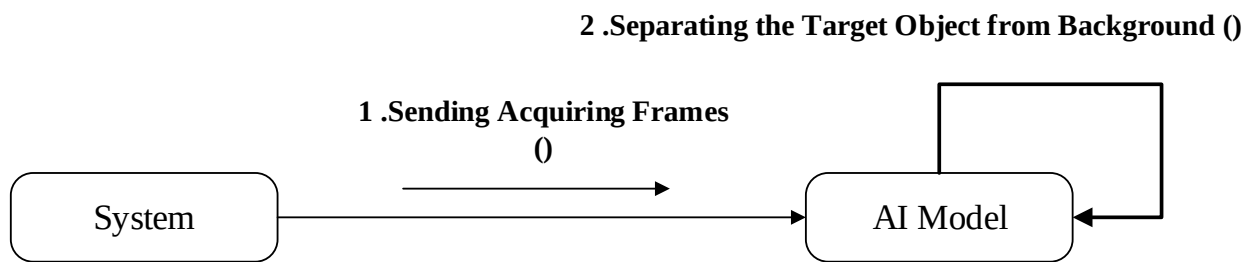


Figure 27: Region Extraction Collaboration Diagram

4.7.4 Feature Extraction

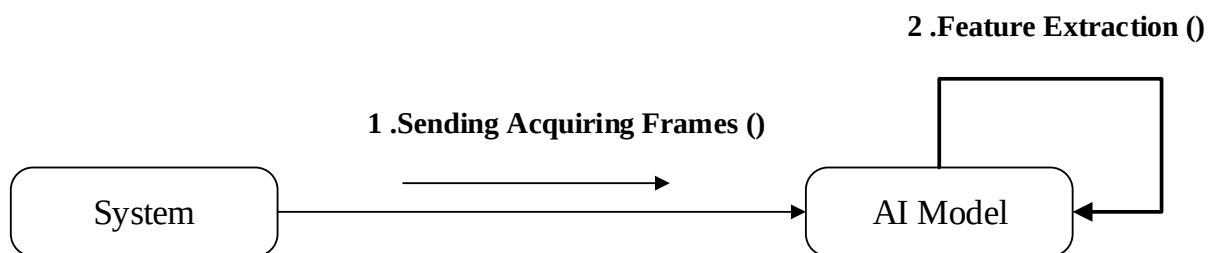


Figure 28: Feature Extraction Collaboration Diagram

4.7.5 Feature Matching

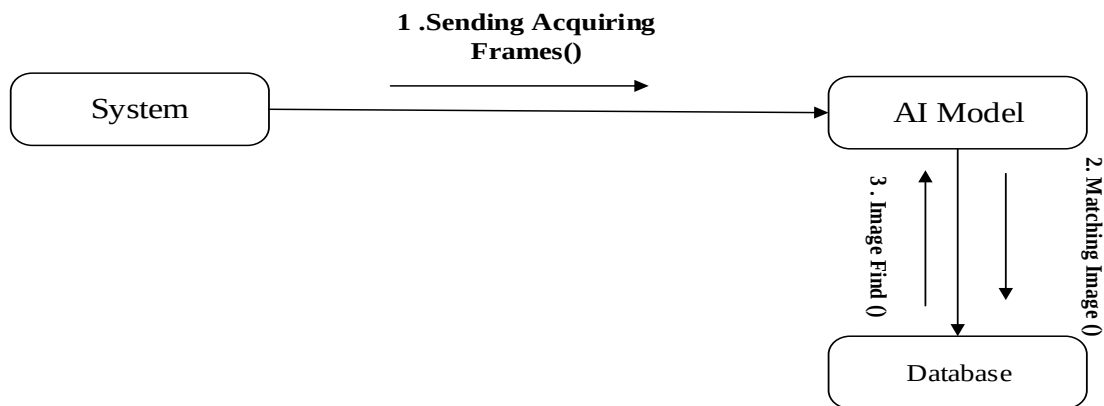


Figure 29: Feature Matching Collaboration Diagram

4.7.6 Pattern Recognition

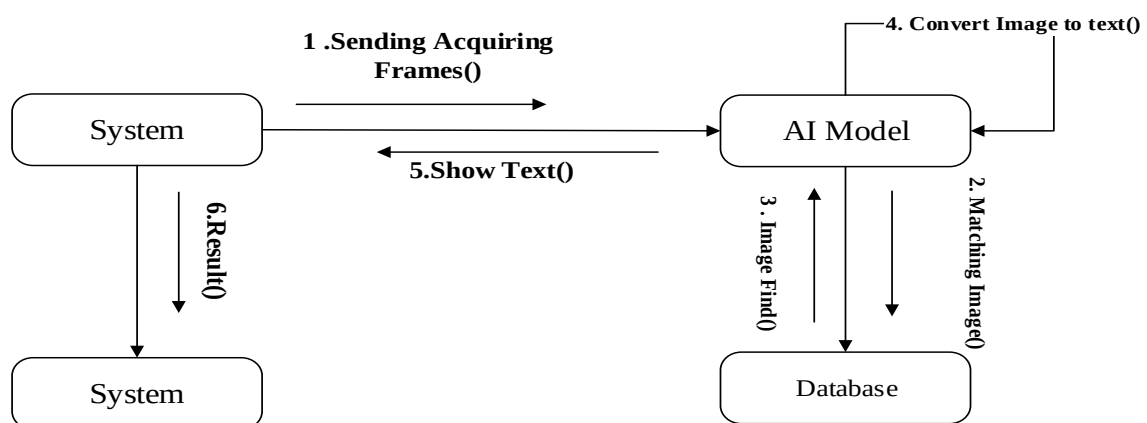


Figure 30: Pattern Recognition Collaboration Diagram

4.8 State Transition Diagram

4.8.1 Image Capturing

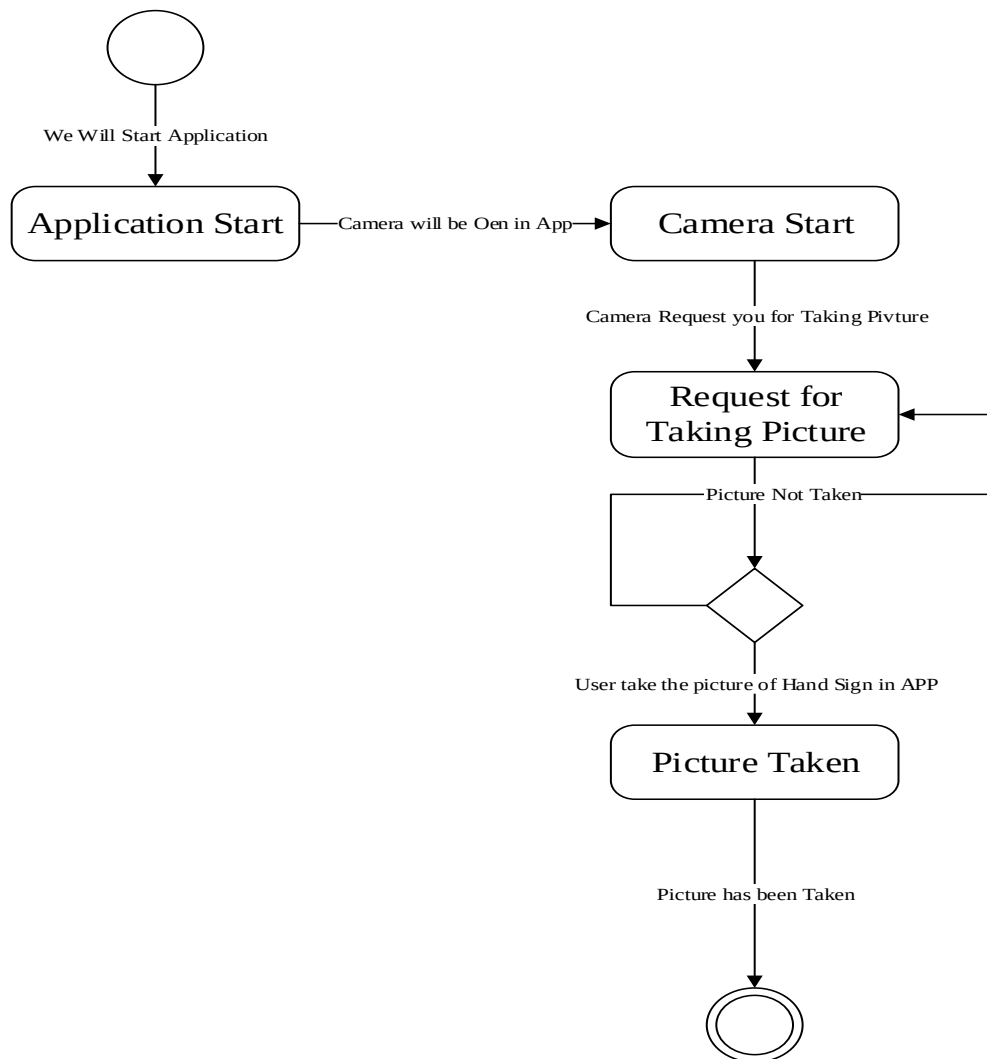


Figure 31: Image Capturing State Transition Diagram

4.8.2 Image Pre-processing

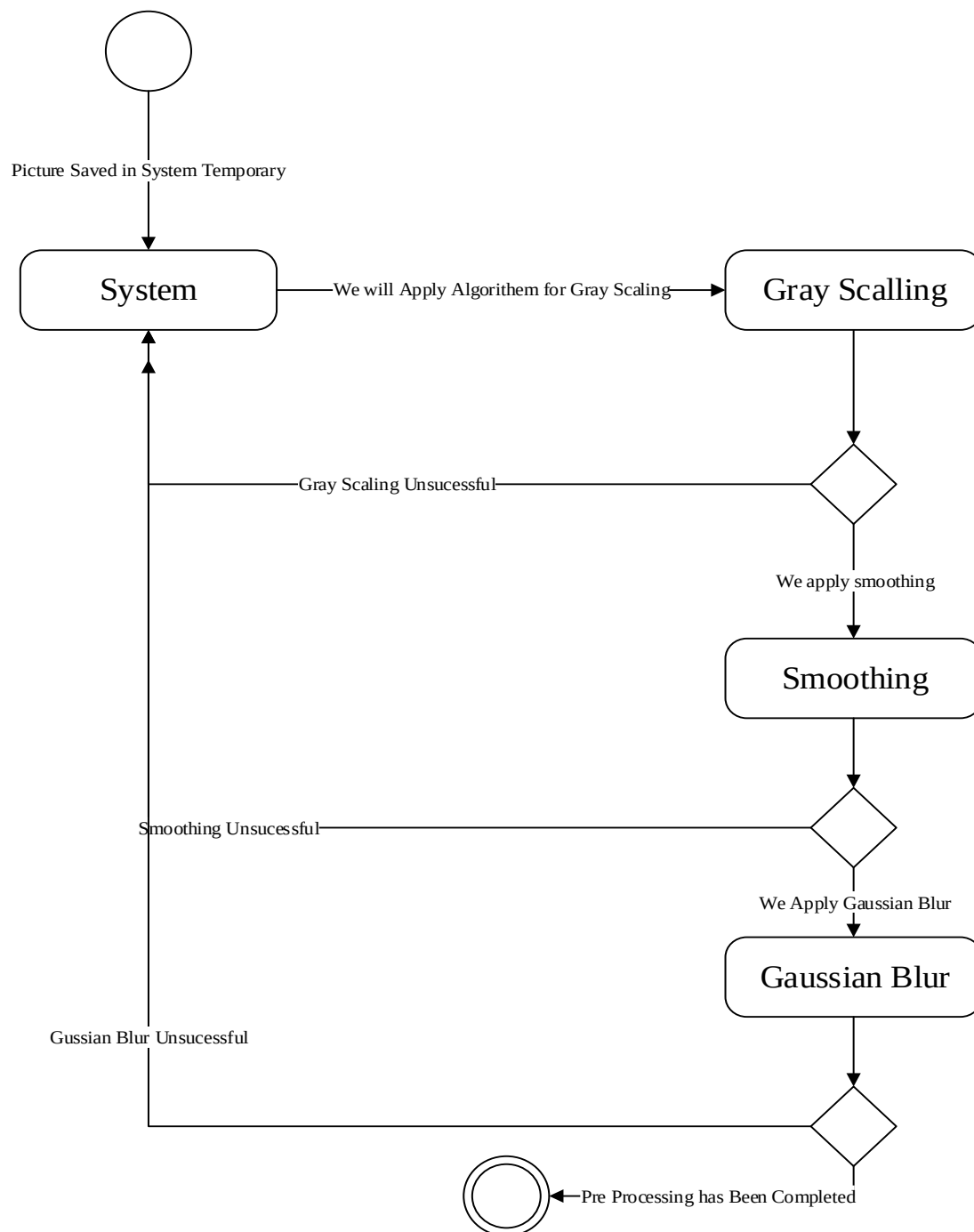


Figure 32: Image Pre-processing State Transition Diagram

4.8.3 Region Extraction

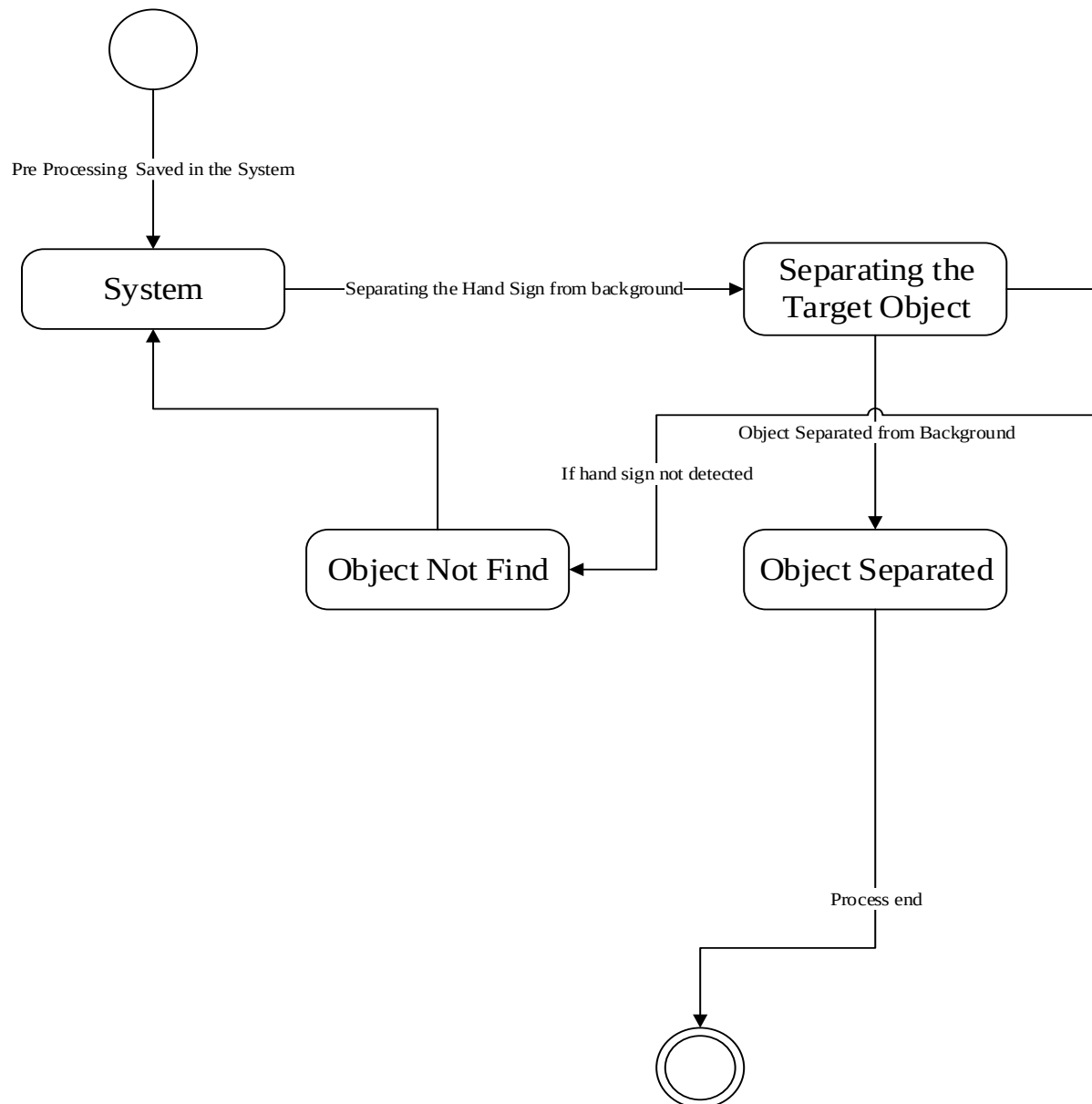


Figure 33: Region Extraction State Transition Diagram

4.8.4 Feature Extraction

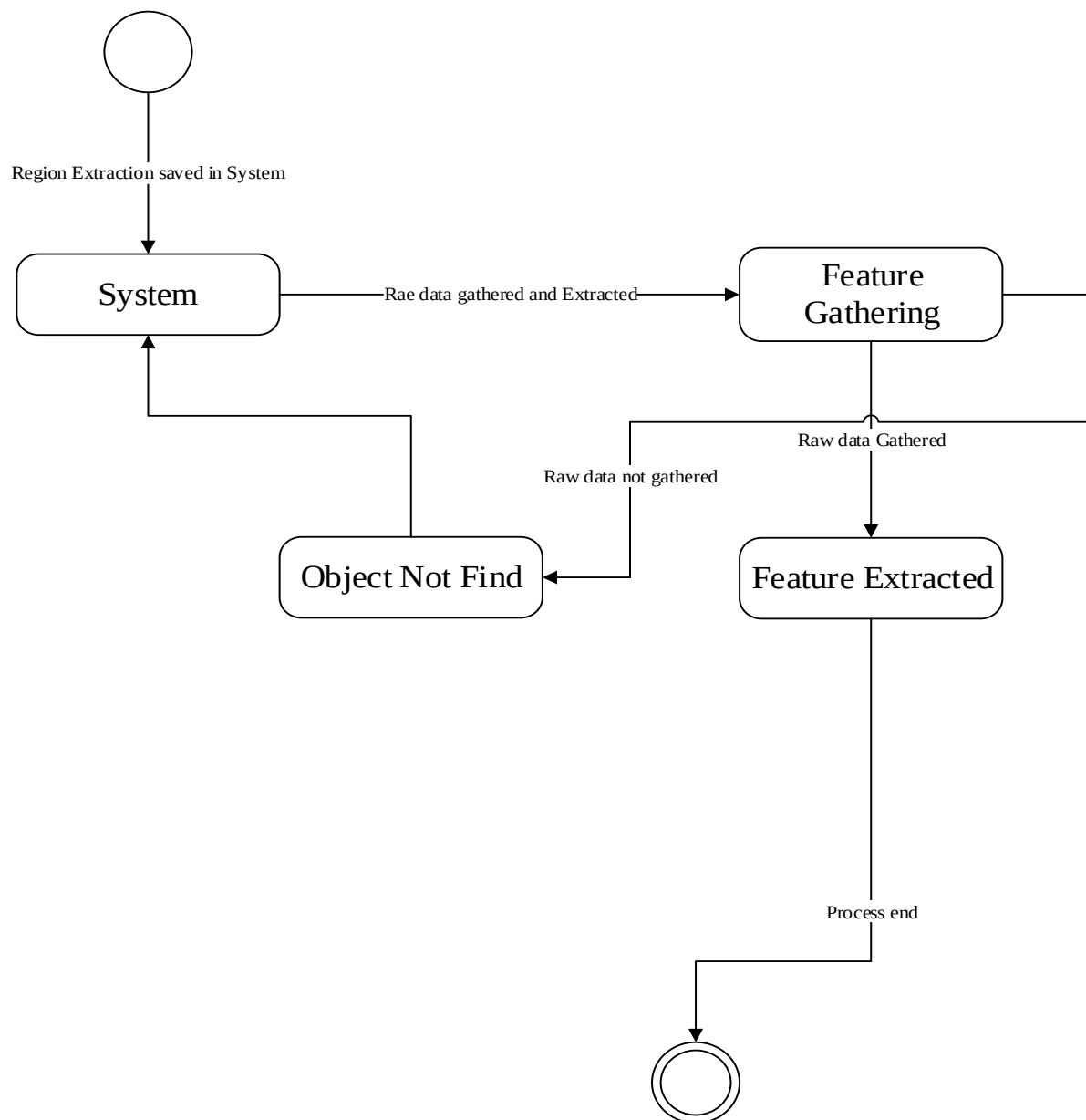


Figure 34: Feature Extraction State Transition Diagram

4.8.5 Feature Matching

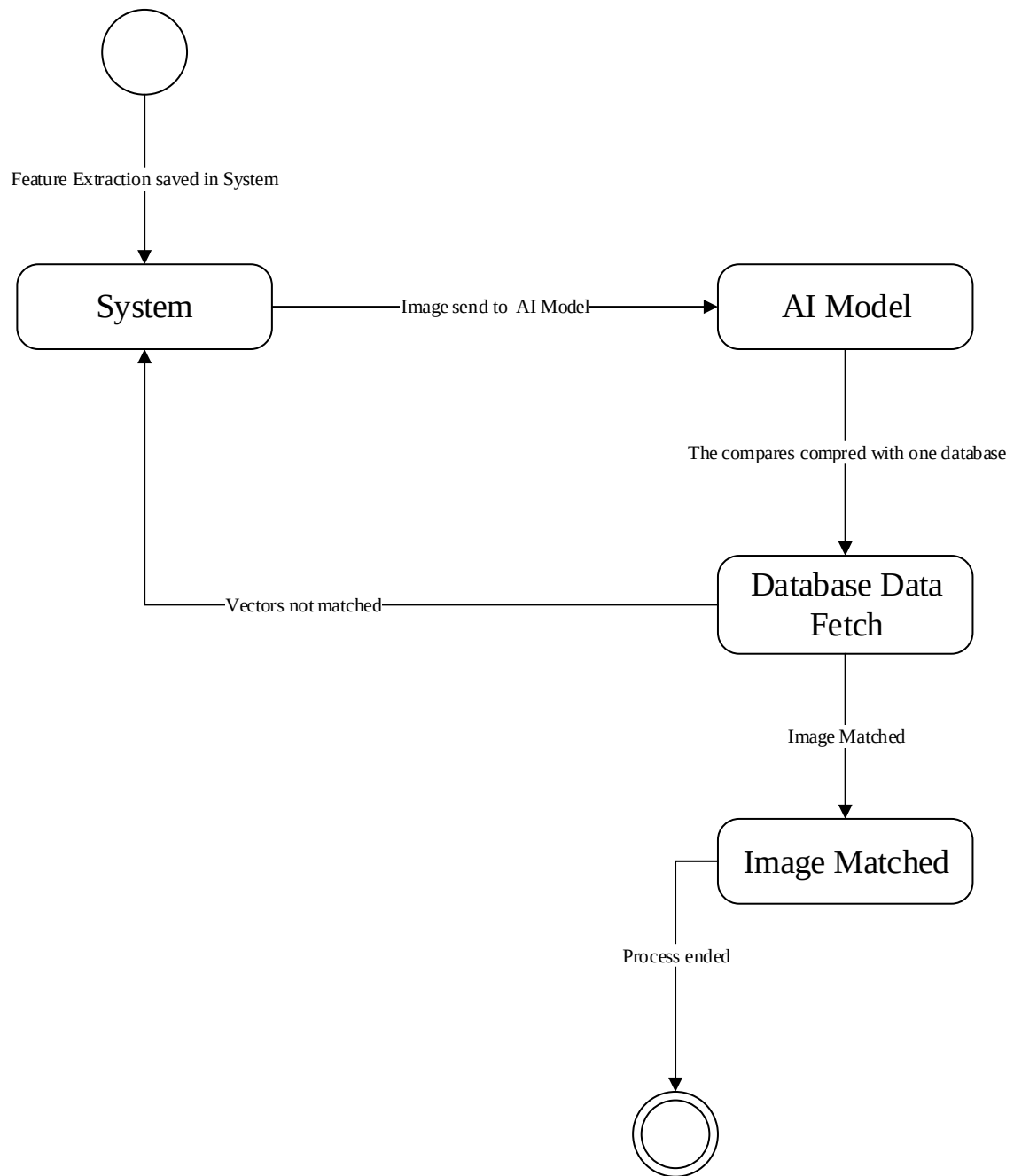


Figure 35: Feature Matching State Transition Diagram

4.8.6 Pattern Recognition

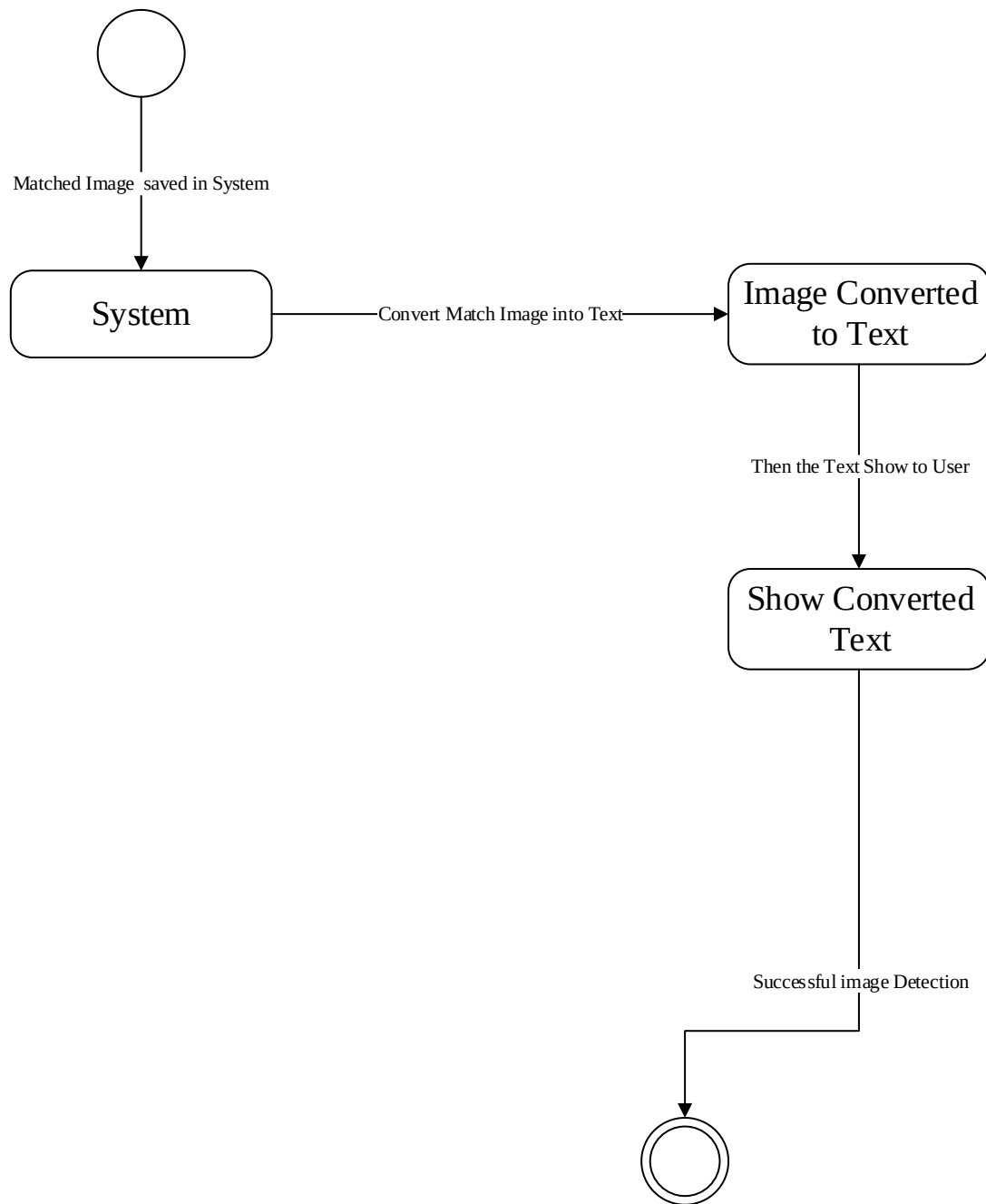


Figure 36: Pattern Recognition State Transition Diagram

4.9 Component Diagram

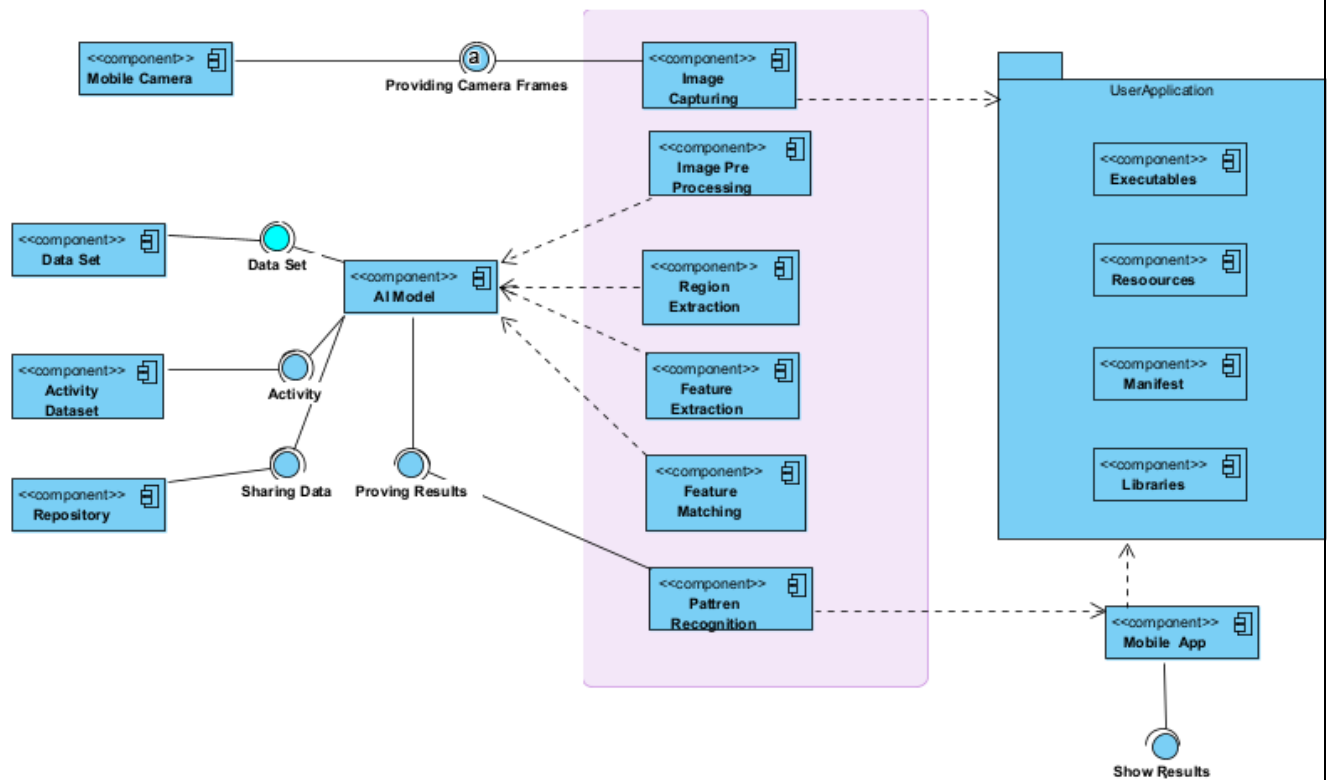


Figure 37: Component Diagram

4.10 Deployment Diagram

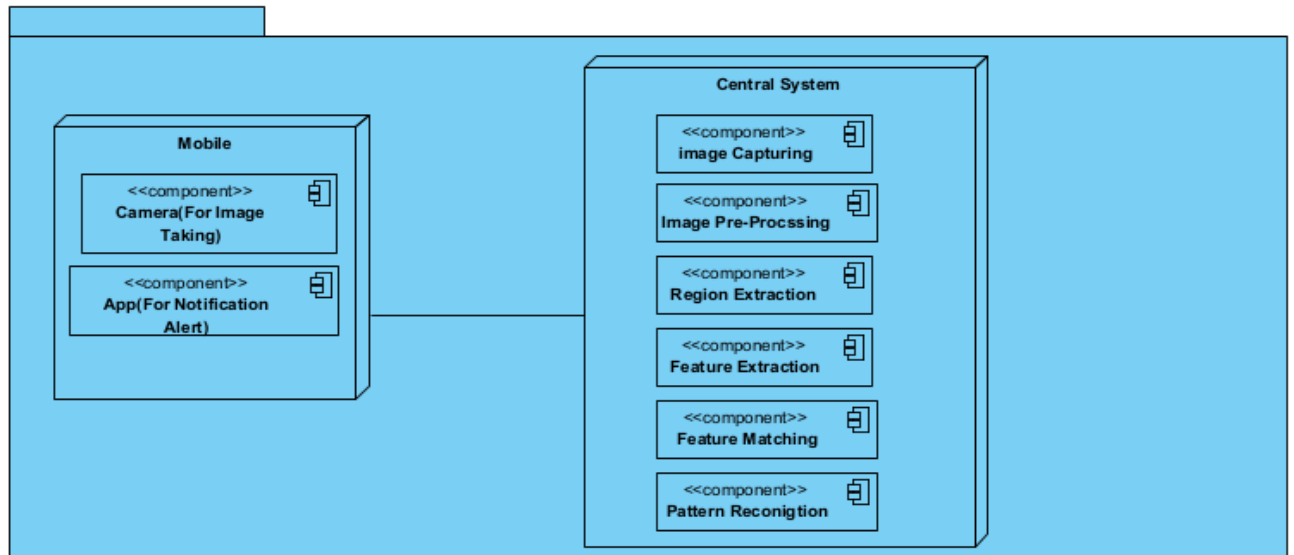


Figure 38: Deployment Diagram

4.11 Data Dictionary

Table 7: Data Dictionary

Fields	Data Type	Size	Description	Source Table
SentID	Integer	255	Identifier	Contain_Sentence
Sen_Meaning	Varchar	200	Meaning	Contain_Sentence
Sen_File_Cont	Varchar	200	File for Sentence Sign	Contain_Sentence
In_ID	Integer	255	Identifier	Individual_Sign
Ind_Meaning	Varchar	200	Meaning	Individual_Sign
Ind_File_Cont	Varchar	200	File for Individual Sign	Individual_Sign
HandID	Integer	255	Identifier	Hand_Shapes
Hand_File_Cont	Varchar	200	Hand Signs Content	Hand_Shapes
FrID	Integer	255	Identifier	Ar_Finger_Spelling
FrMeaning	Varchar	200	Meaning	Ar_Finger_Spelling
Fr_File_Cont	Varchar	200	Finger Sign Content	Ar_Finger_Spelling
ArID	Integer	255	Identifer	Ar_Number
Ar_File_Cont	Varchar	200	Number file Content	Ar_Number
Ar_Meaning	Varchar	200	Number Meaning	Ar_Number

4.12 Logical Schema

Table 8: Logical Schema

Contain_Sentence

SentID	Sen_Meaning	Sen_File_Cont	Individual_Signs	Hand_Shapes
--------	-------------	---------------	------------------	-------------

Individual Signs

Individual_Signs	In_ID	Ind_Meaning	Ind_File_Cont
------------------	-------	-------------	---------------

Hand_Shapes

Hand_Shapes	HandID	Hand_File_Cont
-------------	--------	----------------

Ar_Finger_Spelling

FrID	<u>HandID</u>	FrMeaning	Fr_File_Cont
------	---------------	-----------	--------------

Ar_Number

ArID	<u>HandID</u>	Ar_File_Cont	Ar_Meaning
------	---------------	--------------	------------

Activate Windows

Chapter 5: Testing

Title	Outline
Testing SQA introduction and Test Cases	Test Case Specifications
	Black Box Test Cases
	White Box Test Cases

5.1 Test Case Specifications.

5.1.1 Test Case for Camera Activation

Table 9: Camera Activation Test Case-Positive

Camera Activation Test Case - Positive	
ID	TC_1.1
Priority	High
Description	Open the app and check the camera is working
Reference	-
Actor	Person/User
Pre-requisites	The Camera should be active
Steps	1. User open the app 2. The Camera of the mobile is open
Input	User Open the App
Expected Result	User Open the App/Click on mobile app to open
Status	Tested, Passed

Table 10: Camera Activation Test Case – Negative

Camera Activation Test Case - Negative	
ID	TC_1.2
Priority	High
Description	Open the app and check the camera is working
Reference	-
Actor	Person/User
Pre-requisites	The Camera should be active

Steps	1. User open the app
Input	User Open the App/Click on mobile app to open
Expected Result	The Camera of the mobile is not open or working properly
Status	Tested, Passed

5.1.2 Test Case for Image Capturing

Table 11: Image Capturing Test Case – Positive

Image Capturing Test Case – Positive	
ID	TC_2.1
Priority	High
Description	The Optical device (camera) captures the frames from the perceiving site of the deaf-mute person and save them into memory for processing.
Reference	FR-1
Actor	Camera
Pre-requisites	1. The Camera should be active 2. The frame should be clear
Steps	1. Place the hand in front of the camera in a well-lit area. 2. Capture the image of the deaf-mute person
Input	Capture the image

Expected Result	Image is captured successfully
Status	Tested, Passed

Table 12: Image Capturing Test Case – Negative

Image Capturing Test Case – Negative	
ID	TC_2.2
Priority	High
Description	The Optical device (camera) captures the frames from the perceiving site of the deaf-mute person and save them into memory for processing.
Reference	FR-1
Actor	Camera
Pre-requisites	1. The Camera should be active 2. The frame should be clear
Steps	1. Place the hand in front of the camera in a well-lit area. 2. Capture the image of the deaf-mute person
Input	Capture the image
Expected Result	Image is not captured successfully
Status	Tested, Passed

5.1.3 Test Case for Image Pre-Processing

Table 13: Image Pre-Processing Test Case – Positive

Image Pre-Processing Test Case – Positive	
ID	TC_3.1
Priority	High
Description	In image preprocessing, different algorithms like smoothing, and Gaussian Blur would be applied to the captured image. By use of OpenCV, the input sequence of RGB images gets converted to gray images.
Reference	FR-2
Actor	System
Pre-requisites	1. The Camera should be active 2. AI model should be work properly
Steps	1. Image Resizing: Media Pipe may resize the input image to a specific size or aspect ratio using it resize module. 2. Color Space Conversion: Media Pipe can convert the image to a different color space using its color conversion module, such as converting an RGB image to grayscale or YUV.

	3. Image Augmentation: Media Pipe provides various data augmentation modules, such as flipping, rotating, or cropping the input image, to increase the diversity and robustness of the training data. 4. Filtering: Media Pipe may apply various filters to the input image, such as Gaussian blur, median blur, or bilateral filter, to smooth or sharpen the image.
Input	Resizing and gray scaling the captured image
Expected Result	AI model Resize and Grayscale the captured image Successfully
Status	Tested, Passed

Table 14: Image Pre-Processing Test Case – Negative

Image Pre-Processing Test Case – Negative	
ID	TC_3.2
Priority	High
Description	In image pre-processing, different algorithms like smoothing, and Gaussian Blur would be applied to the captured image. By use of OpenCV, the input sequence of RGB images gets converted to gray images.
Reference	FR-2

Actor	System
Pre-requisites	1. The Camera should be active 2. AI model should be work properly
Steps	1. Image Resizing: Media Pipe may resize the input image to a specific size or aspect ratio using it resize module. 2. Color Space Conversion: Media Pipe can convert the image to a different color space using its color conversion module, such as converting an RGB image to grayscale or YUV. 3. Image Augmentation: Media Pipe provides various data augmentation modules, such as flipping, rotating, or cropping the input image, to increase the diversity and robustness of the training data. 4. Filtering: Media Pipe may apply various filters to the input image, such as Gaussian blur, median blur, or bilateral filter, to smooth or sharpen the image.
Input	Resizing and rescaling the captured image
Expected Result	AI model did not Resize and Grayscale the captured image Successfully

Status	Tested, Passed
--------	----------------

5.1.4 Test Case for Region Extraction

Table 15: Region Extraction Test Case – Positive

Region Extraction Test Case – Positive	
ID	TC_4.1
Priority	High
Description	In Region Extraction In this functionality, the hand feature or hand objects in an image would be separated from its background by applying Background Segmentation.
Reference	FR-3
Actor	System
Pre-requisites	1. Image needs to be High Resolution. 2. Image needs to be Greyscale and Resized.
Steps	1. The system separated the hand image 2. The system removes background by applying background segmentation
Input	The result from AI Model
Expected Result	AI model separated the hand from surrounding region
Status	Tested, Passed

Table 16: Region Extraction Test Case – Negative

Region Extraction Test Case – Negative	
ID	TC_4.2
Priority	High
Description	In Region Extraction In this functionality, the hand feature or hand objects in an image would be separated from its background by applying Background Segmentation.
Reference	FR-3
Actor	System
Pre-requisites	3. Image needs to be High Resolution. 4. Image needs to be Greyscale and Resized.
Steps	3. The system separated the hand image 4. The system removes background by applying background segmentation
Input	The result from AI Model
Expected Result	AI model did not separate the hand from the surrounding region
Status	Tested, Passed

5.1.5 Test Case for Feature Extraction

Table 17: Test Case for Feature Extraction

Feature Extraction Test Case – Positive	
ID	TC_5.1

Priority	High
Description	For feature extraction different features like binary area, centroid, peak calculation, angle calculation, thumb detection, finger region, or edge detection of the hand region are calculated. From the initial set, data is processed and hence builds derived values for the work of Feature extraction. Feature extraction is intended to be non-redundant and informative. It facilitates sequential learning with generalization and in some cases leads to better human interpretations. The raw and initial set of variables is reduced to groups (features) for processing that precisely describe the original set.
Reference	FR-4
Actor	System
Pre-requisites	1. Image needs to be High Resolution. 2. Hand is separated from surrounding region
Steps	The R-CNN algorithm extract the feature
Input	The result from AI Model
Expected Result	The AI model Extracted the feature from surrounding
Status	Tested, Passed

Table 18: Feature Extraction Test Case – Negative

Feature Extraction Test Case – Negative	
ID	TC_5.2
Priority	High
Description	For feature extraction different features like binary area, centroid, peak calculation, angle calculation, thumb detection, finger region, or edge detection of the hand region are calculated. From the initial set, data is processed and hence builds derived values for the work of Feature extraction. Feature extraction is intended to be non-redundant and informative. It facilitates sequential learning with generalization and in some cases leads to better human interpretations. The raw and initial set of variables is reduced to groups (features) for processing that precisely describe the original set.
Reference	FR-4
Actor	System
Pre-requisites	1. Image needs to be High Resolution. 2. Hand is separated from surrounding region
Steps	The R-CNN algorithm extract the feature
Input	The result from AI Model
Expected Result	The AI model did not Extracted the feature from surrounding
Status	Tested, Passed

5.1.6 Test Case for Feature Matching

Table 19: Feature Matching Test Case – Positive

Feature Matching Test Case – Positive	
ID	TC_6.1
Priority	High
Description	A classification system database is very important because that contains predefined sample patterns of the object under consideration that compare with the test object to classify its appropriate class. In a typical classification, the system image is captured by a camera and consequently processed. The neural network is one of the classification algorithms.
Reference	FR-5
Actor	System
Pre-requisites	4. If the feature is extracted perfectly. 5. All the previous algorithms work perfectly.
Steps	3. All models should be able to identify features perfectly. 4. The dataset should be trained with appropriate data pre-processing. 5. The system compares the captured hand signs with the existing dataset. 6. The label hand sign is compared one by one to look for a match.
Input	The result from AI Model

Expected Result	If the sign is recognized with the trained data set. Then the pattern match
Status	Tested, Passed

Table 20: Feature Matching Test Case – Negative

Feature Matching Test Case – Negative	
ID	TC_6.2
Priority	High
Description	A classification system database is very important because that contains predefined sample patterns of the object under consideration that compare with the test object to classify its appropriate class. In a typical classification, the system image is captured by a camera and consequently processed. The neural network is one of the classification algorithms.
Reference	FR-5
Actor	System
Pre-requisites	6. If the feature is extracted perfectly. 7. All the previous algorithms work perfectly.
Steps	7. All models should be able to identify features perfectly. 8. The dataset should be trained with appropriate data pre-processing. 9. The system compares the captured hand signs with the existing dataset.

	10. The label hand sign is compared one by one to look for a match.
Input	The result from AI Model
Expected Result	If the sign is not recognized with the trained data set. Then the pattern did not match
Status	Tested, Passed

5.1.7 Test Case for Pattern Recognition

Table 21: Pattern Recognition Test Case – Positive

Pattern Recognition Test Case – Positive	
ID	TC_7.1
Priority	High
Description	Making use of mathematical algorithms, intending to interpret human gestures is a topic of Gesture Recognition in the field of language technology and computer science. Mobile and computers are input devices that read input of gestures of the hand. Gestures are taken as input when the computer reads the movements of the body and sends them to the computer. The input is then interpreted by making use of artificial intelligence. The recognized word is converted to voice by making use of the Google audio built-in function.
Reference	FR-6
Actor	System

Pre-requisites	3. Model should be able to detect object. 4. AI model should be trained properly on multiple objects.
Steps	The AI model operate some operation from the previous getting information
Input	The result from AI Model
Expected Result	4. The system will run the captured frames from series of block and layer to extract features. 5. The system will predict the bounding boxes for the objects. 6. The system will label the identified objects.
Status	Tested, Passed

Table 22: Pattern Recognition Test Case – Negative

Pattern Recognition Test Case – Negative	
ID	TC_7. 2
Priority	High
Description	Making use of mathematical algorithms, intending to interpret human gestures is a topic of Gesture Recognition in the field of language technology and computer science. Mobile and computers are input devices that read input of gestures of the hand. Gestures are taken as input when the computer reads the movements of the body and sends them to

	the computer. The input is then interpreted by making use of artificial intelligence. The recognized word is converted to voice by making use of the Google audio built-in function.
Reference	FR-6
Actor	System
Pre-requisites	5. Model should be able to detect object. 6. AI model should be trained properly on multiple objects.
Steps	The AI model operate some operation from the previous getting information
Input	The result from AI Model
Expected Result	7. The system will not run the captured frames from series of block and layer to extract features. 8. The system will not predict the bounding boxes for the objects. 9. The system will not label the identified objects.
Status	Tested, Passed

5.2 Black Box Test Cases

5.2.1 Equivalence partitions (EP)

5.2.1.1 Image Capturing

Table 23: Equivalence partition (EP) for Image Capturing

Variable	Valid Classes	Invalid Classes
Lighting Conditions	Bright Light	Low Lighting
		Varied lighting
Distance	Close proximity	Far distance
		Moderate distance
Hand Orientation	Upright hand	Inverted hand
		Sideways hand
Background	Solid Background	Complex Background
		Textured background
Hand Shape	Open hand	Closed hand
		Partially open hand

5.2.1.2 Image Pre-processing

Table 24: Equivalence partition (EP) for Image Pre-processing

Variable	Valid Classes	Invalid Classes
Image Brightness	Adequate brightness	Low brightness
		High brightness
Image Orientation	Correct orientation	Incorrect orientation
		Upside-down orientation
Image Cropping	Appropriate cropping	Incorrect cropping
		Partial cropping

Image Scaling	Proper scaling	Improper scaling
		Uneven scaling
Image Rotation	Correct rotation	Incorrect rotation
		Excessive rotation
Hand Segmentation	Accurate hand segmentation	Inaccurate hand segmentation
		Missing hand segmentation

5.2.1.3 Region Extraction

Table 25: Equivalence partition (EP) for Region Extraction

Variable	Valid Classes	Invalid Classes
Hand Segmentation	Accurate hand segmentation	Inaccurate hand segmentation
		Missing hand segmentation
Hand Region Size	Appropriate hand region size	Oversized hand region
		Undersized hand region
Hand Region Shapes	Proper hand region shape	Distorted hand region shape
		Inappropriate hand contour
Background Removal	Clean hand region	Residual background noise
		Incomplete background removal

5.2.1.4 Feature Extraction

Table 26: Equivalence partition (EP) for Feature Extraction

Variable	Valid Classes	Invalid Classes
----------	---------------	-----------------

Edge Detection	Accurate edge detection	Inaccurate edge detection
		Missing edges
Gradient Orientation	Consistent gradient orientation	Inconsistent gradient orientation
		Misaligned gradients
Shape Descriptors	Distinctive shape feature	Ambiguous or indistinct shapes
		Missing shape information

5.2.1.5 Feature Matching

Table 27: Equivalence partition (EP) for Feature Matching

Variable	Valid Classes	Invalid Classes
Matching Threshold	Appropriate matching threshold	Inappropriate matching threshold
		Overly strict or lenient threshold
Inlier Selection	Accurate inlier selection	Incorrect inlier selection
		Missed or misclassified inliers
Geometric Verification	Accurate geometric verification	Incorrect geometric verification
		False positive or false negatives

5.2.1.6 Pattern Recognition

Table 28: Equivalence partition (EP) for Pattern Recognition

Variable	Valid Classes	Invalid Classes
Feature Selection	Discriminative feature selection	Irrelevant feature selection
		Inappropriate or noisy features
Similarity Measure	Accurate similarity measure	Inaccurate similarity measure
		Incorrect similarity calculation
Pattern Recognition	Correct pattern recognition	Incorrect pattern recognition
		False positives or false negatives

5.2.2 Boundary Value Analysis

5.2.2.1 Image Capturing

Table 29: Boundary Value Analysis for Image Capturing

Variable	Valid Classes	Invalid Classes
Lighting Conditions	Bright lighting	Low lighting
		Varied lighting
Distance	Optimal distance from the camera	Too close to the camera
		Too far from the camera

Hand Orientation	Proper hand orientation	Upside-down hand orientation
		Sideways hand orientation
Background	Clean and uncluttered background	Complex and cluttered background
		Distracting or patterned background
Hand Shape	Clear and well-defined hand shape	Ambiguous or distorted hand shape
		Missing or partially visible hand

5.2.2.2 Image Pre-processing

Table 30: Boundary Value Analysis for Image Pre-processing

Variable	Valid Classes	Invalid Classes
Image Brightness	Adequate brightness	Low brightness
		Very high brightness
Image Orientation	Correct image orientation	Incorrect image orientation
		Upside-down image
Image Cropping	Appropriate cropping	Insufficient cropping
		Excessive cropping
Image Scaling	Proper scaling	Improper scaling
		Enlarged image
Image Rotation	Correct rotation	Incorrect rotation
		Excessive rotation
Hand Segmentation	Accurate hand segmentation	Inaccurate hand segmentation

		No hand segmentation
--	--	----------------------

5.2.2.3 Region Extraction

Table 31: Boundary Value Analysis for Region Extraction

Variable	Valid Classes	Invalid Classes
Hand Segmentation	Accurate hand segmentation	Inaccurate hand segmentation
		No hand segmentation
Region Size	Appropriate region size	Undersized region
		Oversized region
Region Shape	Proper region shape	Distorted region shape
		Irregular region shape
Background Removal	Clean and accurate removal	Residual background
		Incomplete removal

5.2.2.4 Feature Extraction

Table 32: Boundary Value Analysis for Feature Extraction

Variable	Valid Classes	Invalid Classes
Feature Type	Relevant feature type	Irrelevant feature type
		Missing or incomplete features
Feature Extraction Method	Effective feature extraction method	Ineffective feature extraction method
		Incorrect feature extraction algorithm
Feature Representation	Discriminative feature representation	Inaccurate feature representation

		Noisy or distorted features
--	--	-----------------------------

5.2.2.5 Feature Matching

Table 33: Boundary Value Analysis for Feature Matching

Variable	Valid Classes	Invalid Classes
Matching Threshold	Appropriate matching threshold	Inappropriate matching threshold
		Overly strict or lenient threshold
Inlier Selection	Accurate inlier selection	Incorrect inlier selection
		Missed or misclassified inliers
Geometric Verification	Accurate geometric verification	Incorrect geometric verification
		False positives or false negatives

5.2.2.6 Pattern Matching

Table 34: Boundary Value Analysis for Pattern Matching

Variable	Valid Classes	Invalid Classes
Feature Matching	Accurate feature matching	Inaccurate feature matching
		Feature mismatch or false positives
Matching Threshold	Appropriate matching threshold	Inappropriate matching threshold

		Overly strict or lenient threshold
Scale Invariance	Robust to scale variations	Sensitive to scale variations

5.2.3 Decision Table Testing

5.2.3.1 Image Capturing

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
Background Interference	Minimal interference	No significant interference with the hand	No significant interference with the hand	No significant interference with the hand	No significant interference with the hand	Unobstructed background
	Moderate interference	Partial interference with the hand	Partial interference with the hand	Partial interference with the hand	Partial interference with the hand	Partial obstruction in the background
	High interference	Severe interference with the hand	Severe interference with the hand	Severe interference with the hand	Severe interference with the hand	Significant obstruction in the background
Hand Position and Pose	Proper hand position and pose	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Correct hand position and pose
	Partially occluded hand	Hand is partially occluded	Hand is partially occluded	Hand is partially occluded	Hand is partially occluded	Partial obstruction of the hand

	Incorrect hand position or pose	Hand position or pose is incorrect	Hand position or pose is incorrect	Hand position or pose is incorrect	Hand position or pose is incorrect	Incorrect hand position or pose
Focused and Clear Capture	Focused and clear image	Image is focused and clear	Image is focused and clear	Image is focused and clear	Image is focused and clear	Clear and sharp image
	Slightly blurred image	Image is slightly blurred	Image is slightly blurred	Image is slightly blurred	Image is slightly blurred	Slightly blurred image
	Blurred or out-of-focus image	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Blurred or out-of-focus image

Table 35: Decision Table for Image Capturing

5.2.3.2 Image Pre-processing

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
Background Interference	Minimal interference	No significant interference with the hand	No significant interference with the hand	No significant interference with the hand	No significant interference with the hand	Unobstructed background
	Moderate interference	Partial interference with the hand	Partial interference with the hand	Partial interference with the hand	Partial interference with the hand	Partial obstruction in the background

	High interference	Severe interference with the hand	Severe interference with the hand	Severe interference with the hand	Severe interference with the hand	Significant obstruction in the background
Hand Position and Pose	Proper hand position and pose	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Hand is positioned and posed correctly	Correct hand position and pose
	Partially occluded hand	Hand is partially occluded	Hand is partially occluded	Hand is partially occluded	Hand is partially occluded	Partial obstruction of the hand
	Incorrect hand position or pose	Hand position or pose is incorrect	Hand position or pose is incorrect	Hand position or pose is incorrect	Hand position or pose is incorrect	Incorrect hand position or pose
Focused and Clear Capture	Focused and clear image	Image is focused and clear	Image is focused and clear	Image is focused and clear	Image is focused and clear	Clear and sharp image
	Slightly blurred image	Image is slightly blurred	Image is slightly blurred	Image is slightly blurred	Image is slightly blurred	Slightly blurred image
	Blurred or out-of-focus image	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Image is blurred or out-of-focus	Blurred or out-of-focus image

Table 36: Decision Table for Image Pre-processing

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
Image Rescaling	Low rescaling factor	Case-specific downscaled image	Case-specific downscaled image	Case-specific downscaled image	Case-specific downscaled image	Reduced image size
	High rescaling factor	Case-specific up scaled image	Case-specific up scaled image	Case-specific up scaled image	Case-specific up scaled image	Increased image size
	Optimal rescaling factor	Case-specific rescaled image	Case-specific rescaled image	Case-specific rescaled image	Case-specific rescaled image	Properly resized image
Edge Detection	No edge detection	Image without detected edges	Image without detected edges	Image without detected edges	Image without detected edges	Unchanged edge information

5.2.3.3 Region Extraction

Table 37: Decision Table for Region Extraction

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
Image Segmentation	Threshold-based segmentation	Segmented regions based on threshold	Segmented regions based on threshold	Segmented regions based on threshold	Segmented regions based on threshold	Extracted regions based on threshold segmentation

	Color-based segmentation	Segmented regions based on color range	Segmented regions based on color range	Segmented regions based on color range	Segmented regions based on color range	Extracted regions based on color-based segmentation
	Edge-based segmentation	Segmented regions based on detected edges	Segmented regions based on detected edges	Segmented regions based on detected edges	Segmented regions based on detected edges	Extracted regions based on edge-based segmentation
Feature Extraction	Contour-based features	Extracted contour-based features	Extracted contour-based features	Extracted contour-based features	Extracted contour-based features	Extracted contour-based features
	Histogram-based features	Extracted histogram-based features	Extracted histogram-based features	Extracted histogram-based features	Extracted histogram-based features	Extracted histogram-based features
	Texture-based features	Extracted texture-based features	Extracted texture-based features	Extracted texture-based features	Extracted texture-based features	Extracted texture-based features

5.2.3.4 Feature Extraction

Table 38: Decision Table for Feature Extraction

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
------------	--------	--------	--------	--------	--------	--------

Shape-based Features	Yes	Extracted shape-based features	Extracted shape-based features	Extracted shape-based features	Extracted shape-based features	Extracted shape-based features
	No	No shape-based features extracted	No shape-based features extracted	No shape-based features extracted	No shape-based features extracted	No shape-based features extracted
Motion-based Features	Yes	Extracted motion-based features	Extracted motion-based features	Extracted motion-based features	Extracted motion-based features	Extracted motion-based features
	No	No motion-based features extracted	No motion-based features extracted	No motion-based features extracted	No motion-based features extracted	No motion-based features extracted

5.2.3.5 Feature Matching

Table 39: Decision Table for Feature Matching

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
Shape Matching	Yes	Shape match found	Shape match found	Shape match found	Shape match found	Hand sign shape matched
	No	Shape match not found	Shape match not found	Shape match not found	Shape match not found	Hand sign shape not matched

Motion Matching	Yes	Motion match found	Motion match found	Motion match found	Motion match found	Hand sign motion matched
	No	Motion match not found	Motion match not found	Motion match not found	Motion match not found	Hand sign motion not matched

5.2.3.6 Pattern Recognition

Table 40: Decision Table for Pattern Recognition

Conditions	Values	Case-1	Case-2	Case-3	Case-4	Effect
template Matching	Yes	Template match found	Template match found	Template match found	Template match found	Hand sign pattern matched
	No	Template match not found	Template match not found	Template match not found	Template match not found	Hand sign pattern not matched
Neural Network	Yes	Neural network match found	Neural network match found	Neural network match found	Neural network match found	Hand sign pattern matched

	No	Neural network match not found	Neural network match not found	Neural network match not found	Neural network match not found	Hand sign pattern not matched
--	----	--------------------------------	--------------------------------	--------------------------------	--------------------------------	-------------------------------

5.2.4 State transition Testing

5.2.4.1 Image Capturing

Table 41: State Transition Testing for Image Capturing

State	Event/Action	Next State
Idle	Hand is not moving	Ready
Ready	Hand moves above a threshold velocity	Capturing
Capturing	Hand is still moving above the threshold velocity	Capturing
Capturing	Hand stops moving above the threshold velocity	Done
Done	No new hand movement detected for a timeout period	Idle

5.2.4.2 Image Preprocessing

Table 42: State Transition Testing for Image Preprocessing

State	Event/Action	Next state
Idle	Image Captured	Preprocessing
Preprocessing	Image preprocessing complete	Classification

Classification	Hand sign recognized	Done
Classification	Hand sign not recognized	Re-preprocessing
Re-preprocessing	Image preprocessing complete	Classification

5.2.4.3 Region Extraction

Table 43: State Transition Testing for Region Extraction

State	Event/Action	Next state
Idle	Image captured	Region Extraction
Region Extraction	Hand region extracted	Classification
Region Extraction	Hand region not extracted	Re-extraction
Re-extraction	Hand region extracted	Classification

5.2.4.4 Feature Extraction

Table 44: State Transition Testing for Feature Extraction

State	Event/Action	Next State
Idle	Image captured	Feature Extraction
Feature Extraction	Features extracted	Classification
Feature Extraction	Features not extracted	Re-extraction
Re-extraction	Features extracted	Classification

5.2.4.5 Feature Matching

Table 45: State Transition Testing for Feature Matching

State	Event/Action	Next State
Idle	Features extracted	Feature Matching
Feature Matching	Match found	Classification
Feature Matching	Match not found	Re-matching
Re-matching	Match found	Classification
Re-matching	Match not found	Fail

5.2.4.6 Pattern Recognition

Table 46: State Transition Testing for Pattern Matching

State	Event/Action	Next State
Idle	Features extracted	Pattern Matching
Pattern Matching	Pattern found	Classification
Pattern Matching	Pattern not found	Re-matching
Re-matching	Pattern found	Classification
Re-matching	Pattern not found	Fail

5.2.5 Use Case Testing

5.2.5.1 Image Capturing

Table 47: Testing Use Case for Image Capturing

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to capture images of hand gestures in a variety of lighting conditions.	UC-01	TC-01	The system captures an image of a hand gesture in a dark room.	User	The system should be able to capture images of hand gestures in a variety of lighting conditions, such as different brightness levels and color temperatures.
The system should be able to capture images of hand gestures in a variety of poses and orientations.	UC-02	TC-02	The system captures an image of a hand gesture with the fingers spread apart.	User	The system should be able to capture images of hand gestures in a variety of poses and orientations,

					such as different hand shapes and positions.
The system should be able to capture images of hand gestures at a variety of resolutions	UC-03	TC-03	The system captures an image of a hand gesture at a resolution of 100x100 pixels.	User	The system should be able to capture images of hand gestures at a variety of resolutions, such as different widths, heights, and aspect ratios.

5.2.5.2 Image Pre-processing

Table 48: Testing Use Case for Image Pre-processing

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to				User	The system should be able to remove

remove noise from images of hand gestures.	UC-01	TC-01	The system takes an image of a hand gesture as input and outputs a noise-free image		noise from images of hand gestures in a variety of environmental conditions, such as different lighting levels and backgrounds.
The system should be able to segment hand gestures from background images.	UC-02	TC-02	The system takes an image of a hand gesture and a background image as input and outputs an image of the hand gesture with the background removed.	User	The system should be able to segment hand gestures from background images in a variety of poses and orientations.
The system should be able to normalize images of hand gestures.	UC-03	TC-03	The system takes an image of a hand gesture as input and outputs an image of the hand gesture	User	The system should be able to normalize images of hand gestures in a variety of resolutions

			that has been normalized to a standard size and scale.		and aspect ratios.
--	--	--	--	--	--------------------

5.2.5.3 Region Extraction

Table 49: Testing Use Case for Region Extraction

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to extract the region of interest (ROI) from an image of a hand gesture.	UC-01	TC-01	The system takes an image of a hand gesture as input and outputs the ROI of the hand gesture.	User	The system should be able to extract the ROI of the hand gesture in a variety of poses and orientations.
The system should be able to extract the ROI of the hand gesture in real time.	UC-02	TC-02	The system takes a live video feed of a hand gesture as input and outputs the ROI of the hand gesture in real time.	User	The system should be able to extract the ROI of the hand gesture in real time even if the hand gesture is moving quickly.
The system should be able to extract	UC-03	TC-03	The system takes an image of a hand	User	The system should be able to extract the

the ROI of the hand gesture from a variety of backgrounds.			gesture with a variety of backgrounds as input and outputs the ROI of the hand gesture.		ROI of the hand gesture from a variety of backgrounds, such as different colors, textures, and patterns.
--	--	--	---	--	--

5.2.5.4 Feature Extraction

Table 50: Testing Use Case for Feature Extraction

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to extract features from the ROI of a hand gesture.	UC-07	TC-07	The system takes the ROI of a hand gesture as input and outputs a set of features that describe the hand gesture.	User	The system should be able to extract features from the ROI of a hand gesture in a variety of poses and orientations.
The system should be able to extract features from the ROI of a hand gesture in real time.	UC-08	TC-08	The system takes a live video feed of a hand gesture as input and outputs a set of features that describe the hand gesture in real time.	User	The system should be able to extract features from the ROI of a hand gesture in real time even if the hand gesture is moving quickly.
The system should be able to extract features from the ROI of a hand gesture from a	UC-09	TC-09	The system takes the ROI of a hand gesture with a variety of backgrounds as input and outputs a	User	The system should be able to extract features from the ROI of a hand gesture from a variety of backgrounds, such as

variety of backgrounds.			set of features that describe the hand gesture.		different colors, textures, and patterns.
-------------------------	--	--	---	--	---

5.2.5.5 Feature Matching

Table 51: Testing Use Case for Feature Matching

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to match features from a new hand gesture to a set of known hand gestures.	UC-10	TC-10	The system takes the features of a new hand gesture as input and outputs the most likely match from a set of known hand gestures.	User	The system should be able to match features from a new hand gesture to a set of known hand gestures in a variety of poses and orientations.
The system should be able to match features from a new hand gesture to a set of known hand gestures in real time.	UC-11	TC-11	The system takes the features of a new hand gesture as input and outputs the most likely match from a set of known hand gestures in real time.	User	The system should be able to match features from a new hand gesture to a set of known hand gestures in real time even if the hand gesture is moving quickly.
The system should be able to match features from a new hand gesture to a set of known hand gestures from a variety of backgrounds.	UC-12	TC-12	The system takes the features of a new hand gesture with a variety of backgrounds as input and outputs the most likely match from a	User	The system should be able to match features from a new hand gesture to a set of known hand gestures from a variety of backgrounds, such as different colors, textures, and patterns.

			set of known hand gestures.		
--	--	--	-----------------------------	--	--

5.2.5.6 Pattern Recognition

Table 52: Testing Use Case for Pattern Recognition

Description	Use Case ID	Test Case ID	Main Scenario	Actor/Users	Extension
The system should be able to recognize a hand gesture based on the features extracted from the hand gesture.	UC-13	TC-13	The system takes the features of a new hand gesture as input and outputs the most likely hand gesture from a set of known hand gestures.	User	The system should be able to recognize a hand gesture based on the features extracted from the hand gesture in a variety of poses and orientations.
The system should be able to recognize a hand gesture based on the features extracted from the hand gesture in real time.	UC-14	TC-14	The system takes the features of a new hand gesture as input and outputs the most likely hand gesture from a set of known hand gestures in real time.	User	The system should be able to recognize a hand gesture based on the features extracted from the hand gesture in real time even if the hand gesture is moving quickly.
The system should be able to recognize a hand gesture based on the features extracted from the hand gesture from a	UC-15	TC-15	The system takes the features of a new hand gesture with a variety of backgrounds as input and outputs the most likely hand	User	The system should be able to recognize a hand gesture based on the features extracted from the hand gesture from a variety of backgrounds,

variety of			gesture from a set of known hand gestures.		such as different colors, textures, and patterns.
------------	--	--	--	--	---

5.3 White Box Testing

White box testing is a software testing technique that involves examining the internal structure and workings of a software application. Also known as clear box testing or structural testing, white box testing focuses on verifying the code and logic of the software. During white box testing, testers use their knowledge of the internal structure of the software to design and execute tests that verify that each line of code is functioning as intended. This can include testing individual functions, checking for the correct use of variables, verifying the flow of control statements, and more.

5.3.1 Cyclometric complexity

Cyclometric complexity is software metric that is used to measure the complexity of a program's control flow. It is a quantitative measure of the number of independent paths that can be executed through a program's source code.

5.3.1.1 Cyclometric Complexity for Image capturing

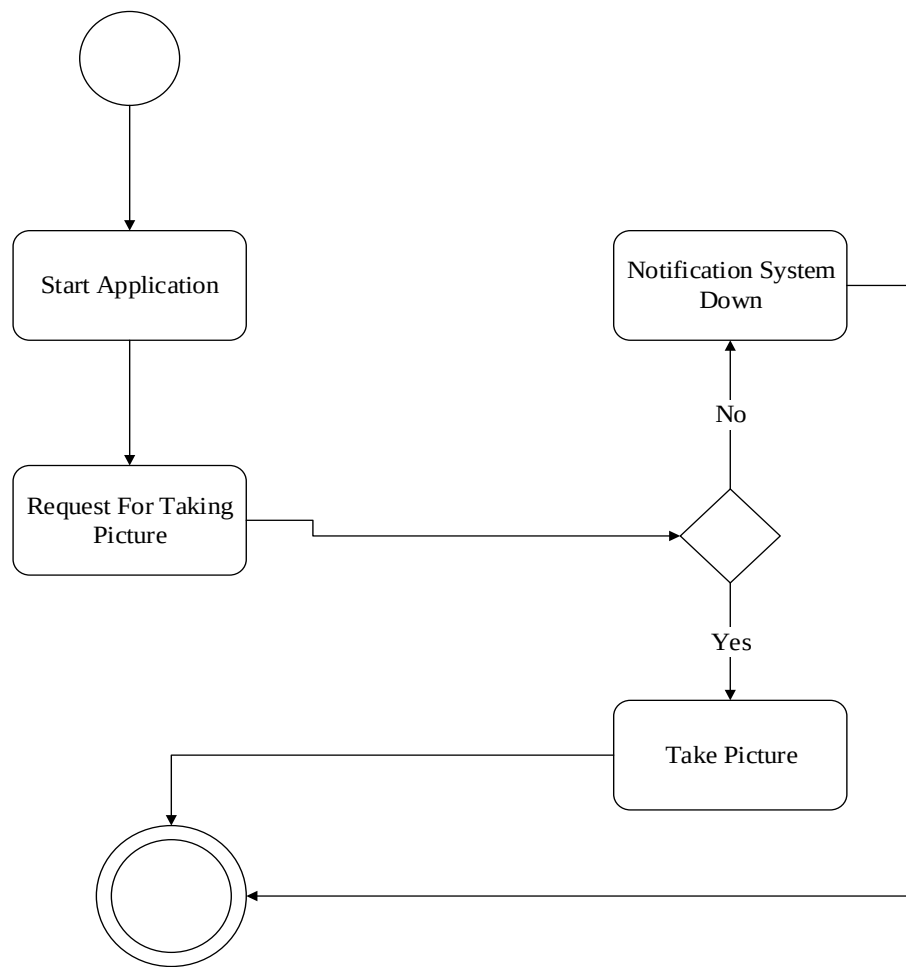


Figure 39: Cyclometric Complexity for Image capturing

Formula: $M = E - N + 2P$, where

E = no of edges = 8

N = no of nodes = 7

$M = 8 - 7 + 2(1)$

$M = 3$

5.3.1.2 Cyclometric Complexity for Image Pre-Processing

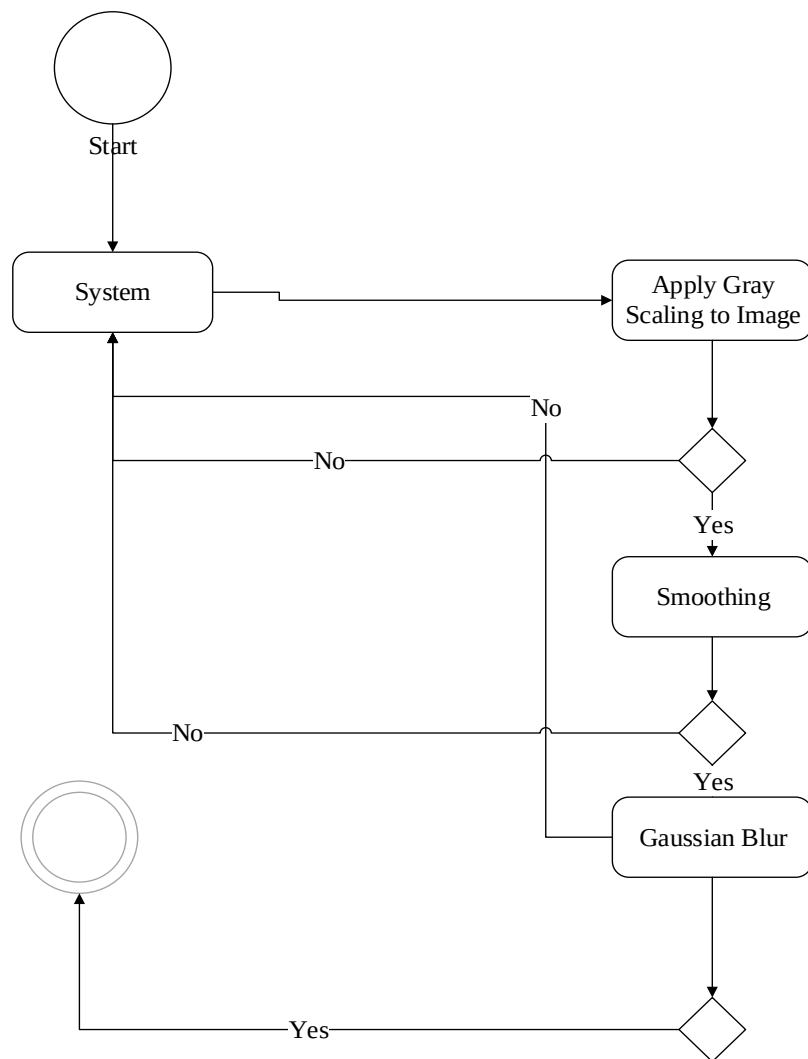


Figure 40: Cyclometric Complexity for Image Pre-Processing

Formula: $M = E - N + 2P$, where

E = no of edges = 10

N = no of nodes = 9

$M = 10 - 9 + 2(1)$

$M = 3$

5.3.1.3 Cyclometric Complexity for Region Extraction

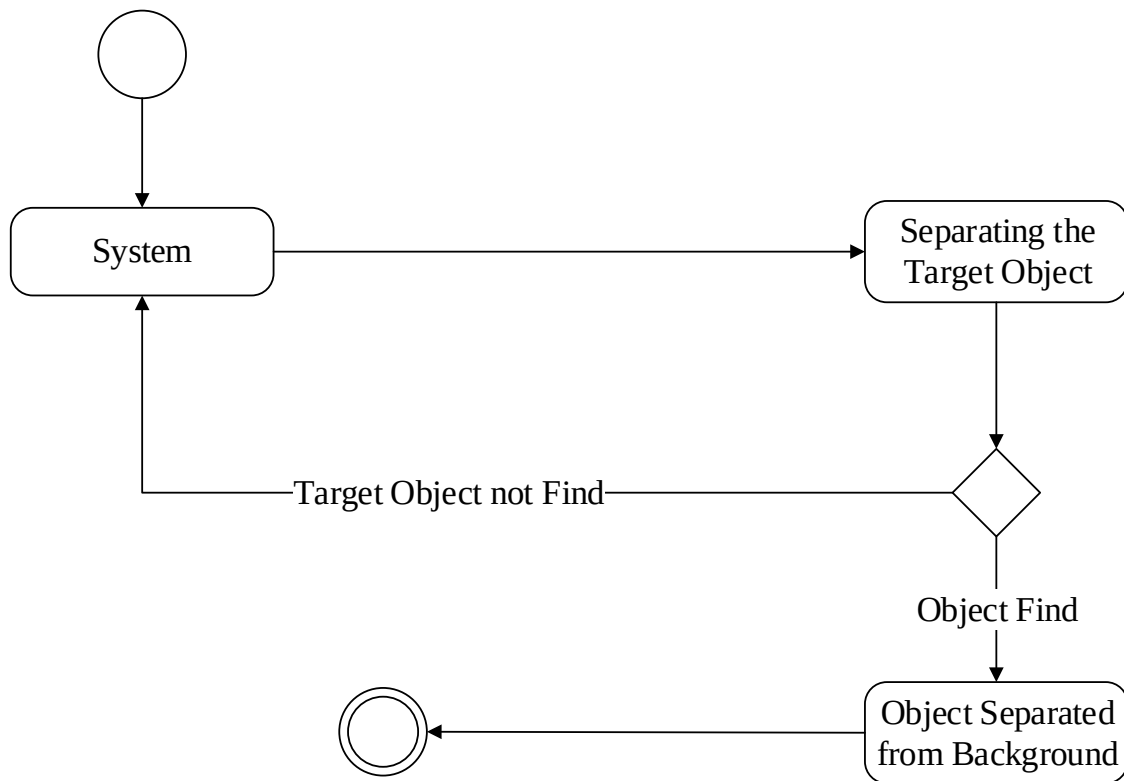


Figure 41: Cyclometric Complexity for Region Extraction

Formula: $M = E - N + 2P$, where

E = no of edges = 6

N = no of nodes = 6

$M = 6 - 6 + 2(1)$

$M = 2$

5.3.1.4 Cyclometric Complexity for Feature Extraction

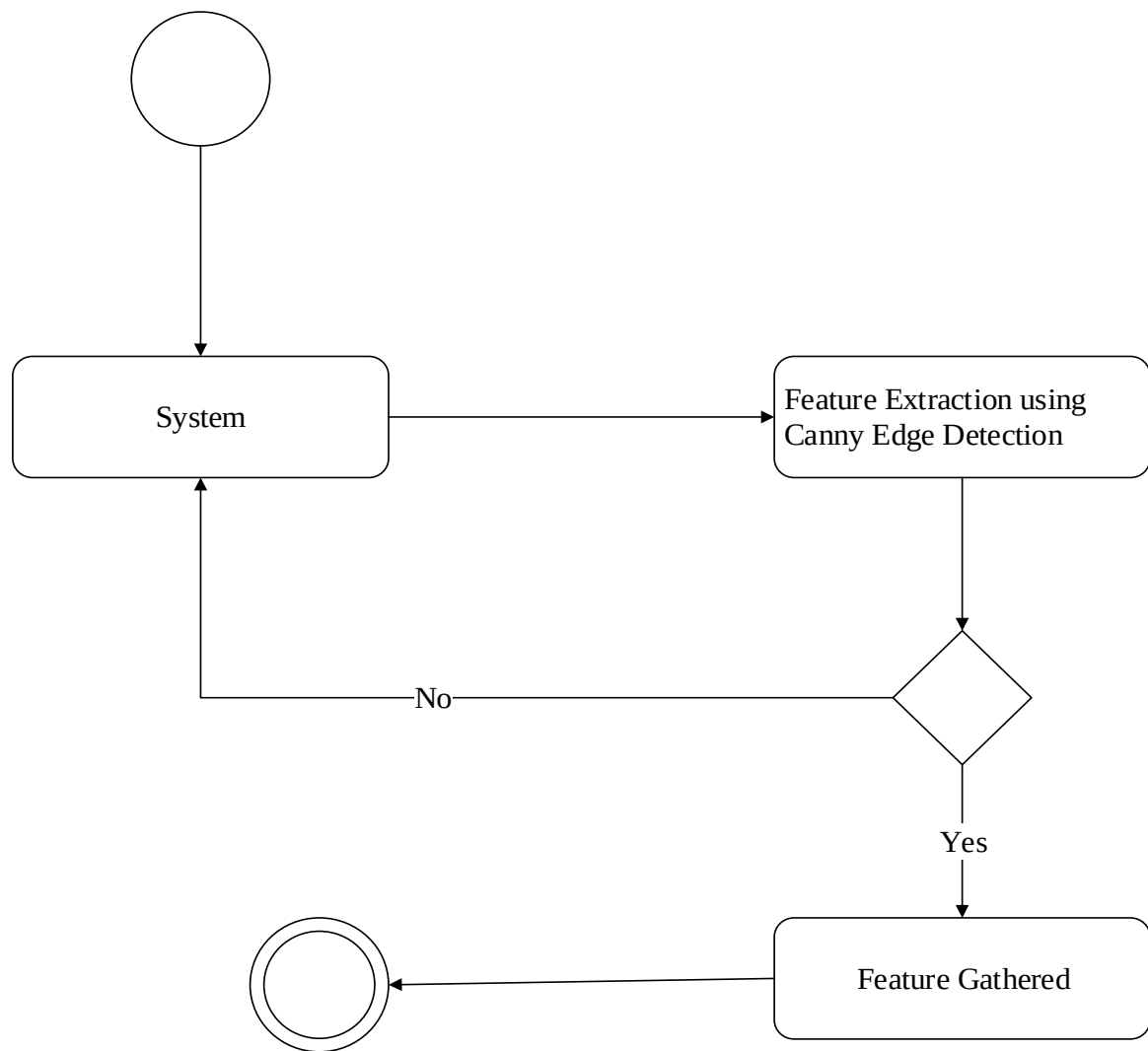


Figure 42: Cyclometric Complexity Feature Extraction

Formula: $M = E - N + 2P$, where

E = no of edges = 6

N = no of nodes = 6

$M = 6 - 6 + 2(1)$

$M = 2$

5.3.1.5 Cyclometric Complexity for Feature Matching

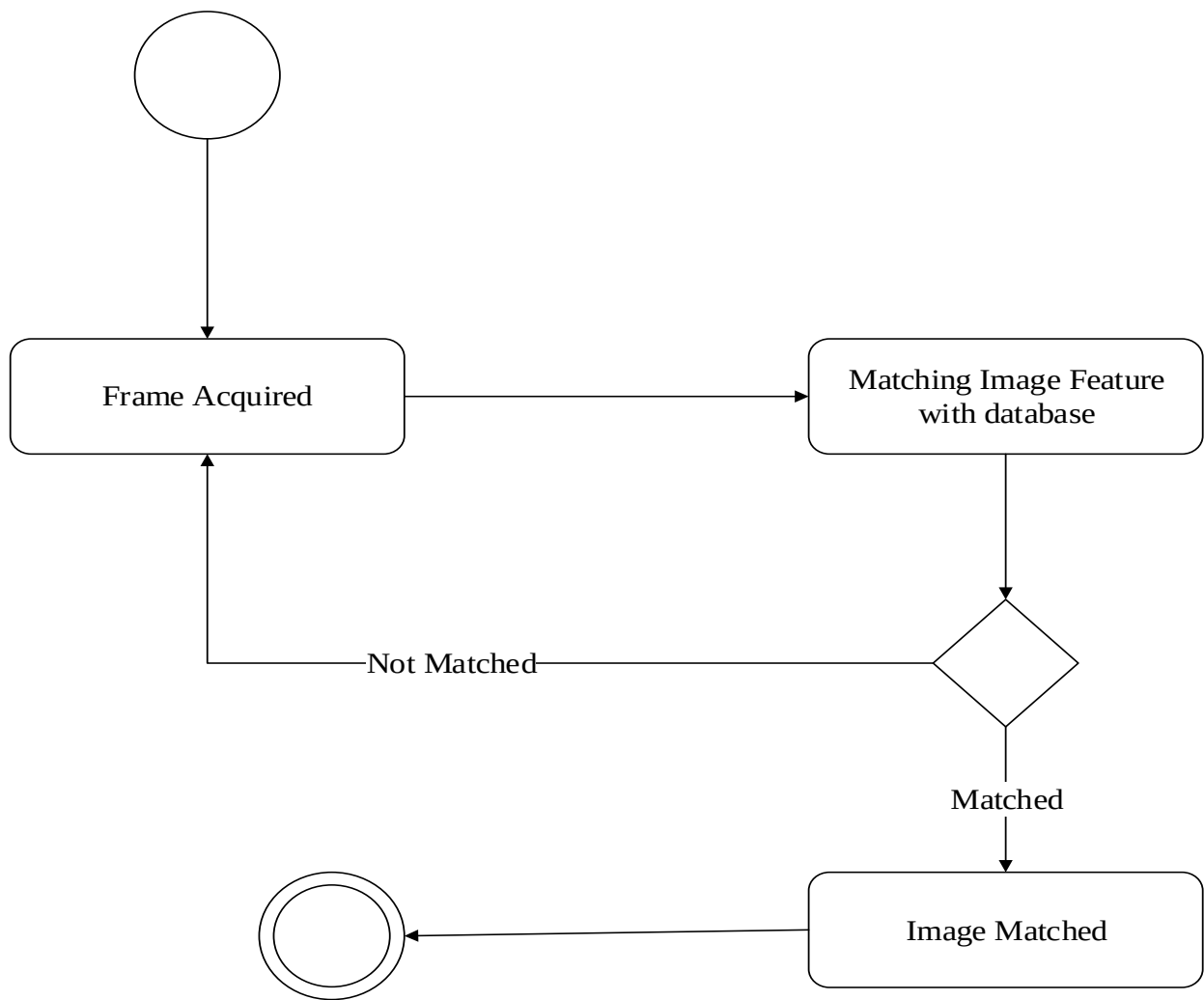


Figure 43: Cyclometric Complexity Feature Matching

Formula: $M = E - N + 2P$, where

E = no of edges = 6

N = no of nodes = 6

$M = 6 - 6 + 2(1)$

$M = 2$

5.3.1.6 Cyclometric Complexity for Pattern Recognition

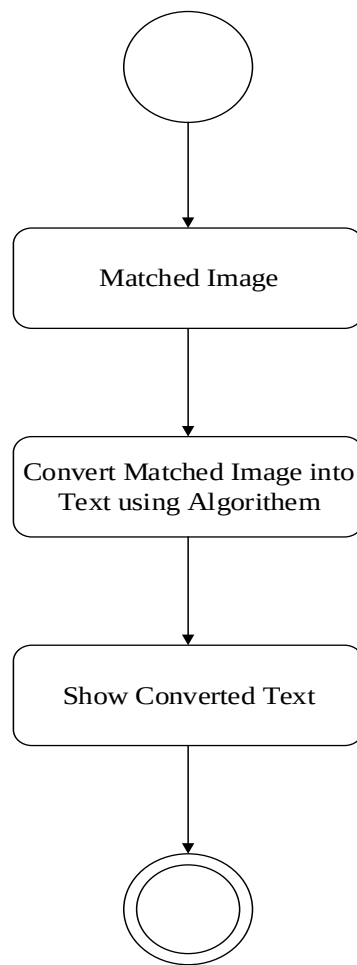


Figure 44: Cyclometric Complexity Pattern Recognition

Formula: $M = E - N + 2P$, where

E = no of edges = 4

N = no of nodes = 5

$M = 4 - 5 + 2(1)$

$M = 1$

5.4 Performance Testing

As we already know, Performance Testing is all about testing and evaluation of speed, responsiveness and stability of system. In Performance Testing, we tested the speed, accuracy

and stability of Text-To-Speech (TTS), Voice Recognition and Speech Recognition of our system. In the case of our project, performance testing can help ensure that the website performs efficiently and reliably for users, including visually impaired or blind individuals.

5.5 Stress Testing

The system has been deliberately tested in intense and harsh environment to be able to determine its stability when great number of users are using it simultaneously. It assesses how the website handles high loads, heavy traffic, or resource limitations and checks if it degrades gracefully without crashing or becoming unresponsive. Stress Testing also involved testing beyond normal operational capacity, in order to observe results. The results were accurate and system worked without any pinch.

5.6 System Testing

As we know it already, System Testing (ST) is a Black Box Testing Technique for complete evaluation of entire system against specified requirements. In this testing, the functionalities of the system are tested from an end-to-end perspective. After performing System Testing, carried by our team mates who are independent of development, we are now ensured that our System works smoothly without any issues or problem faced.

5.7 Regression Testing

In Regression Testing, System was checked against the boundary conditions to see if it works. Algorithms works correctly, application doesn't behave abnormally. Some changes were made in order to make the system work more effectively and smoothly, and it is to ensure that the system works effectively as well as efficiently after further improvements, updates and changes.

Here are few examples of Regression Testing of our project:

Table 53: Regression Testing Table

Test Case ID	Description	Scope	Priority	Automation Status	Regression Status
TC001	Image capturing	Overall system functionality	High	Automated	Passed
TC002	Image pre-processing	Overall system functionality	High	Automated	Passed
TC003	Region extraction	Overall system functionality	High	Automated	Passed
TC004	Feature extraction	Overall system functionality	High	Automated	Passed
TC005	Feature matching	Overall system functionality	High	Automated	Passed

TC006	Pattern recognition	Overall system functionality	High	Automated	Passed
-------	---------------------	------------------------------	------	-----------	--------

This table can be expanded further based on the specific requirements and functionalities of your project. The regression status should be updated after each regression testing cycle to reflect the current state of the test cases.

Types of Regression Tests: The “Regression Tests” are executed to verify if recent code changes for defect fixing or enhancement have not broken any other parts of the application. These tests include functionalities like updating news articles, deleting user accounts, verifying news articles, resetting user passwords, and changing user account passwords.

Final Regression Tests: For the “Final Regression Tests,” these tests are performed to validate the build that hasn’t changed for a period of time before it is deployed or shipped to customers. These tests cover critical functionalities like login, user registration, news article creation, user profile editing, and news search.

- Test Case 1: Basic Hand Sign Recognition Testing
- Test Case 2: Complex Hand Sign Recognition Testing
- Test Case 3: Real-Time Recognition Testing
- Test Case 4: Robustness and Error Handling

Regression Tests: The “Regression Tests” are executed to verify if recent code changes for defect fixing or enhancement have not broken any other parts of the application. These tests include functionalities like updating news articles, deleting user accounts, verifying news articles, resetting user passwords, and changing user account passwords

- Test Case 5: Performance and Responsiveness Testing
- Test Case 6: Integration and Compatibility Testing
- Test Case 7: User Interface and User Experience Testing

We can expand the list of test cases based on the specific functionalities and areas of our project that require regression testing. Each test case should have a clear description and be executed

regularly throughout the software development life cycle to ensure the stability and integrity of the application.

5.7.1 Selecting Regression Test

1. Requires knowledge about the system and how it affects existing functionalities:

- Test Case 1: Image acquisition module
- Test Case 2: image processing

2. Tests are selected based on the area of frequent defects:

- Test Case 3: Complex Hand Sign Recognition
- Test Case 4: Real-Time Recognition

3. Tests are selected to include areas that have undergone code changes frequently:

- Test Case 5: Image Acquisition
- Test Case 6: Image Processing

4. Tests are selected based on the criticality of the features:

- Test Case 16: Accurate Hand Sign Recognition
- Test Case 17: Handling Complex Gestures
- Test Case 18: Lighting and Background Variations

These test cases cover various aspects of our project, including core functionalities, areas with frequent defects, code change-prone areas, and critical features. We can further expand the list and tailor it to match our specific project requirements and priorities.

5.7.2 Regression Testing Steps

1. Select the Tests for Regression:

- Review the existing test cases and identify the tests that are suitable for regression testing.
- Focus on areas that have undergone recent code changes, frequent defects, or critical features.

2. Choose the apt tool and automate the Regression Tests:

- Evaluate and select a suitable automation tool that meets your project requirements.
- Use the chosen tool to automate the selected regression test cases.
- Develop automation scripts or test cases to execute the regression tests.

3. Verify applications with Checkpoints:

- Define checkpoints or expected outcomes for each regression test case.
 - Execute the automated regression tests and compare the actual results with the expected checkpoints.
 - Identify any deviations or failures.
- 4. Manage Regression Tests/update when required:**
- Maintain a regression test suite that includes all relevant test cases.
 - Review and update the test cases as needed to align with the changes in the application.
- 5. Schedule the tests:**
- Plan and schedule the execution of regression tests at appropriate intervals.
 - Consider executing regression tests after each code change, build, or deployment.
- 6. Analyze the results:**
- Analyze the test results from regression testing.
 - Identify any regressions, failures, or inconsistencies introduced by recent code changes.

Chapter 6: Tools and Techniques

Title	Outline
Tools and Techniques	Tool Used

6.1 Tools Used

6.1.1 Python

Python is a high-level, interpreted programming language that is widely used for various applications. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and simplicity, making it a popular choice among beginners and experienced programmers alike. Some key features of Python include:

1. **Easy-to-read syntax:** Python uses a clean and straightforward syntax that resembles natural language, making it easy to write and understand code.
2. **Interpretation:** Python code is executed line by line by an interpreter, which eliminates the need for compiling before running the code. This allows for rapid development and experimentation.
3. **Multi-paradigm:** Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming styles. This flexibility makes it suitable for various types of projects.
4. **Extensive standard library:** Python comes with a vast standard library that provides ready-to-use modules and functions for performing a wide range of tasks, from file operations to network programming.
5. **Large community and ecosystem:** Python has a large and active community of developers who contribute to the language's growth. There are numerous third-party libraries and frameworks available for Python, expanding its capabilities and making it applicable to different domains such as web development, data analysis, scientific computing, artificial intelligence, and more.

Python is known for its simplicity and readability, which contribute to its ease of learning and quick development. It has gained popularity in various fields due to its versatility, making it a go-to language for many programmers.

6.1.2 Jupiter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and explanatory text. It provides an interactive computing environment that supports various programming languages, including Python, R, Julia, and more. The name "Jupyter" is derived from the combination of three core programming languages it supports: Julia, Python, and R. Jupyter Notebook was originally based on the IPython project, which provided an interactive shell for Python. Over time, it evolved into a standalone application that supports multiple languages and rich multimedia capabilities. The key features of Jupyter Notebook include:

1. **Notebook interface:** Jupyter Notebook presents a web-based interface where you can create and edit notebooks. Notebooks are organized into cells that can contain code, text, mathematical equations, visualizations, and more.
2. **Code execution:** You can write and execute code directly in the notebook cells. Each cell can be executed independently, allowing for an interactive and iterative development process.
3. **Rich media support:** Jupyter Notebook enables the integration of multimedia elements such as images, videos, and interactive plots alongside code and text.
4. **Markdown and LaTeX support:** You can use Markdown syntax to format text and add headings, lists, tables, and more. Additionally, you can include mathematical equations using LaTeX notation.
5. **Data exploration and visualization:** Jupyter Notebook integrates well with data analysis and visualization libraries such as Pandas, NumPy, Matplotlib, and Seaborn, allowing you to explore data and create interactive visualizations within the notebook.
6. **Collaboration and sharing:** Notebooks can be easily shared with others, making it convenient for collaborative work. They can be exported to various formats, including HTML, PDF, and slideshows.

Jupyter Notebook has gained popularity among data scientists, researchers, educators, and developers due to its interactive and exploratory nature. It provides an effective environment for data analysis, prototyping, and sharing computational workflows.

6.1.3 Android studio

Android Studio is the official integrated development environment (IDE) for developing Android applications. It is a powerful tool that provides a comprehensive environment for Android app development, including coding, debugging, testing, and deploying applications. Android Studio is based on IntelliJ IDEA, a popular Java IDE, and is specifically designed for building Android apps. It offers a range of features and tools that simplify the development process and enhance productivity. Some key features of Android Studio include:

1. **Code Editor:** Android Studio provides a code editor with advanced features like syntax highlighting, code completion, refactoring, and code analysis. It supports multiple programming languages, including Java and Kotlin, which are commonly used for Android app development.
2. **Layout Editor:** Android Studio includes a visual editor that allows developers to design app layouts using a drag-and-drop interface. It provides a WYSIWYG (What You See Is What You Get) experience, making it easier to create user interfaces.
3. **Android Virtual Device (AVD) Manager:** AVD Manager is a tool within Android Studio that enables developers to create and manage virtual Android devices for testing apps on various screen sizes, hardware configurations, and Android versions.
4. **Gradle Build System:** Android Studio uses Gradle as the build system, which automates the process of building, testing, and packaging Android apps. Gradle simplifies dependency management and enables customizing build configurations.
5. **Debugger and Profiler:** Android Studio includes a debugger that allows developers to analyze and debug their code during runtime. It also provides a profiler to measure app performance, identify bottlenecks, and optimize resource usage.
6. **Testing and Emulation:** Android Studio offers built-in testing frameworks for unit testing, integration testing, and UI testing. It also provides tools for emulating device sensors and network conditions to simulate real-world scenarios.
7. **Integration with Firebase and Google Cloud Platform:** Android Studio integrates seamlessly with Firebase and Google Cloud Platform, providing developers with access to a

wide range of services for app development, such as authentication, database, storage, and analytics.

Android Studio provides a robust and feature-rich environment for developing Android applications, catering to both beginners and experienced developers. It offers a wealth of resources, documentation, and a supportive community, making it the go-to IDE for Android app development.

6.1.4 Open CV

OpenCV (Open-Source Computer Vision Library) is an open-source computer vision and machine learning software library. It provides a wide range of tools and algorithms for image and video processing, object detection and tracking, facial recognition, and more. OpenCV is written in C++ and has bindings for various programming languages, including Python. Here are some key features and functionalities of OpenCV:

- 1. Image and Video Processing:** OpenCV offers a rich set of functions for reading, manipulating, and processing images and videos. It provides tools for basic operations like resizing, cropping, and rotating images, as well as advanced functionalities like filtering, morphological operations, and histogram equalization.
- 2. Object Detection and Tracking:** OpenCV includes pre-trained models and algorithms for detecting and tracking objects in images and videos. It supports techniques such as Haar cascades, HOG (Histogram of Oriented Gradients), and deep learning-based approaches like YOLO (You Only Look Once) and SSD (Single Shot MultiBox Detector).
- 3. Feature Extraction and Matching:** OpenCV provides methods for extracting features from images, such as corners, edges, and descriptors like SIFT (Scale-Invariant Feature Transform) and SURF (Speeded-Up Robust Features). It also includes algorithms for matching and finding correspondences between features in different images.
- 4. Camera Calibration and 3D Reconstruction:** OpenCV enables camera calibration, which involves estimating camera parameters like focal length, distortion, and principal point. This information is crucial for tasks like 3D reconstruction, augmented reality, and camera pose estimation.

5. **Machine Learning Integration:** OpenCV integrates with machine learning frameworks like TensorFlow and PyTorch, allowing you to apply trained models for tasks like object recognition, image classification, and semantic segmentation.

6. **Real-Time Applications:** OpenCV is optimized for real-time applications and can leverage hardware acceleration, such as GPUs (Graphics Processing Units), to achieve high-speed processing and performance.

OpenCV is widely used in various domains, including robotics, computer vision research, augmented reality, surveillance, and autonomous vehicles. Its extensive collection of algorithms, combined with its open-source nature, has made it a popular choice for developers and researchers working in the field of computer vision and image processing.

6.1.5 MediaPipe

A media pipeline refers to a sequence of interconnected components or stages that process and manipulate media data, such as audio, video, or images, in a structured manner. It involves the flow of data through various processing steps to achieve a desired outcome or perform specific tasks. Media pipelines are commonly used in multimedia applications, streaming services, video processing, and other related domains. In a media pipeline, each stage or component performs a specific operation or transformation on the media data and passes it on to the next stage. The pipeline can be designed to handle real-time data streaming or batch processing, depending on the application requirements. Here are some common stages or components found in a media pipeline:

1. **Data Ingestion:** The media data is acquired from a source, such as a camera, microphone, file, or network stream. This stage involves capturing or reading the raw media data and converting it into a suitable format for further processing.

2. **Pre-processing:** The acquired media data may undergo pre-processing operations, such as noise reduction, filtering, resampling, or normalization. These operations aim to improve the quality of the data or prepare it for subsequent stages.

3. **Feature Extraction:** This stage involves extracting relevant features from the media data. For example, in image processing, it may involve detecting edges, extracting textures, or

identifying key points. In audio processing, it could involve extracting pitch, spectral features, or audio descriptors.

4. Analysis and Processing: The extracted features or data are analyzed and processed using algorithms or techniques specific to the application. This stage may involve tasks like object recognition, speech recognition, video encoding, compression, or any other domain-specific processing.

5. Post-processing and Enhancement: Once the core processing is completed, additional post-processing steps can be applied to enhance the media data. This may include noise removal, image/video enhancement, color correction, or any other improvement techniques.

6. Output and Presentation: The final processed media data is prepared for output or presentation. This could involve encoding the data into a specific format, packaging it for streaming or storage, or rendering it on a display or audio output device.

The specific components and operations in a media pipeline can vary greatly depending on the application and desired functionality. Designing an efficient and effective media pipeline requires considering factors such as data formats, computational requirements, latency constraints, and overall system performance.

6.1.6 PyCharm

PyCharm is an integrated development environment (IDE) used primarily for Python development. It is developed by JetBrains, a software development company known for creating powerful development tools. PyCharm provides a range of features and tools to enhance productivity and streamline the Python development process. Some key features of PyCharm include:

- 1. Code Editor:** PyCharm offers a robust code editor with syntax highlighting, code completion, and intelligent code analysis. It supports various Python versions and provides support for other languages like JavaScript, HTML, CSS, and more.
- 2. Debugging:** PyCharm allows developers to debug their Python code effectively. It provides breakpoints, stepping through code, variable inspection, and other debugging tools to identify and fix issues in the code.

3. **Integrated Terminal:** PyCharm includes an integrated terminal that allows developers to execute Python scripts and interact with the command line without leaving the IDE.
4. **Version Control:** PyCharm supports popular version control systems like Git, Mercurial, and Subversion. It provides integration with these systems, allowing developers to easily manage and track changes to their code.
5. **Testing:** PyCharm supports various testing frameworks, such as unittest, pytest, and doctest. It provides tools for running tests, viewing test results, and generating code coverage reports.
6. **Code Refactoring:** PyCharm offers powerful code refactoring capabilities, allowing developers to efficiently modify their code structure without introducing errors. It helps improve code quality, readability, and maintainability.
7. **Project Management:** PyCharm assists in managing Python projects by providing project templates, virtual environments, and package management tools. It helps organize and navigate through large codebases.
8. **Plugins and Extensions:** PyCharm supports a wide range of plugins and extensions that can be installed to extend its functionality. These plugins cover areas like frameworks, databases, web development, and more.

PyCharm is available in two editions: PyCharm Community Edition (free and open-source) and PyCharm Professional Edition (commercial, with additional features for web and enterprise development). Overall, PyCharm is a powerful IDE that can greatly enhance productivity and provide a comprehensive development environment for Python programmers.

6.1.7 Tensorflow

TensorFlow is an open-source machine learning framework developed by Google. It is designed to facilitate the development and deployment of machine learning models and neural networks. TensorFlow is widely used in various domains, including research, academia, and industry, for tasks such as image and speech recognition, natural language processing, and predictive analytics. Here are some key features and concepts associated with TensorFlow:

1. **Computational Graph:** TensorFlow uses a computational graph representation to define and execute complex computations. The graph consists of nodes that represent mathematical operations and edges that represent the flow of data (tensors) between these operations.

2. **Tensors:** Tensors are the fundamental data structures in TensorFlow. They are multi-dimensional arrays that can hold numeric data, such as scalars, vectors, matrices, or higher-dimensional arrays. Tensors flow through the computational graph, allowing operations to be performed on them.
3. **High-level APIs:** TensorFlow provides high-level APIs, such as Keras and Estimators, which simplify the process of building and training neural networks. Keras, in particular, is a user-friendly API that allows for rapid prototyping and supports various neural network architectures.
4. **Eager Execution:** TensorFlow supports eager execution, which enables immediate evaluation of operations without explicitly building and running a computational graph. Eager execution facilitates interactive debugging and quick experimentation.
5. **GPU and TPU Support:** TensorFlow leverages the computational power of GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units) to accelerate training and inference tasks. This allows for faster model training and better performance on large datasets.
6. **Model Deployment:** TensorFlow provides tools and libraries to deploy trained models in various environments, including mobile devices, web servers, and cloud platforms. TensorFlow Serving and TensorFlow Lite are examples of deployment options for different use cases.
7. **TensorFlow Hub:** TensorFlow Hub is a repository of pre-trained models and model components that can be easily integrated into TensorFlow projects. It offers a wide range of models across different domains, saving time and resources during model development.
8. **TensorFlow Extended (TFX):** TFX is a platform built on top of TensorFlow that focuses on end-to-end machine learning workflows. It provides components for data ingestion, preprocessing, model training, model validation, and serving in a scalable and production-ready manner.

6.1.8 Matplotlib

Matplotlib is a popular data visualization library for Python. It provides a wide range of tools and functions for creating high-quality plots, charts, and figures to visualize data in a visually appealing and informative way. Here are some key features and concepts associated with Matplotlib:

1. **Plotting Functions:** Matplotlib provides a variety of plotting functions that allow you to create different types of plots, such as line plots, scatter plots, bar plots, histograms, pie charts, and more. These functions can be used to represent data in 2D or 3D.
2. **Object-Oriented Interface:** Matplotlib can be used through a state-based interface or an object-oriented interface. The object-oriented interface allows for greater control and customization of plots by directly manipulating the plot objects and their properties.
3. **Figures and Axes:** In Matplotlib, a figure represents the entire plotting area or window, while axes represent individual plots or subplots within the figure. You can create multiple subplots within a single figure to organize and display different plots together.
4. **Customization and Styling:** Matplotlib provides extensive options for customizing and styling your plots. You can modify aspects such as line styles, colors, markers, labels, titles, legends, gridlines, background, and more. This allows you to tailor the appearance of your plots to meet specific requirements.
5. **Data Visualization Types:** Matplotlib supports a wide range of data visualization types, including line plots for representing trends and continuous data, scatter plots for visualizing relationships between variables, bar plots for comparing categorical data, histograms for displaying data distributions, and more.
6. **Support for LaTeX:** Matplotlib supports LaTeX expressions in text elements, including axis labels, titles, and annotations. This feature is useful for displaying mathematical symbols and equations in your plots.
7. **Saving and Exporting:** Matplotlib allows you to save your plots in various formats, such as PNG, JPEG, PDF, SVG, and more. You can also interact with plots in interactive environments, including Jupyter Notebook, by using the %matplotlib magic command.
8. **Integration with NumPy and Pandas:** Matplotlib integrates well with other popular Python libraries, such as NumPy and Pandas, which are commonly used for data manipulation and analysis. This integration enables seamless visualization of data stored in NumPy arrays or Pandas DataFrames.

Matplotlib is highly customizable and provides a rich set of features, making it a powerful tool for creating publication-quality visualizations. It is widely used in fields such as data science, scientific research, engineering, finance, and more. Additionally, Matplotlib has a large and active community that contributes to its development and provides a wide range of resources, tutorials, and examples to support users in creating compelling visualizations.

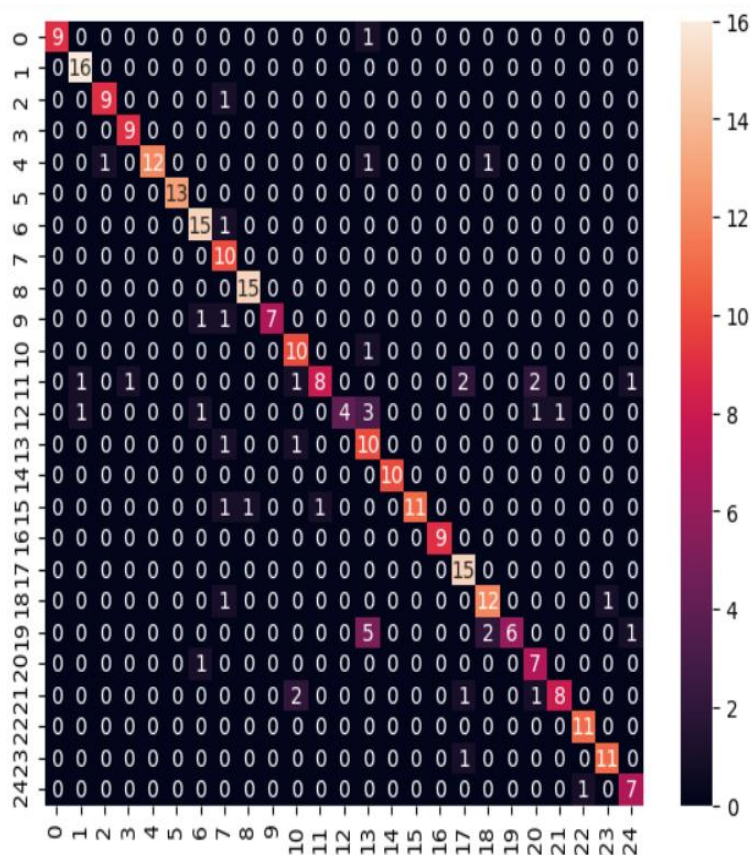
6.2 Results

Following are the results we got after training our model:

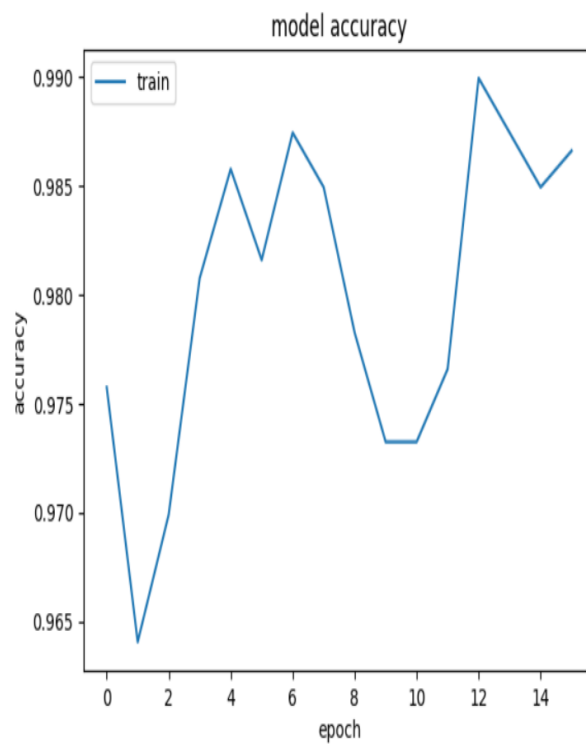
6.2.1 Classification Report

Classification Report				
	precision	recall	f1-score	support
0	1.00	0.90	0.95	10
1	0.89	1.00	0.94	16
2	0.90	0.90	0.90	10
3	0.90	1.00	0.95	9
4	1.00	0.80	0.89	15
5	1.00	1.00	1.00	13
6	0.83	0.94	0.88	16
7	0.62	1.00	0.77	10
8	0.94	1.00	0.97	15
9	1.00	0.78	0.88	9
10	0.71	0.91	0.80	11
11	0.89	0.50	0.64	16
12	1.00	0.36	0.53	11
13	0.48	0.83	0.61	12
14	1.00	1.00	1.00	10
15	1.00	0.79	0.88	14
16	1.00	1.00	1.00	9
17	0.79	1.00	0.88	15
18	0.80	0.86	0.83	14
19	1.00	0.43	0.60	14
20	0.64	0.88	0.74	8
21	0.89	0.67	0.76	12
22	0.92	1.00	0.96	11
23	0.92	0.92	0.92	12
24	0.78	0.88	0.82	8
accuracy			0.85	300
macro avg	0.88	0.85	0.84	300
weighted avg	0.88	0.85	0.84	300

6.2.2 Confusion Matrix



6.2.3 Graph



Chapter 7: Summary and Conclusion

Title	Outline
Summary and Conclusion	Summary
	Conclusion

7.1 Summary and Conclusion

7.1.1 Summary

A hand sign recognition system mobile app is a smartphone application that utilizes artificial intelligence and computer vision techniques to identify and interpret hand gestures or signs performed by the user. Here is a summary of a hand sign recognition system mobile app:

1. **User Interface:** The mobile app provides a user-friendly interface where users can interact with the system using their smartphone's camera.
2. **Image Acquisition:** The app captures real-time images using the smartphone's camera as input for hand gesture recognition.
3. **Hand Detection and Tracking:** The app applies hand detection and tracking algorithms to locate and track the user's hand in the captured images. This step ensures that the system focuses only on the hand region for gesture recognition.
4. **Feature Extraction:** Relevant features are extracted from the hand gestures captured by the app. These features may include hand shape, finger positions, motion, or any other distinguishing characteristics that differentiate different gestures.
5. **Gesture Classification:** The app employs machine learning or deep learning models to classify and recognize the hand gestures based on the extracted features. These models have been trained on a labeled dataset of hand gesture examples to accurately recognize different gestures.
6. **Real-Time Recognition and Feedback:** The app provides real-time feedback to the user by instantly recognizing and interpreting their hand gestures. This can be in the form of visual feedback on the screen.
7. **Custom Gestures and User Personalization:** The app may allow users to define and personalize their own custom gestures for specific actions or commands within the app. This customization enhances user experience and flexibility.
8. **Accuracy and Performance:** The app's performance is evaluated based on its accuracy in correctly recognizing hand gestures and providing timely feedback. Techniques like model

optimization and real-time optimization may be employed to ensure efficient processing and high recognition accuracy.

A hand sign recognition system mobile app offers users a convenient and intuitive way to interact with their smartphones, providing gesture-based control and functionality. It can be used for a variety of purposes, including accessibility for individuals with mobility impairments, gaming, virtual reality applications, or even as an alternative input method for certain tasks.

7.1.2 Conclusion

In conclusion, a hand sign recognition system, whether implemented as a standalone system or as a mobile app, utilizes artificial intelligence and computer vision techniques to recognize and interpret hand gestures or signs. It involves steps such as hand detection and tracking, feature extraction, gesture classification, and real-time feedback to accurately recognize and interpret user gestures. These systems offer a natural and intuitive way for users to interact with technology, enabling applications in sign language translation, human-computer interaction, accessibility, gaming, and more. With advancements in machine learning and mobile technology, hand sign recognition systems have become increasingly accurate, efficient, and accessible, providing users with a seamless and interactive experience.

Chapter 8: User Manual

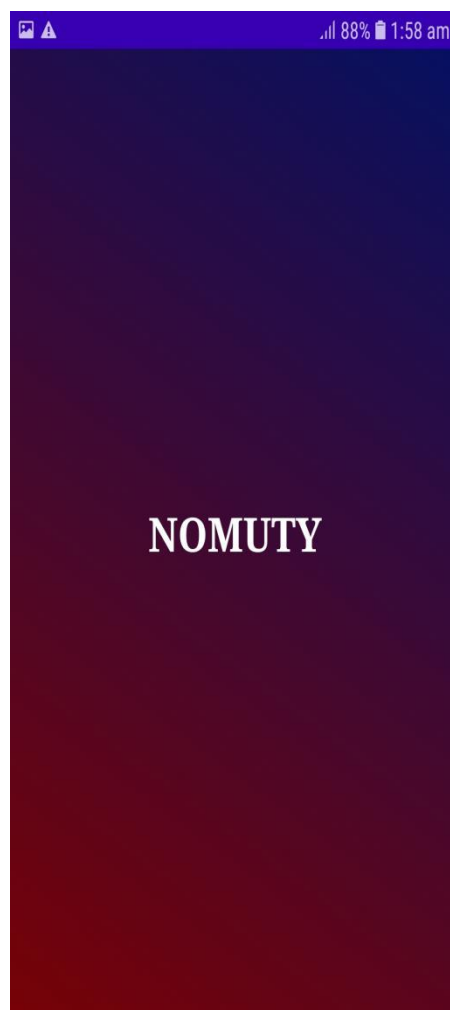
Title	Outline
User Manual	User Manual

8.1 User Manual

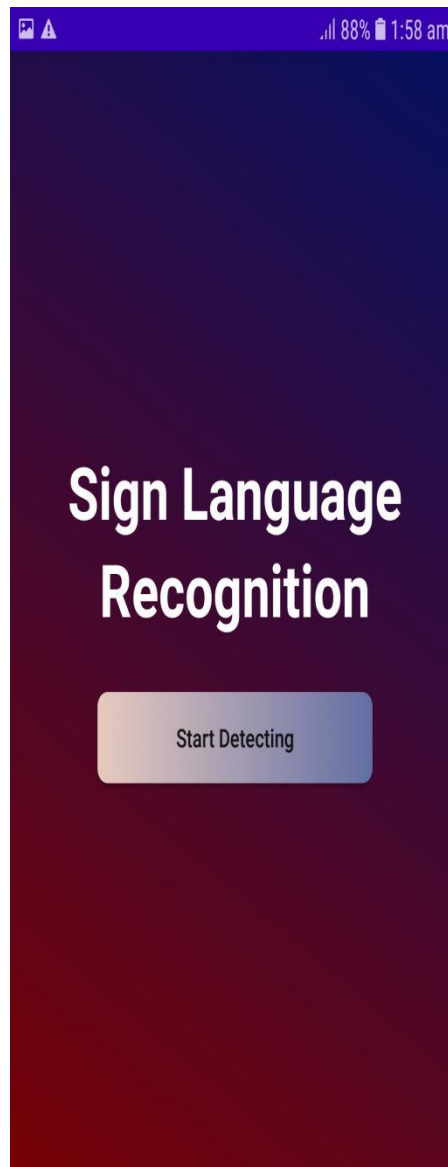
A user manual, also known as a user guide or instruction manual, is a document that provides detailed instructions, information, and guidance on how to operate or use a particular product, system, or software. It is designed to assist users in understanding the features, functions, and proper usage of the product or system.

The purpose of a user manual is to provide clear and comprehensive instructions to users, enabling them to effectively and safely use the product or system. It typically includes step-by-step procedures, diagrams, screenshots, and explanations to guide users through various tasks, settings, troubleshooting, and maintenance.

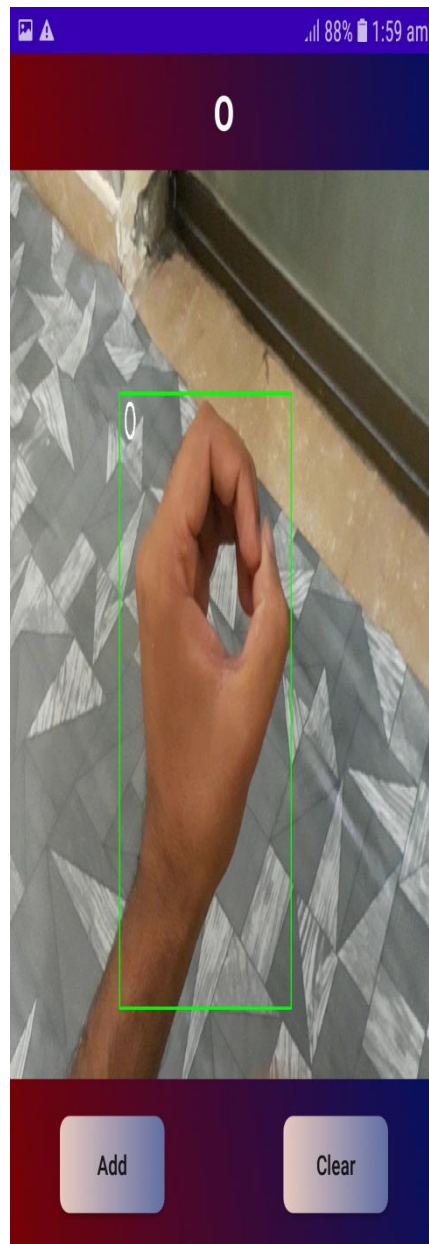
8.1.1 Title Page



8.1.2 Introduction



8.1.3 Main Page:



Chapter 9: Lessons Learnt and Future Enhancements

Title	Outline
Lessons Learnt and Feature Enhancements	Lessons Learnt
	Future Enhancements

9.1 Lessons Learnt

During the development and research of this project we learned about the fundamentals of Python, AI, object detection techniques, CNN (Convolutional Neural Network) DNN (Deep Neural Networks), Dataset Creation (Labeling and Annotations), training different AI models on datasets as well as performing different types of testing.

We also learned about basic grounds for research and development as well as official documentation procedure, dividing tasks among our team and managing our resources effectively.

9.2 Future Enhancements

While this project holds enormous potential and possibilities in the realm of AI. During the development of this project. Various feature and techniques were not possible to implement due to time, technologies and resources restrictions. Therefore, these features were listed for future upgrades of this project.

The list of such features/upgrades is as follows:

1. We can add voice feature that can convert textual data into voice
2. We can translate the text into different languages
3. Improving the user interface (UI) and interaction design can enhance the usability and user experience of the system. Intuitive UI elements, clear instructions, visual feedback, and customizable settings can make the app more user-friendly and accessible.
4. Continuously improving the accuracy of the hand sign recognition system is crucial. This can be achieved by refining the underlying machine learning or deep learning algorithms, training the system with more diverse and extensive datasets, and incorporating advanced techniques such as ensemble models or transfer learning.

9.3 References

[1] Shubham Kr. Mishra, Sheona Sinha, Sourabh Sinha, and Saurabh Bilgaiyan. Recognition of Hand Gestures and Conversion of Voice for Betterment of Deaf and Mute People.

[2] Chen, F., Fu, C., Huang, C.: Hand gesture recognition using a real-time tracking method and hidden Markov models. *Image Vis. Comput.* 21, 745–758 (2003)

[3] Coa, Z., Wei, Z., Zhang, G.: A no-reference sharpness metric based on the notion of relative blur for Gaussian blurred images. *J. Vis. Commun. Image Represent.* 25(7), 1763–1773 (2014)

[4] Shinde, S.S., Autee, R.M., Bhosale, V.K.: Real-time two-way communication approach for hearing impaired and dumb person based on image processing. In: *IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, pp. 1–5. IEEE (2016)

[5] Shi, D., Zhu, L., Cheng, Z., Li, Z., Zhang, H.: Unsupervised multi-view feature extraction with dynamic graph learning. *J. Vis. Commun. Image Represent.* 56, 256–264 (2018)

